

Data Engineering on Google Cloud — Day 2 of 2

Iowa Department of Management, Department of IT · October 6, 2026 · Student slide pack

- Slides follow the class agenda
- Hands-on activities: <https://roittraining.github.io/gcp-demos/bigquery.html>
- Labs are in the Google Skills classroom

Agenda — Tuesday, October 6 (1 of 2) · morning

09:00–09:15	Recap quiz — which GCP tool replaces which piece of your stack? (+ Day-1 spillover)
09:15–09:45	Module 7 — Intro to Modern Data Engineering: lake vs warehouse vs lakehouse
09:45–10:30	Module 8 — Building a Data Lakehouse (break 10:30–10:45)
10:45–11:15	Federated Query with BigQuery — the linked-server pattern, live
11:15–11:35	Exploring external tables — Bronze made tangible
11:35–12:00	Module 9 — Modernizing Data Warehouses with BigQuery & BigLake

QUIZ

LAB

DIN

Agenda — Tuesday, October 6 (2 of 2) · afternoon

12:00–13:00	Lunch	
13:00–13:45	Querying External Data & Apache Iceberg Tables — the Bronze-layer lab	LAB
13:45–14:00	Iceberg deep-dive + the BigLake table upgrade	DEMO
14:00–14:35	Module 10 — Governance + the Medallion: Google's Bronze/Silver/Gold next to yours (break 14:35–14:50)	
14:50–15:20	Module 11 — BQML + the Power BI last mile (compressed)	
15:20–16:30	Capstone workshop — map one of YOUR extract jobs; course summary; flex, Q&A & evaluations	WORKSHOP

QUIZ

Day-1 recap — which GCP tool replaces which piece of your stack?

15 minutes — 09:00–09:15 · Group warm-up — confer at your table, answer aloud; nothing scored

1. Nightly extract jobs → which service? SQL Server Agent → which scheduler? SSIS → what?
2. File drops → where do they land first, what loads them? Linked servers → the BigQuery way?
3. T-SQL procedures → where does that discipline live? SSAS cubes → the analogue?
4. The 60M-row race: CSV ~2 min vs Parquet seconds — the lesson? A query bills on what?
5. Bronze, Silver, Gold — one sentence each, in your target design

Why it matters — retrieval practice in your own vocabulary: name what each piece of your SQL Server stack becomes on Google Cloud — the map for the next two days.

M7 — What this block gets you

Lake, warehouse, lakehouse: the vocabulary of your own architecture doc.

- ✓ The industry words for what you are already building— lake, warehouse, lakehouse
- ✓ Why warehouses alone and lakes alone both fail— and why the lakehouse won
- ✓ Every term mapped to your target design— SQL Server DW → BigQuery, Bronze/Silver/Gold
- ✓ Why 'open' formats (Parquet, Iceberg) are your insurance against lock-in
- ✓ The deck's built-in quiz, run as a group— if you can answer it, you can defend the design doc

Day 2, first block — 09:00 to 10:30

09:00–09:15

Recap quiz — which GCP tool replaces which piece of your current stack?

QUIZ

09:15–09:45

M7 — Intro to modern DE: lake vs warehouse vs lakehouse (deck quiz pages at the end run as a group)

09:45–10:30

M8 — Building a data lakehouse: GCS, open formats, Iceberg, federated queries

10:30–10:45

Break

QUIZ

Which GCP tool replaces which piece of your current stack?

15 minutes · Whole room — confer at your table

1. Instructor reads each question aloud — 10 questions, no trick ones
2. Confer with your table for about 30 seconds per question
3. One person answers for the group — nothing is scored
4. Name, out loud, what each piece of your SQL Server stack becomes on Google Cloud
5. Jot your answers down — the key is on the next slide

Why it matters — retrieval practice before the lakehouse modules start — proving Day 1 stuck, in your own vocabulary, not Google's.

Quiz key — your stack, translated

The short version of yesterday; the deck quiz at the end of this module checks the new vocabulary.

- Nightly extract jobs— Datastream change data capture, replicating continuously
- SQL Server Agent— Cloud Scheduler; cheapest version is a BigQuery scheduled query
- SSIS packages— Dataform for SQL-first ELT; Dataflow only when SQL is not enough
- Staging file drops— land in Cloud Storage (Bronze), load via bq load, LOAD DATA, or DTS
- Linked servers— federated queries and external tables, no copy, always current
- SSAS cubes— materialized views and Gold marts, built and scheduled by Dataform

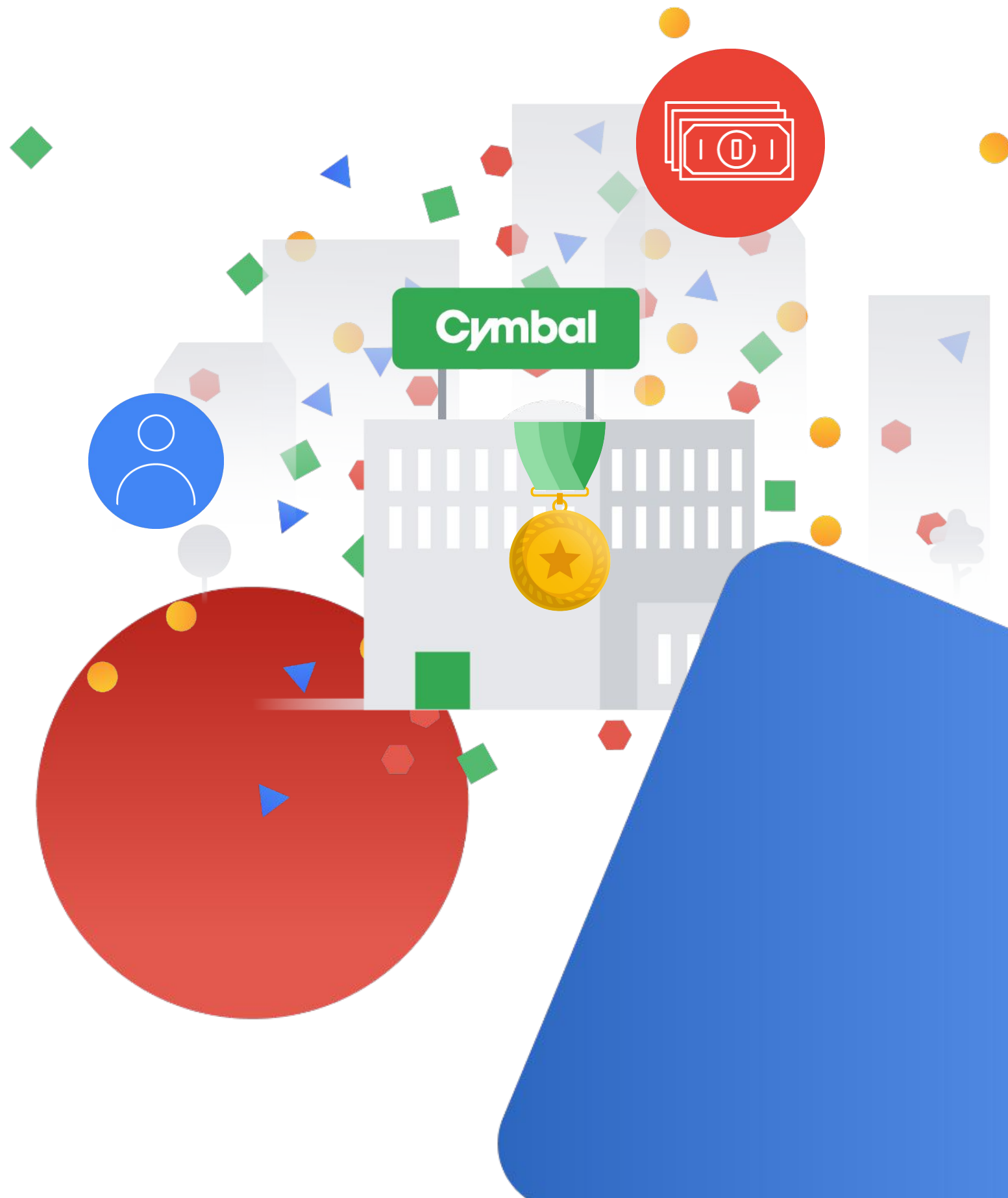
For years, organizations have relied on two primary methods for large-scale data storage: **data lakes and data warehouses.**

This is the vocabulary of your own architecture doc.

Frame for the next 30 minutes — lake, warehouse and lakehouse are the words in the DOM DoIT target design, not textbook terms.

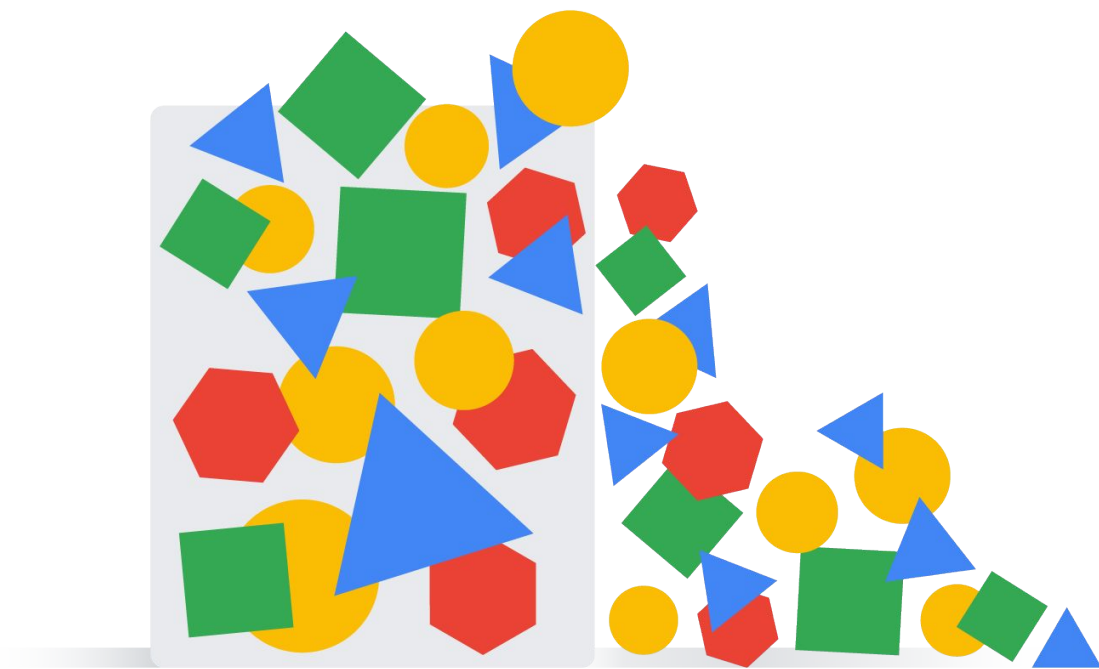
“

How are organizations, such as Cymbal (an e-commerce company), **managing extensive datasets** with data lakes and data warehouses?



Data lakes

Think of a data lake as a vast reservoir of data. It stores enormous amounts of raw data in its native format.



01

Structured data

02

Semistructured data

03

Unstructured data

Data lakes

01

Data lakes operate on the principle of "store everything now, figure out how to use it later."

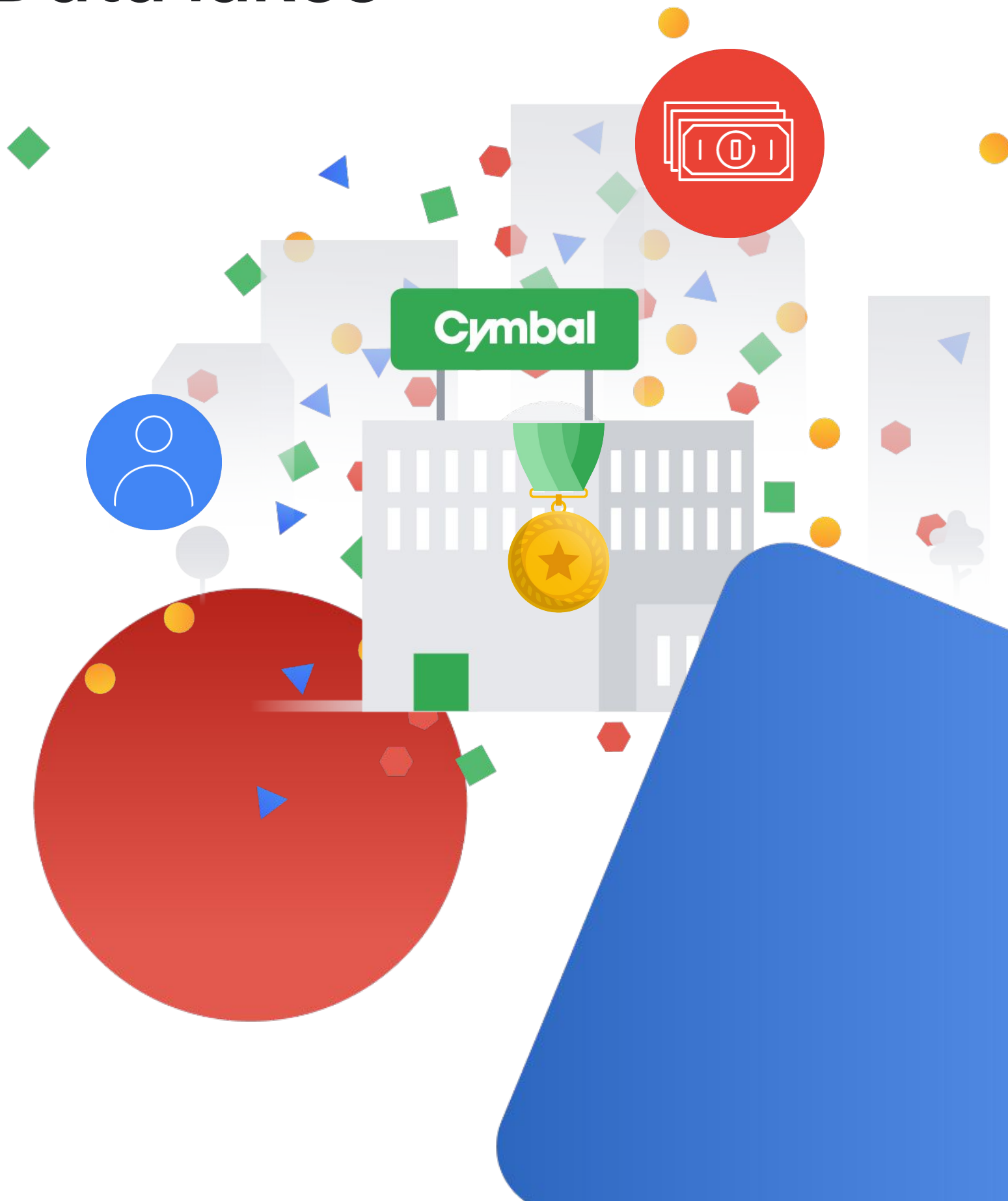
02

This approach applies a structure to data upon reading (schema-on-read), not during storage.

03

Data lakes are well suited for data exploration, machine learning (ML), and big data processing.

Data lakes



“

Data scientists need access to original, **unaltered clickstream data** to build a new recommendation engine.



Data lakes: Advantages

01

Flexibility: Stores all data types.

02

Agility: Fast to ingest data.

03

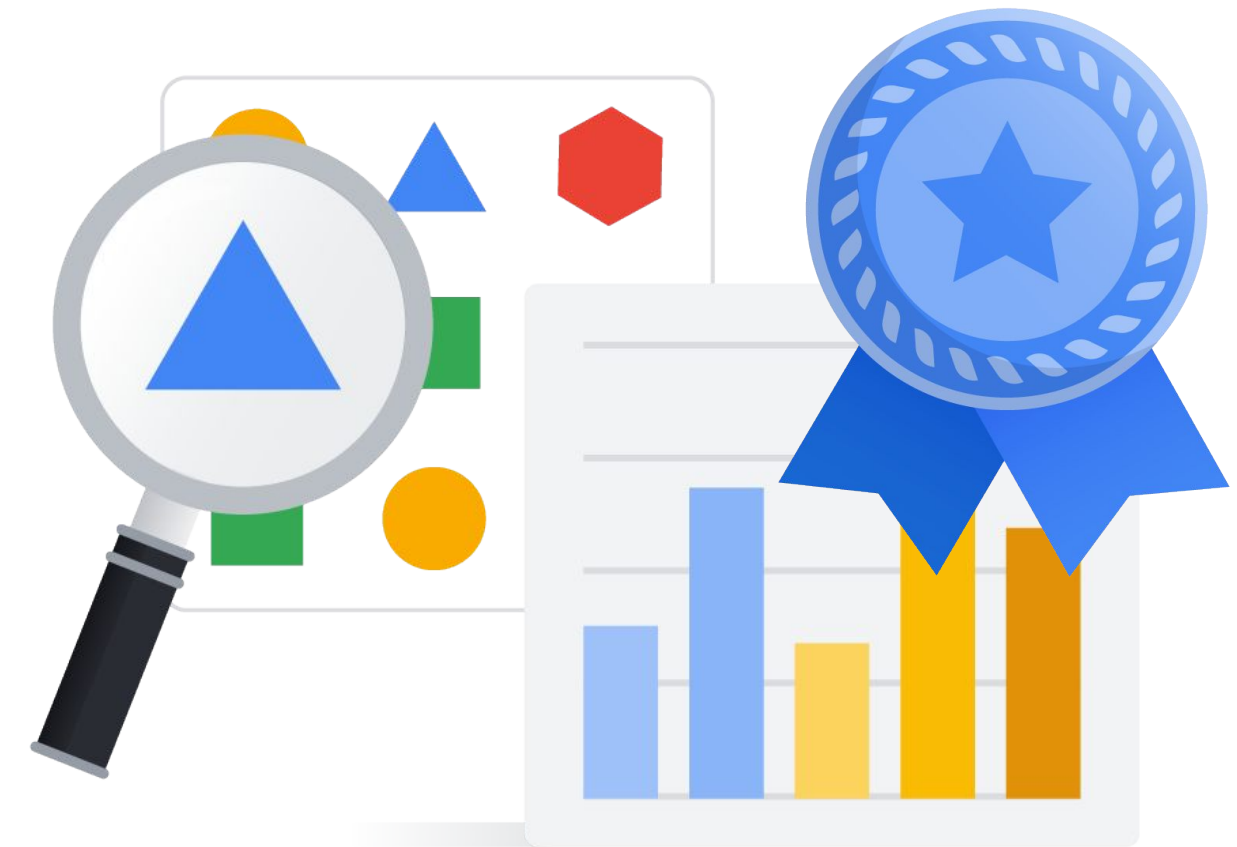
Scalability: Can grow to exabyte scale.

04

Cost-effectiveness: Uses low-cost object storage.

05

Support for advanced analytics: Ideal for AI/ML model training.



Data lakes: Disadvantages

01

Risk of becoming a "data swamp": Can become a disorganized collection of unusable data.

02

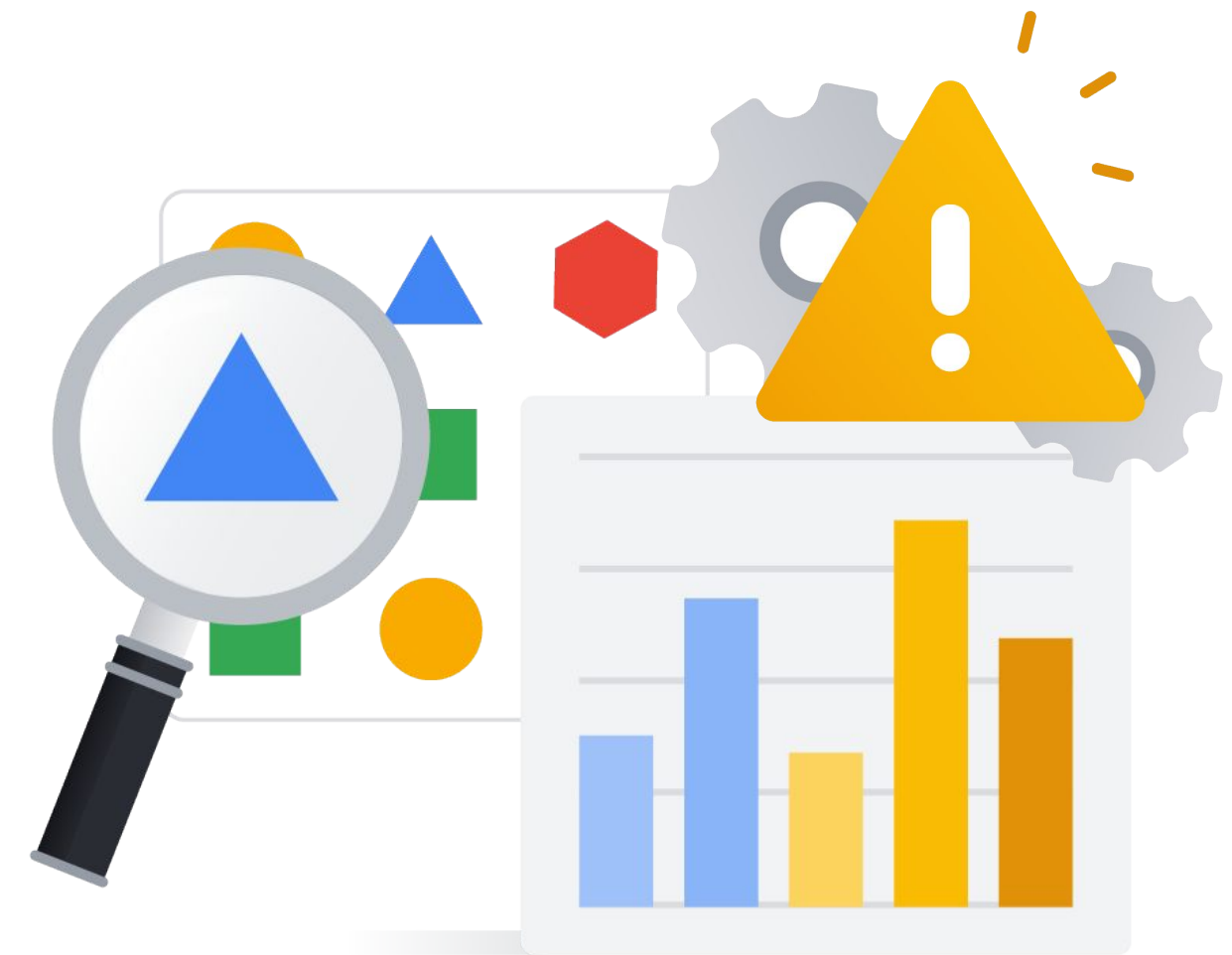
Management complexity: Requires significant overhead to maintain.

03

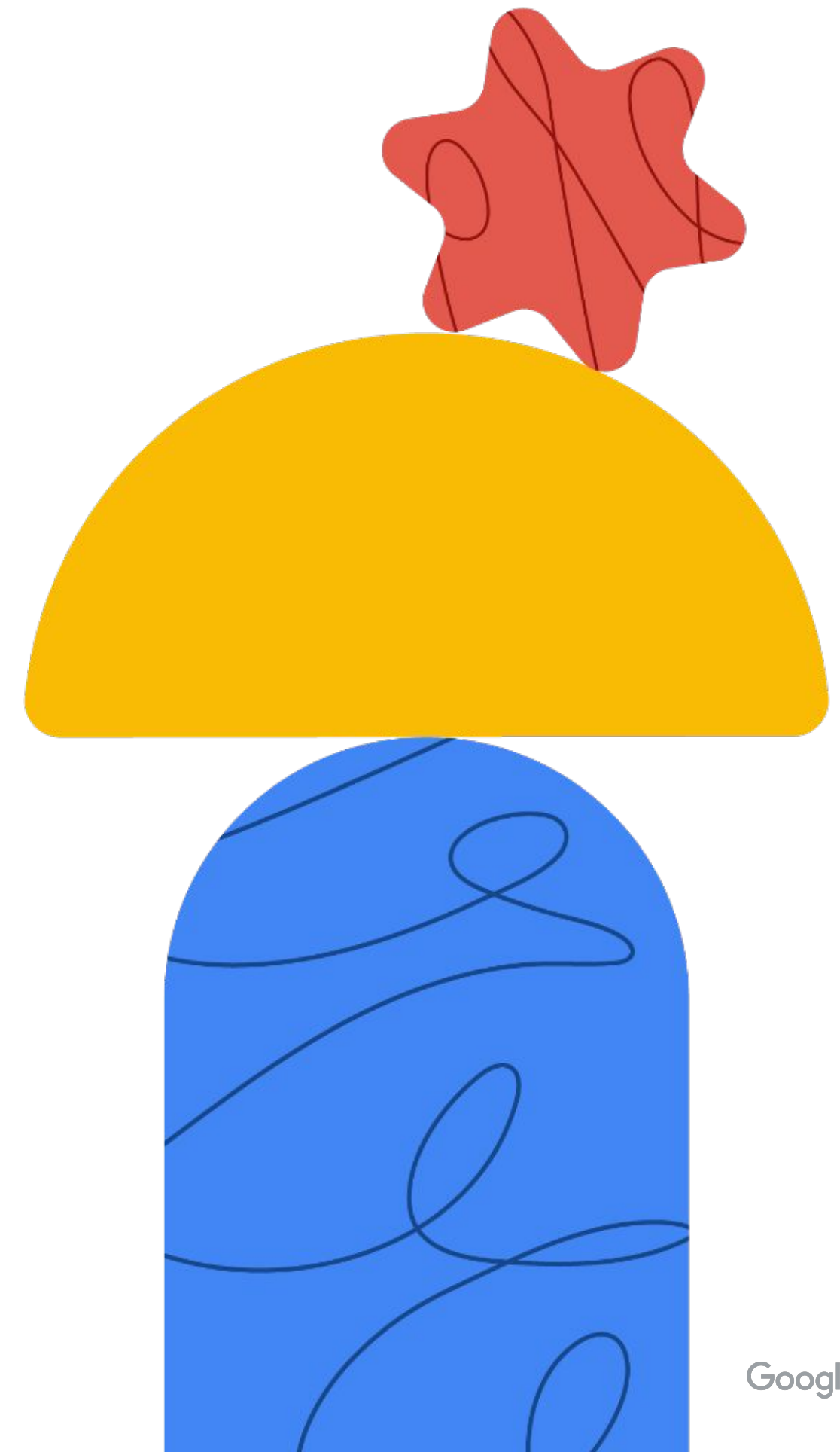
Time-consuming analysis: Data needs to be cleaned and wrangled before it is usable.

04

Security risks: Raw data formats can increase security and compliance risks.



A **data warehouse** is
like a highly organized
library.



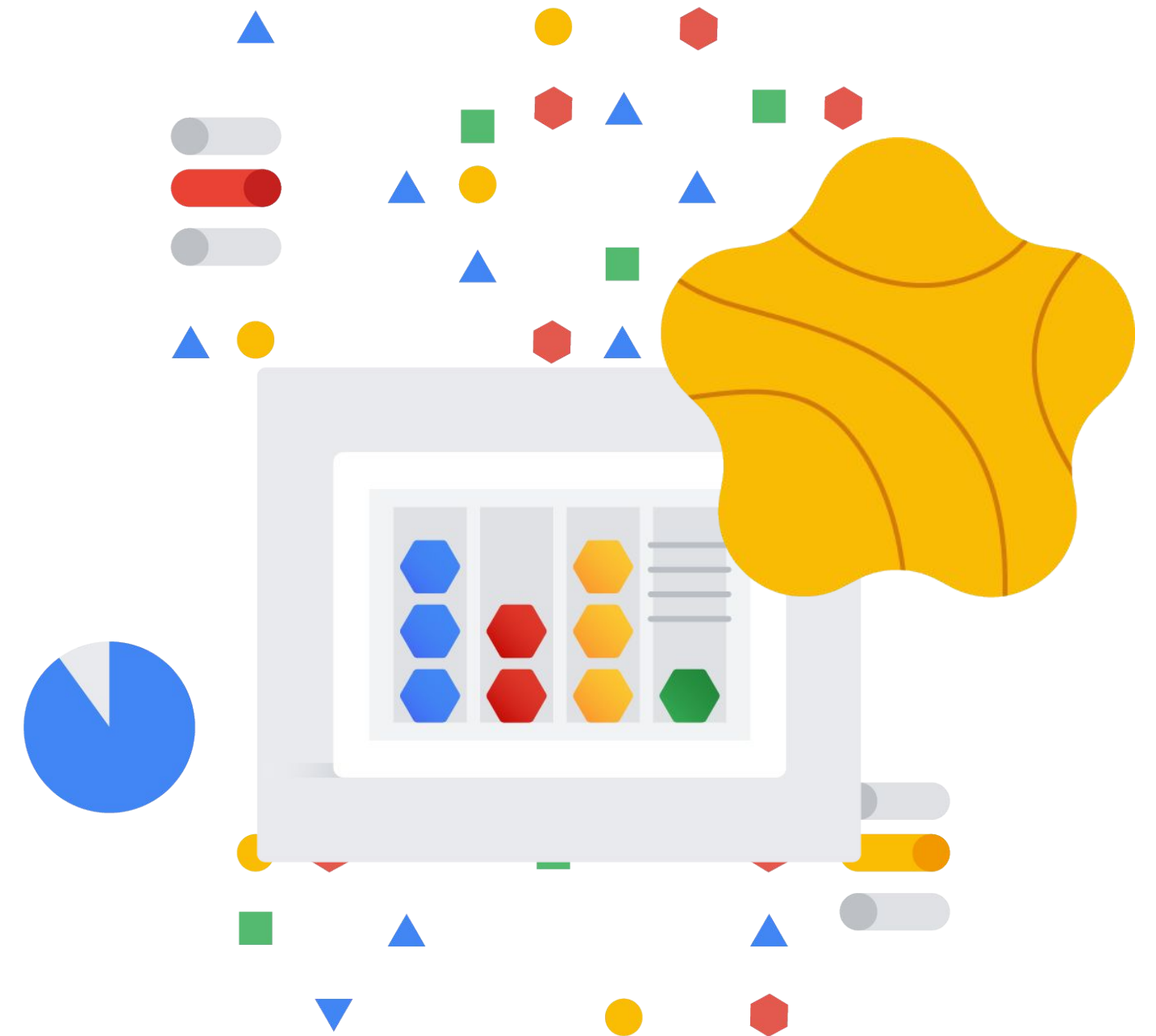
Data warehouses

01

Data is cleaned, transformed, and structured before storage.

02

This process, schema-on-write, optimizes data for analysis and business intelligence (BI).



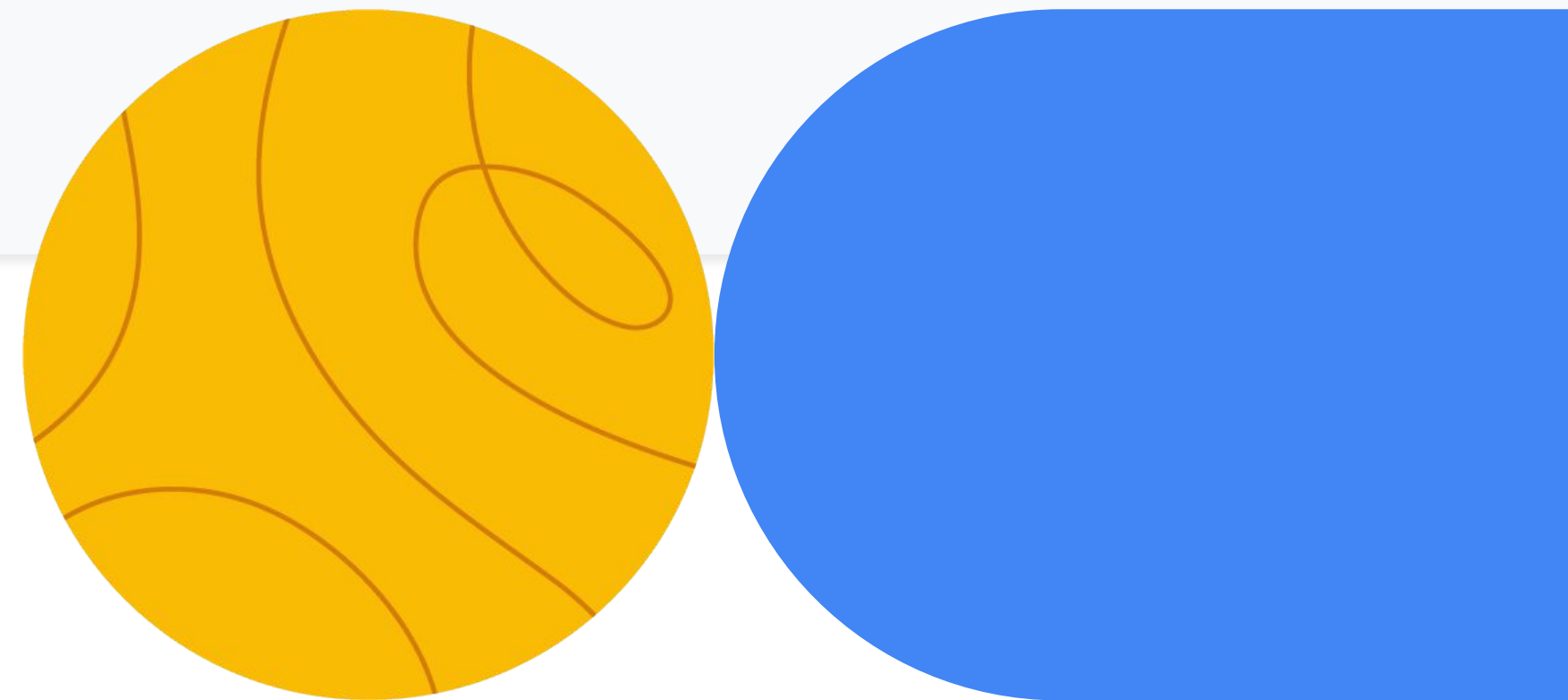
Data warehouses

Structure and quality first

The intended use of the data must be determined before the data warehouse is designed.

Cymbal example

- Key metrics, such as customer lifetime value and daily sales figures, are already defined.
- The data warehouse is then built to provide fast and accurate reports on these specific metrics.



Data warehouses are perfect for generating business reports and dashboards where **speed and accuracy are critical.**

Data warehouses: Advantages

01

Speed: Optimized for high-performance queries.

02

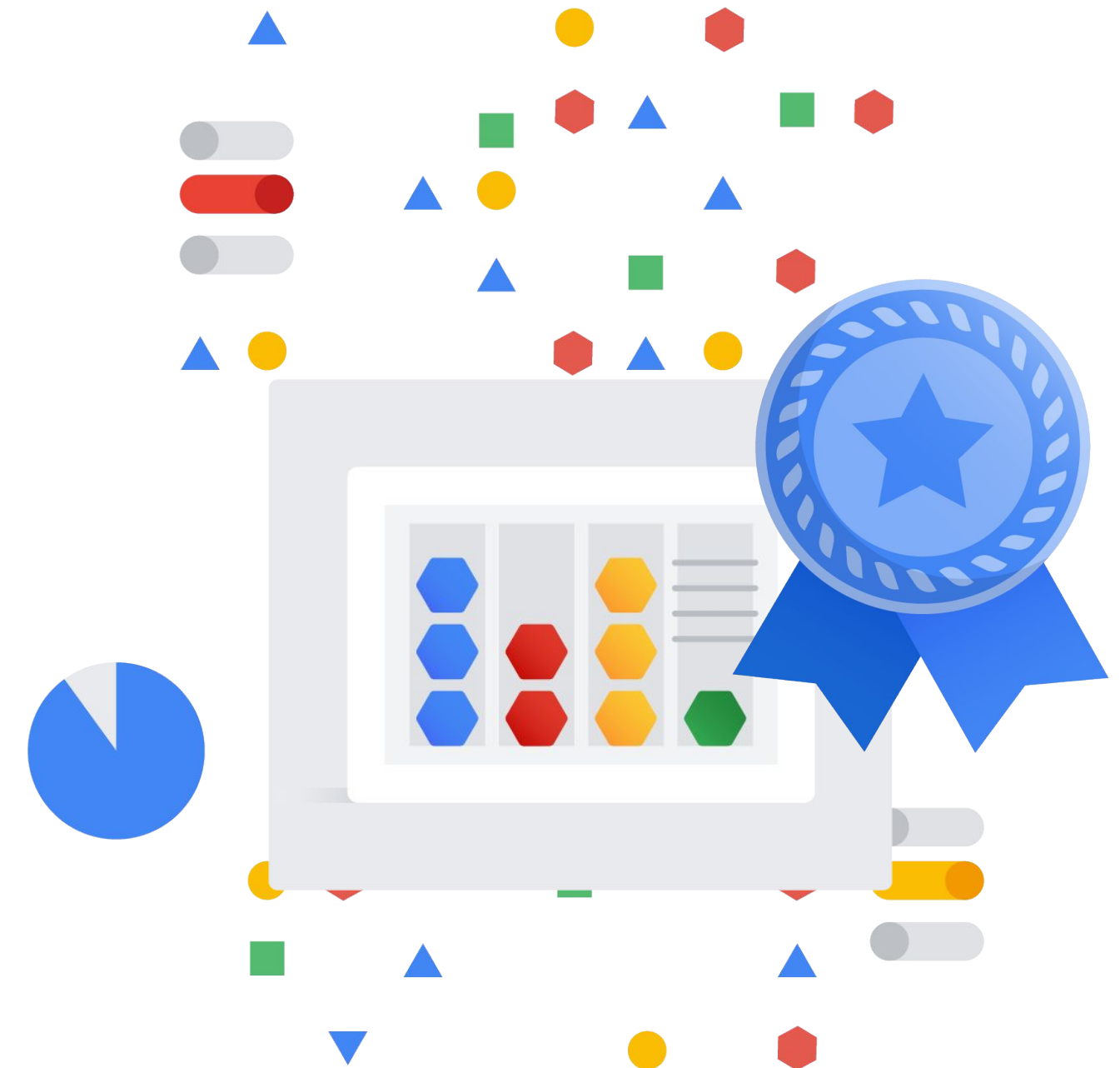
High-quality data: Provides consistent and reliable information.

03

Business focused: Directly answers common business questions.

04

Historical intelligence: Enables deep analysis of trends over time.



Data warehouses: Disadvantages

01

Inflexibility: Cannot easily accommodate new data types or unstructured data.

02

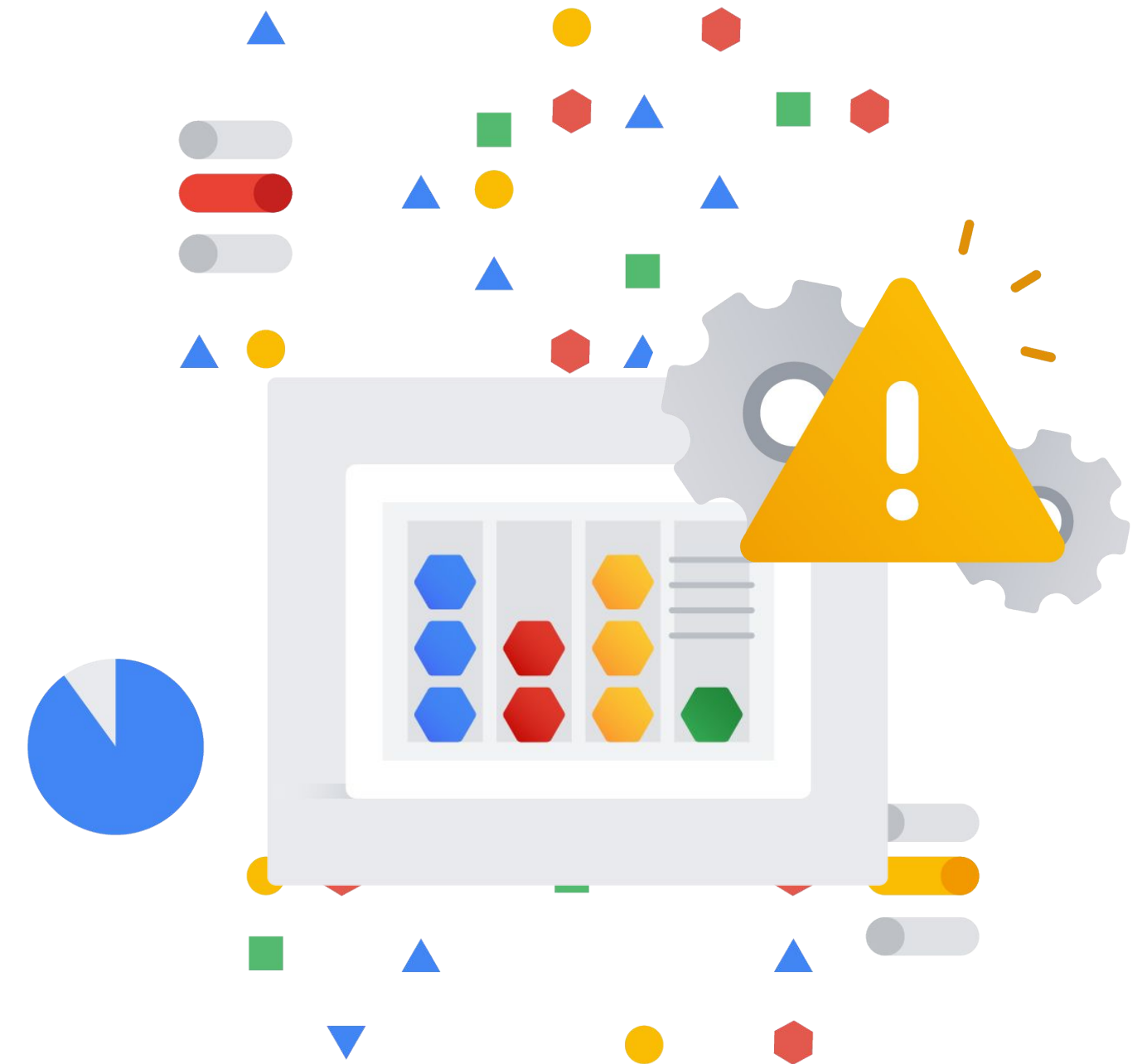
Cost: Often expensive to build and maintain.

03

Limited data types: Primarily designed for structured data only.

04

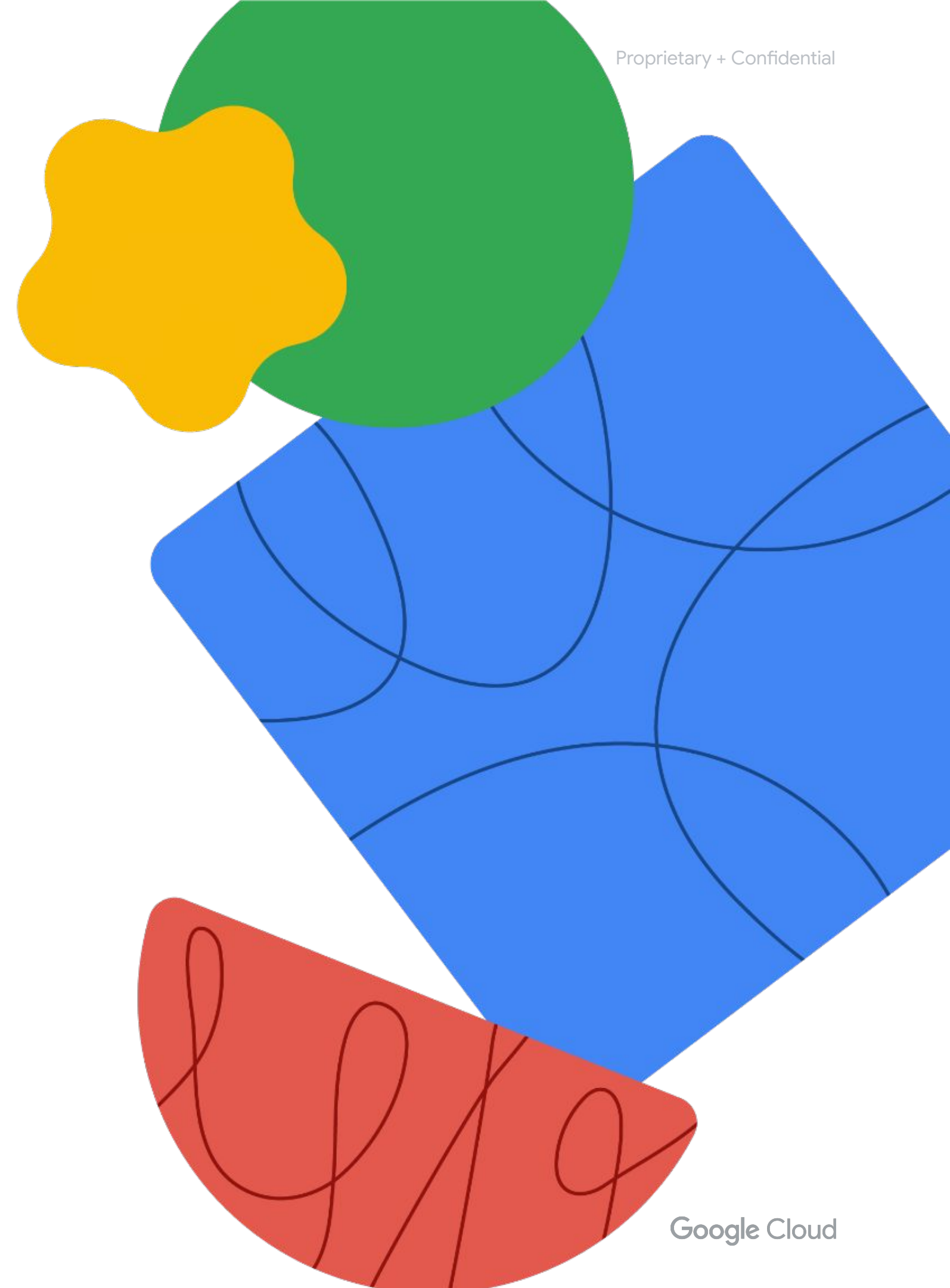
Long development time: Can take a long time to design and implement.



Neither data lakes or data warehouses excel at supporting the **full range of an organization's data needs.**

The modern approach: Data lakehouse

Combines the flexibility and low cost of a data lake with the speed and accuracy of a data warehouse.



Lake, warehouse, lakehouse — in your project's terms

Your target design IS a lakehouse design; this module gives you the words to defend it.

- The lake— Cloud Storage buckets; your Bronze layer, raw data landed as-is
- The warehouse— BigQuery; your Silver and Gold, cleaned and business-ready
- The lakehouse— the two acting as one system, BigQuery querying both
- Schema-on-write— your world today: everything modeled before it lands
- Schema-on-read— land raw in Bronze first, model into Silver/Gold when ready
- One engine owning the data— replaced by storage and compute scaled independently

Data lakehouse

01

Unified platform

Combines low-cost storage of a data lake with data warehouse management and query performance.

02

Versatile workloads

Supports traditional BI, modern data science, and AI without data movement or duplication.

03

Optimized architecture

Implements metadata and governance on open-format files in low-cost object storage (such as Google Cloud Storage) for a hybrid approach.

Data lakehouse

Data silo challenges

Cymbal's sales data was in a data warehouse and its customer review data was in a data lake. The two could not be analyzed together easily.

Integrated data analysis

With a data lakehouse, Cymbal can now run a single query to correlate customer sentiment from text reviews with sales trends from its transaction database.

Data lakehouse: Key features

01 Support for most data formats

02 Flexible schema-on-read or schema-on-write

03 Access for all types of data users

04 Cost flexibility based on needs

05 Unified data governance

06 ACID transaction support



Lakehouse technology

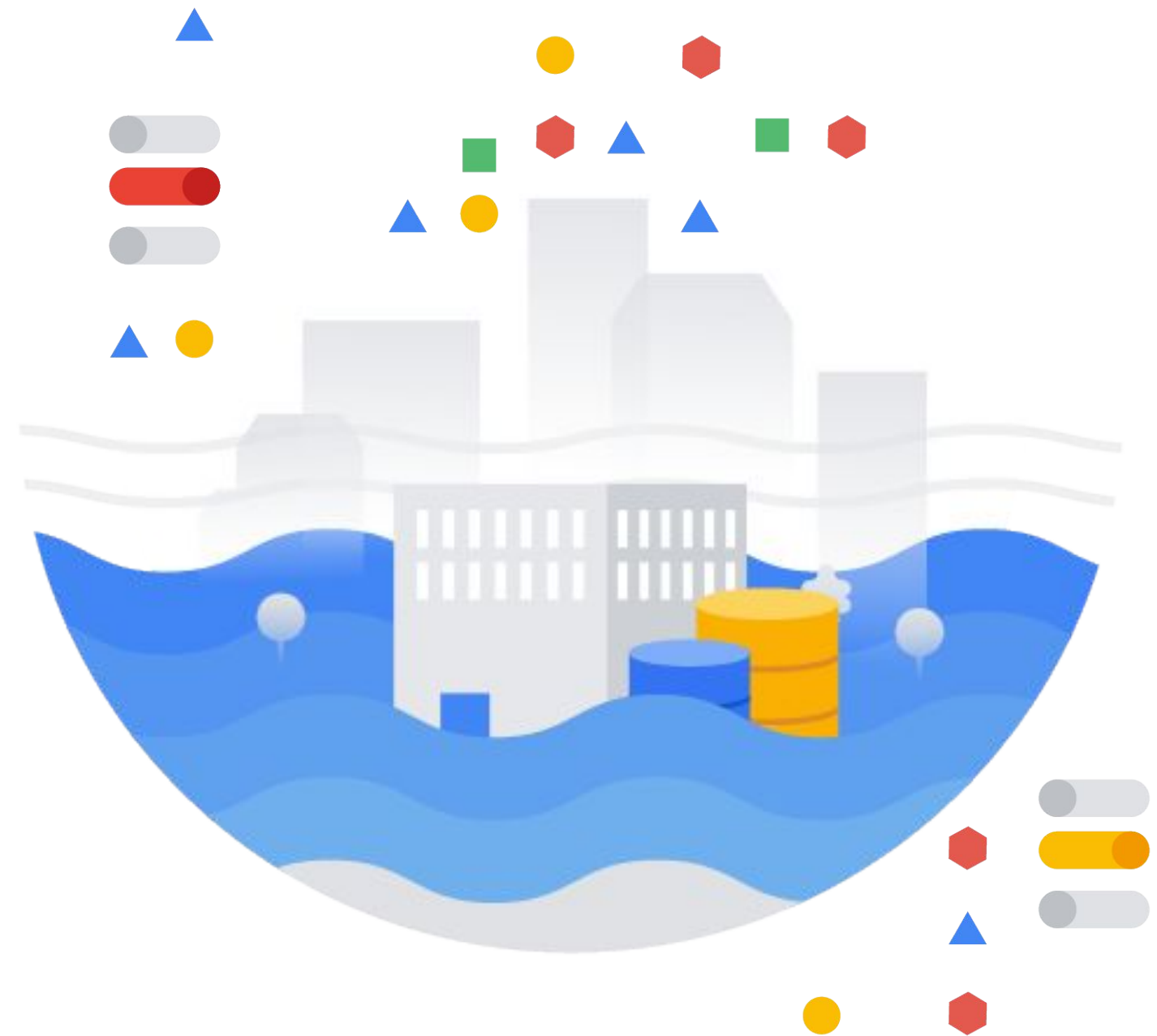
- Google Cloud uses Lakehouse to implement data lakehouses.
- Lakehouse extends Google Cloud's governance and query features to data in Google Cloud or other major clouds.
- Manage and query data in its original location using open formats, such as Parquet, ORC, and Avro.
- Utilize BigQuery's powerful query engine.

'Open' is the load-bearing word

- Open file formats— Parquet, ORC, Avro: no single engine owns your bytes
- Open table formats— Apache Iceberg adds ACID, schema evolution, time travel to files
- Your Bronze files stay readable by non-Google engines— insurance against lock-in
- Multiple engines— BigQuery, Spark, others — work safely on the same tables
- This is why the Bronze decision (Parquet on GCS, not CSV) was yesterday's real lesson

Data lakehouse benefits

- Unified data storage and access
- Reduced data redundancy
- Unified governance
- Broken-down data silos
- Improved flexibility and scalability



QUIZ

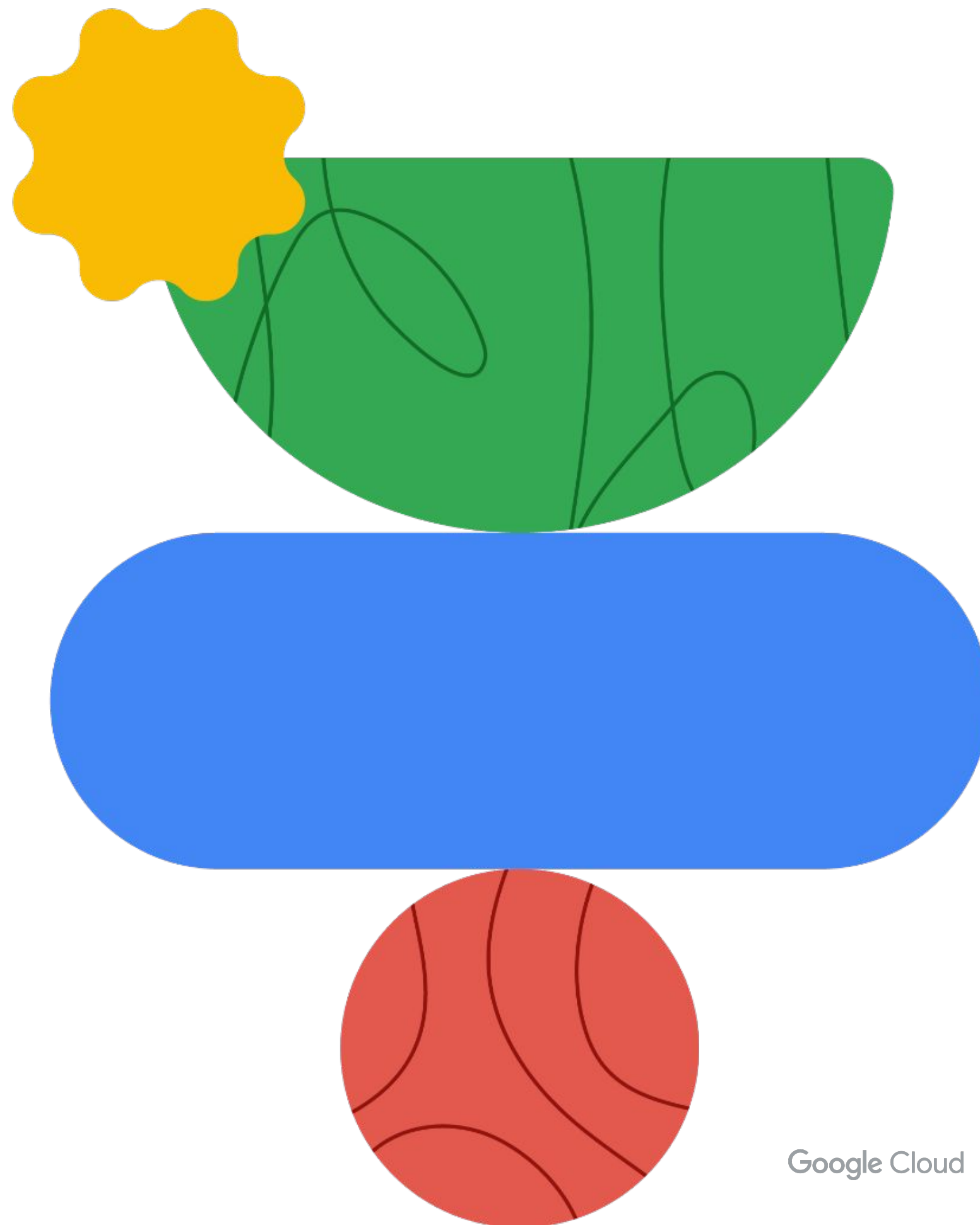
Built-in quiz — run it as a group

Last 5–8 minutes of the block · Whole room — the quiz pages that follow in this deck

1. Instructor shows each question page and reads it aloud
2. Confer at your table for about 30 seconds, then answer for the group
3. Treat wrong answers as vocabulary gaps to fix now, not as grades
4. Four questions with answer pages — lake, warehouse, lakehouse, and the Cymbal scenario

Why it matters — if you can answer this quiz out loud, you can narrate your own architecture doc to anyone who asks 'why not just a warehouse?'

Quiz



Quiz | Question 01

Question

Which of the following best describes the primary characteristic of a data lake?

- A. Data is cleaned, transformed, and structured before storage, optimized for BI.
- B. It is a highly organized library designed for fast and accurate reports based on predefined metrics.
- C. It stores enormous amounts of raw data in its native format with schema-on-read, ideal for exploration and ML.
- D. It primarily handles structured data and is optimized for high-performance queries on consistent information.

Quiz | Question 01

Answer

Which of the following best describes the primary characteristic of a data lake?

- A. Data is cleaned, transformed, and structured before storage, optimized for BI.
- B. It is a highly organized library designed for fast and accurate reports based on predefined metrics.
- C. It stores enormous amounts of raw data in its native format with schema-on-read, ideal for exploration and ML.
- D. It primarily handles structured data and is optimized for high-performance queries on consistent information.



Quiz | Question 02

Question

What is a key disadvantage of a traditional data warehouse?

- A. It risks becoming a "data swamp" without proper governance.
- B. It is inflexible and cannot easily accommodate new data types or unstructured data.
- C. It primarily uses low-cost object storage, leading to security risks with raw data.
- D. It is ideal for AI/ML model training due to its support for advanced analytics.

Quiz | Question 02

Answer

What is a key disadvantage of a traditional data warehouse?

- A. It risks becoming a "data swamp" without proper governance.
- B. It is inflexible and cannot easily accommodate new data types or unstructured data.
- C. It primarily uses low-cost object storage, leading to security risks with raw data.
- D. It is ideal for AI/ML model training due to its support for advanced analytics.



Quiz | Question 03

Question

How does a data lakehouse architecture combine the best features of data lakes and data warehouses?

- A. By enforcing a strict schema-on-write approach for all data types to ensure high data quality.
- B. By creating separate, specialized platforms for BI and AI workloads to maximize performance.
- C. By implementing a metadata and governance layer on top of open-format files in low-cost object storage.
- D. By always requiring data to be moved and duplicated between a data lake and a data warehouse.

Quiz | Question 03

Answer

How does a data lakehouse architecture combine the best features of data lakes and data warehouses?

- A. By enforcing a strict schema-on-write approach for all data types to ensure high data quality.
- B. By creating separate, specialized platforms for BI and AI workloads to maximize performance.
- C. By implementing a metadata and governance layer on top of open-format files in low-cost object storage.
- D. By always requiring data to be moved and duplicated between a data lake and a data warehouse.



Quiz | Question 04

Question

Cymbal, an e-commerce company, needs a solution to support traditional BI, modern data science, and AI workloads on a single, governed copy of its data while also breaking down data silos between sales and customer review data. Which architecture is best suited for this need?

- A. A data warehouse, such as BigQuery, for high-speed, interactive BI on structured data.
- B. A data lake, using Cloud Storage, for initial, low-cost storage of massive volumes of raw data for AI/ML research.
- C. A data lakehouse architecture with BigQuery and Lakehouse.
- D. Separate data lakes and data warehouses, with manual data integration as needed.

Quiz | Question 04

Question

Cymbal, an e-commerce company, needs a solution to support traditional BI, modern data science, and AI workloads on a single, governed copy of its data while also breaking down data silos between sales and customer review data. Which architecture is best suited for this need?

- A. A data warehouse, such as BigQuery, for high-speed, interactive BI on structured data.
- B. A data lake, using Cloud Storage, for initial, low-cost storage of massive volumes of raw data for AI/ML research.
- C. A data lakehouse architecture with BigQuery and Lakehouse.
- D. Separate data lakes and data warehouses, with manual data integration as needed.



M7 — What you now have

- ✓ The lake/warehouse/lakehouse vocabulary— and why the lakehouse collapses both silos into one platform
- ✓ Your design in those terms— GCS is the lake (Bronze), BigQuery the warehouse (Silver/Gold), one system
- ✓ The case for open formats— Parquet and Iceberg keep your Bronze free of lock-in
- ✓ The group quiz answered— vocabulary you can reuse when you defend the architecture doc
- ✓ Next: M8 builds the lakehouse itself— storage options, Iceberg, federated queries (09:45–10:30)

WATCH FIRST · 3:27 · GOOGLE CLOUD TECH

Bucket Options in Cloud Storage

- ✓ Naming, locations, storage classes — the three decisions you make once per bucket, and the foundation of your Bronze layer.
- ✓ Everything this morning lands in a bucket: Parquet files, Hive partitions, Iceberg tables. This is the vocabulary for all of it.



Scan to open on YouTube

youtu.be/8DMOJ6Lgm7s

M8 — What this block gets you

- ✓ GCS + open formats (Parquet, Iceberg)— the Bronze foundation in your own architecture doc
- ✓ Apache Iceberg— warehouse behavior (ACID, schema evolution, time travel) on raw files in a bucket
- ✓ BigQuery as the one SQL engine over Bronze, Silver and Gold— managed tables and lake files alike
- ✓ Federated queries— your linked-server / OPENQUERY pattern, from BigQuery, without moving data
- ✓ Hands-on: the federated-query lab, then external tables on Hive-partitioned Parquet— Bronze made tangible

This block and the hands-on morning

09:45–10:30 M8 — Building a data lakehouse: GCS, open formats, Iceberg, federated queries

10:30–10:45 Break

10:45–11:15 Lab: Federated Query with BigQuery — live operational data, no copy

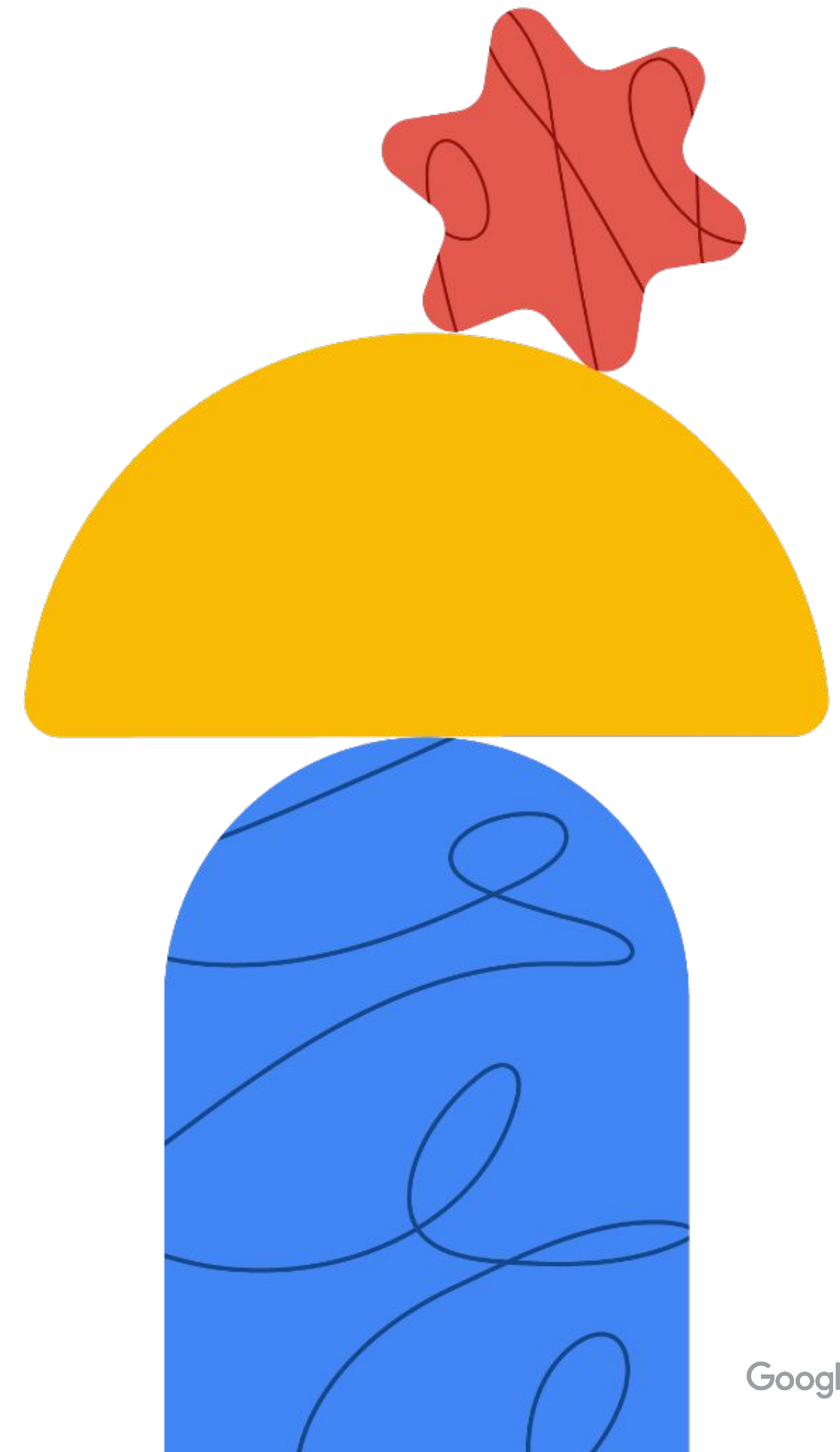
LAB

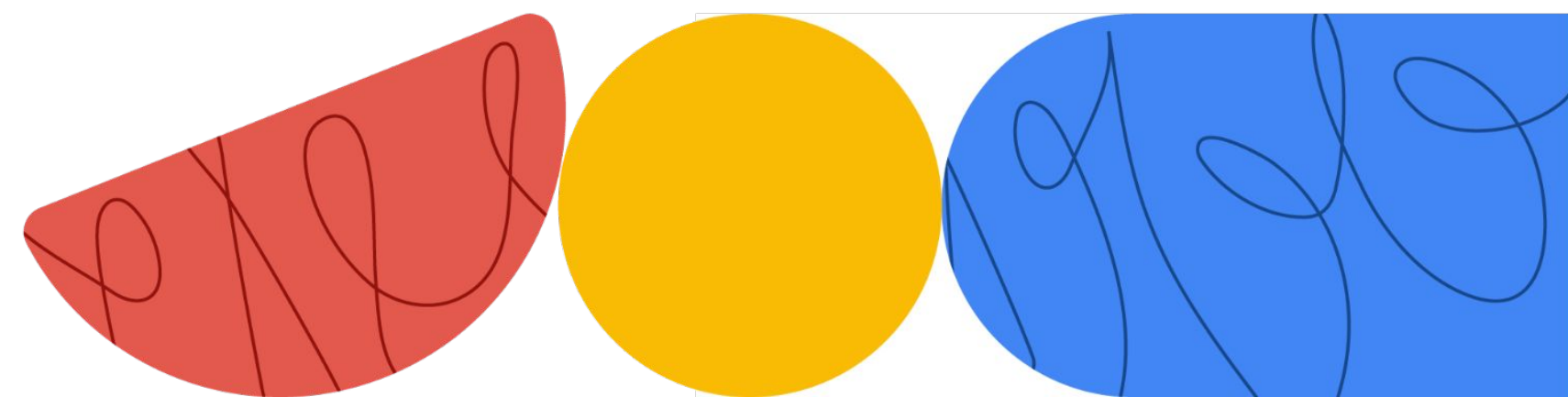
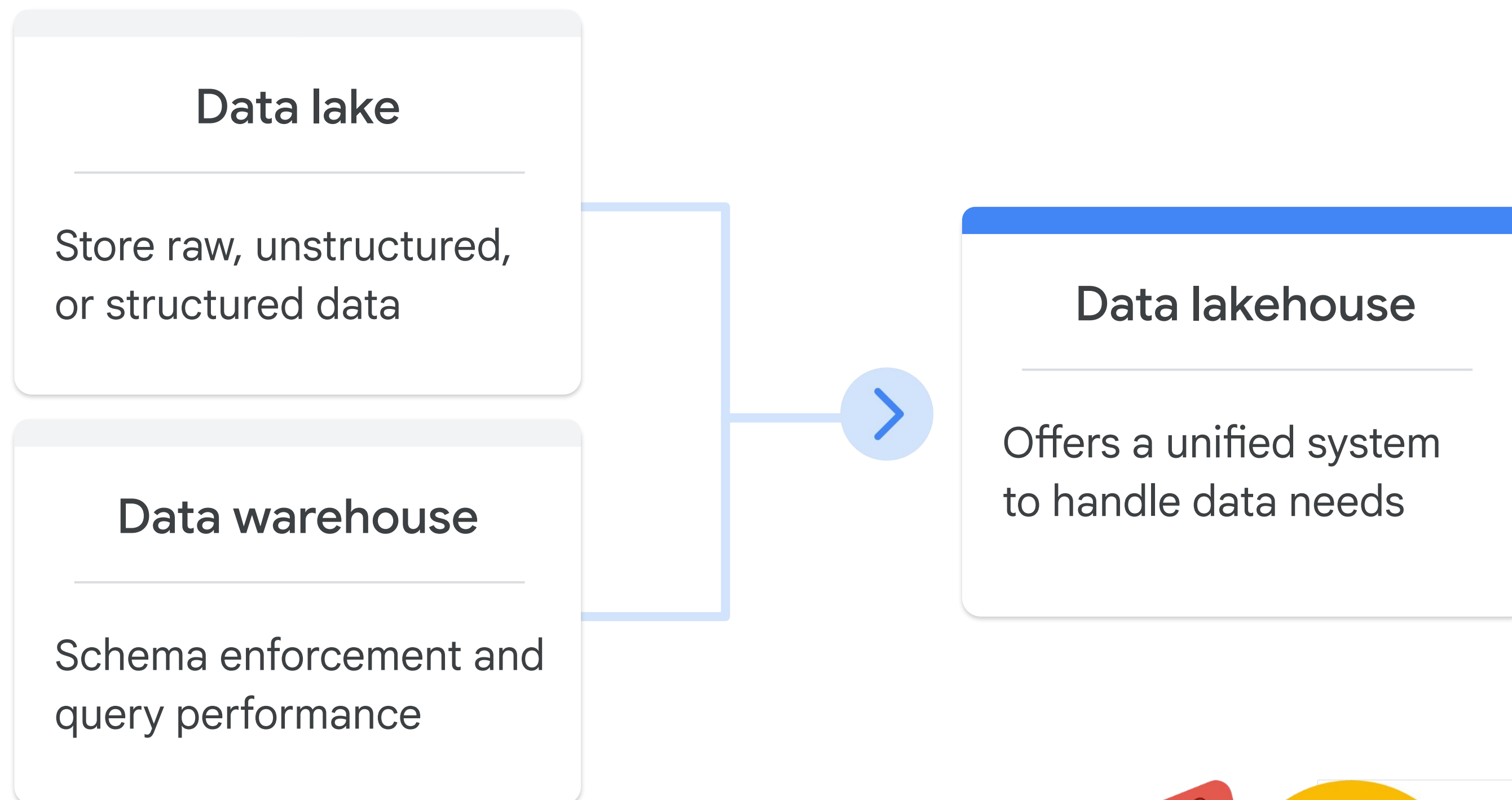
11:15–11:35 DIN: Exploring external tables — Hive-partitioned Parquet on GCS

DIN

11:35–12:00 M9 — Modernizing data warehouses with BigQuery and BigLake

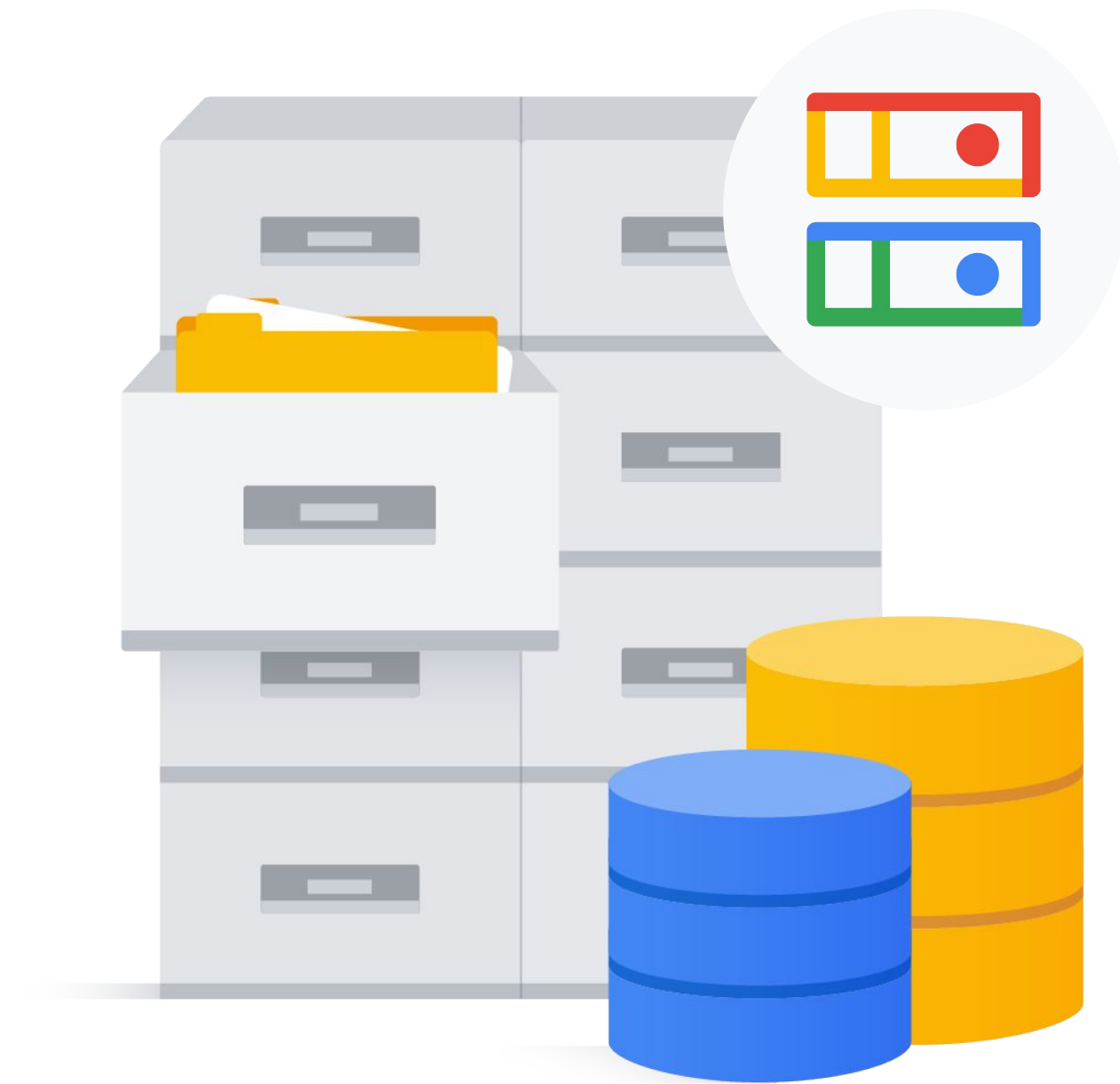
Data storage for data lakehouses on Google Cloud





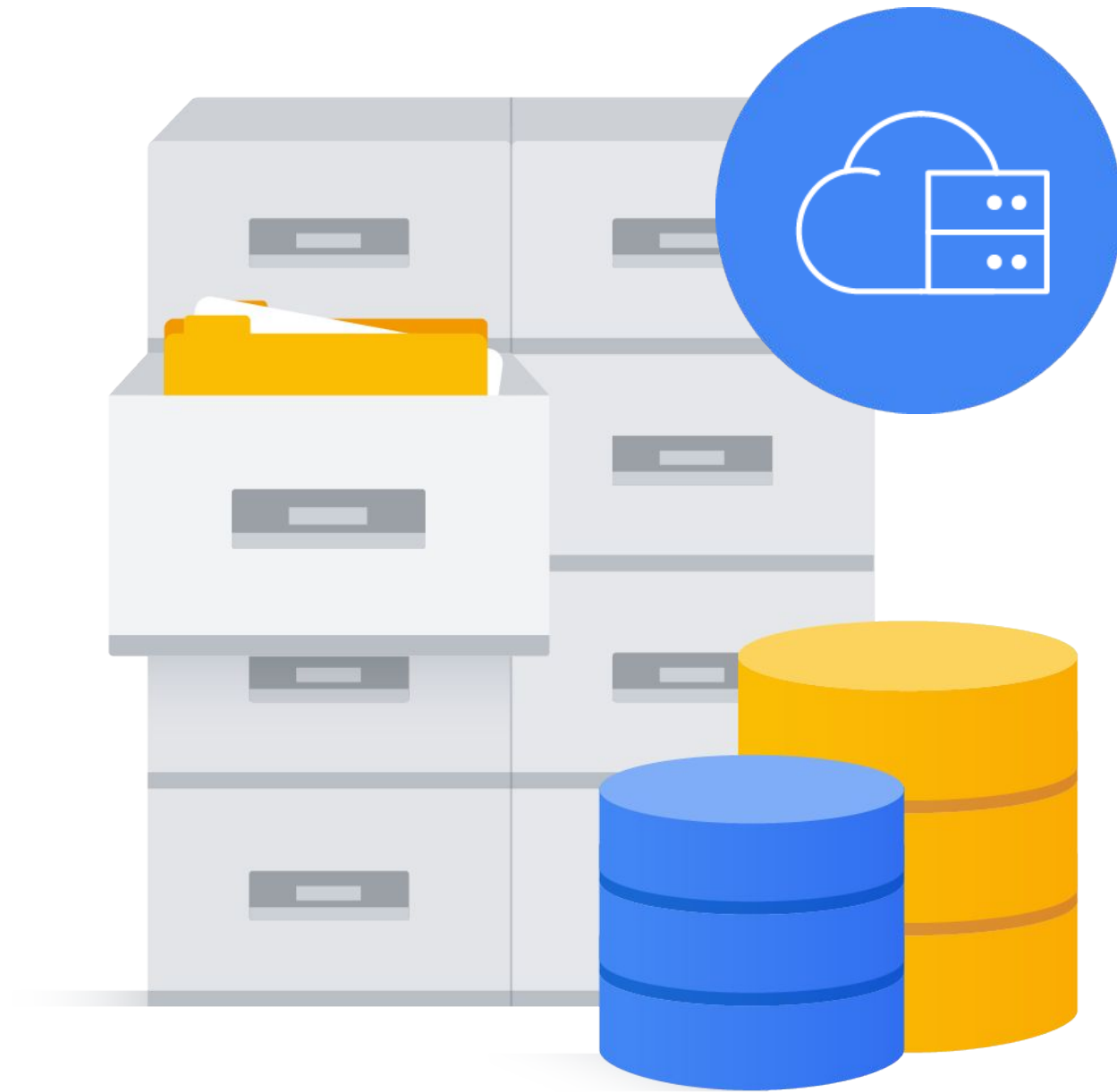
Cloud Storage

A massively scalable, highly durable, and cost-effective object storage service.



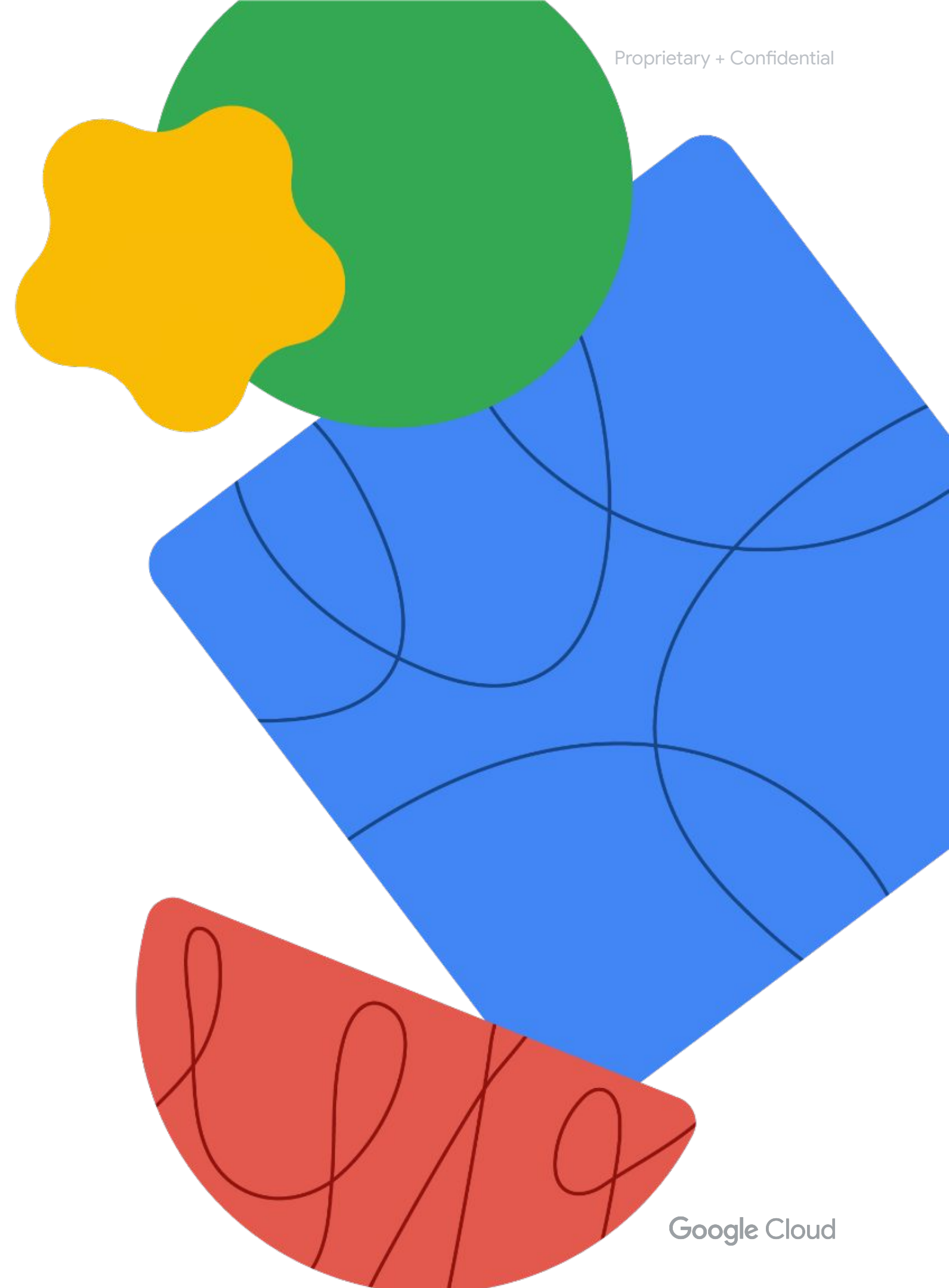
Other storage options

If your strategy is multicloud, you can keep your data in another major provider's cloud storage.



Introduction to Apache Iceberg open table format

A data lakehouse needs structure and performance. Open table formats, such as Apache Iceberg provide this.

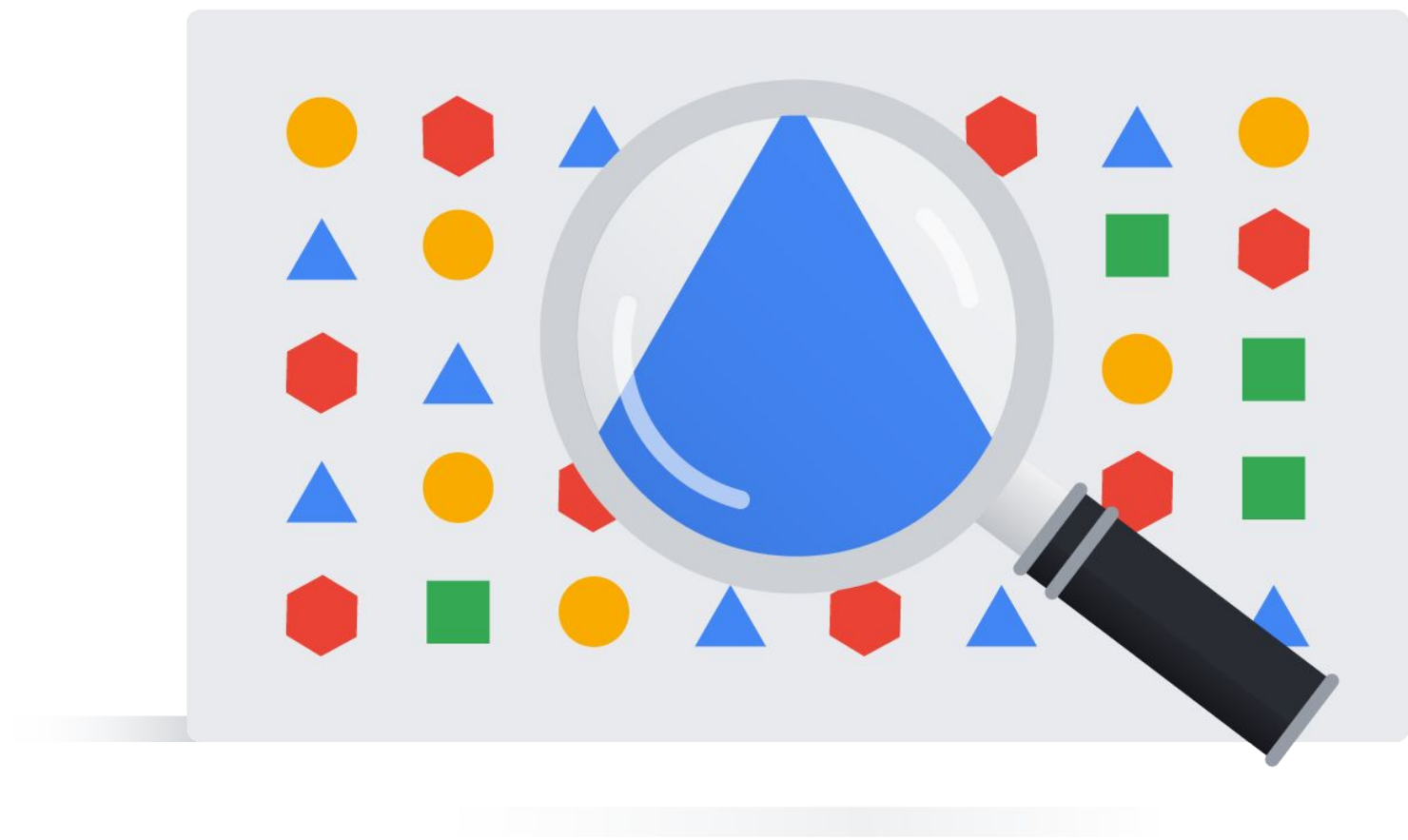


Efficient querying with Apache Iceberg (1 of 2)

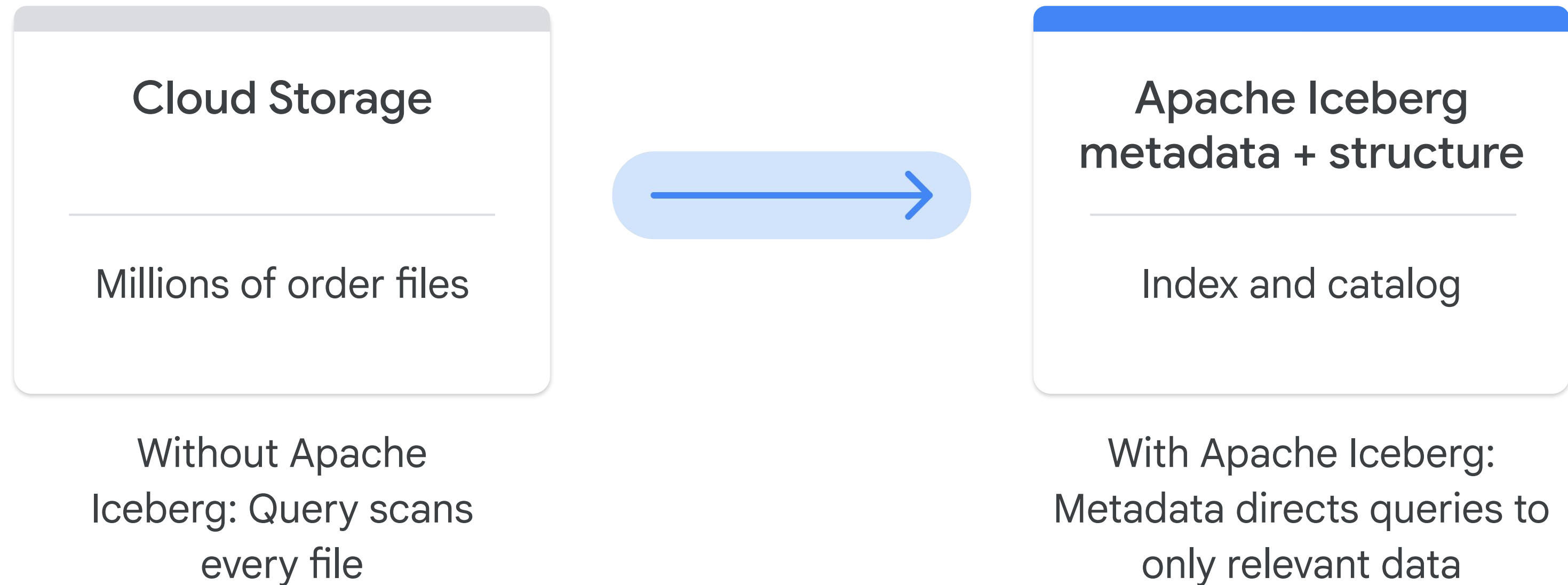
Cloud Storage

Millions of order files

Without Apache
Iceberg: Query scans
every file



Efficient querying with Apache Iceberg (2 of 2)



What does Apache Iceberg provide?

01

Schema evolution

Allows schema changes (such as adding and renaming columns) without impacting existing data or applications.

02

Hidden partitioning

Automatically manages how data is partitioned for efficient querying.

03

Time travel

Enables querying past data versions for auditing and debugging, allowing users to reproduce exact data states from previous points in time.

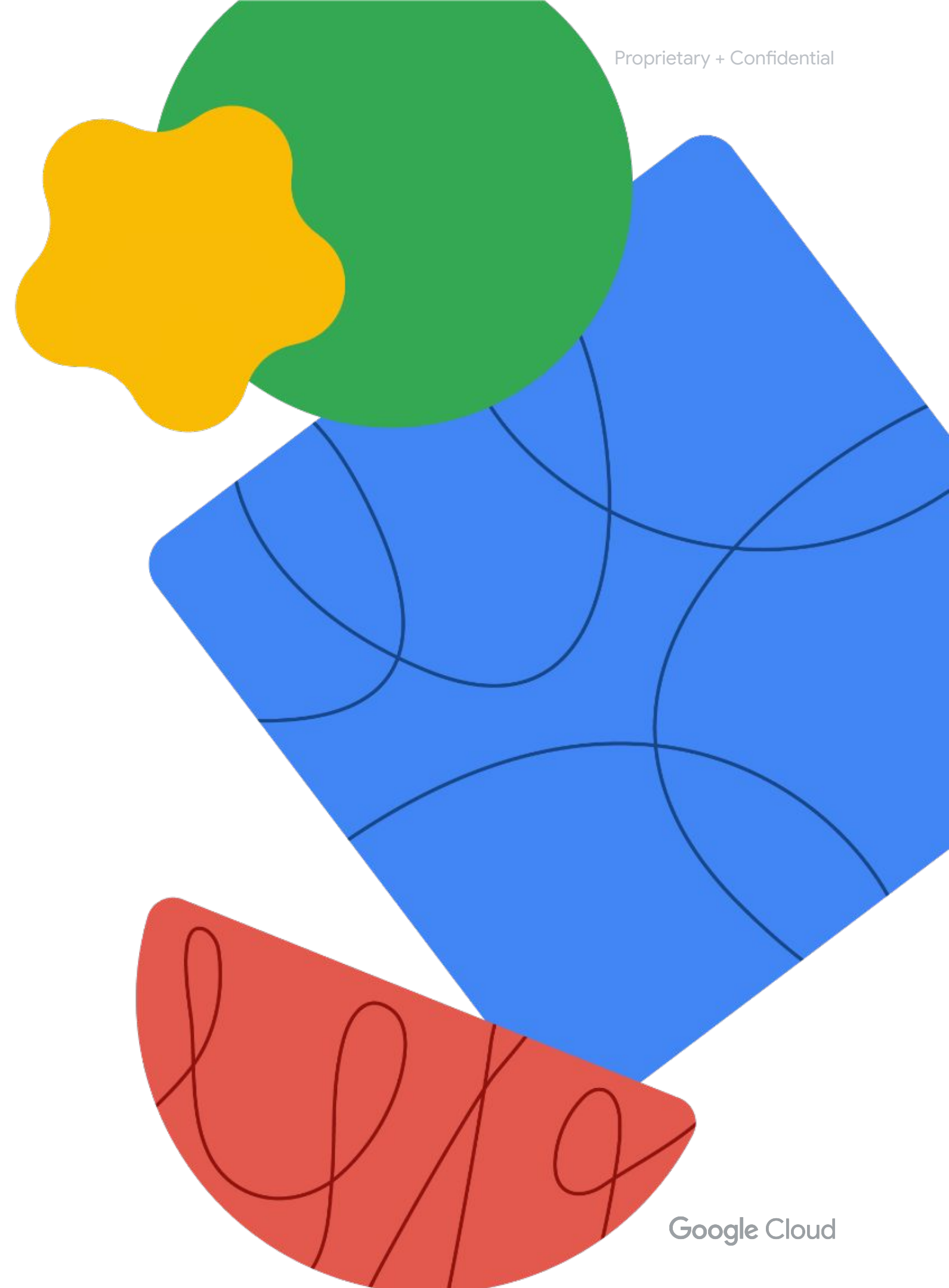
04

Atomic transactions

Enable reliable, concurrent data operations, preventing corruption when multiple processes read/write to the same tables.

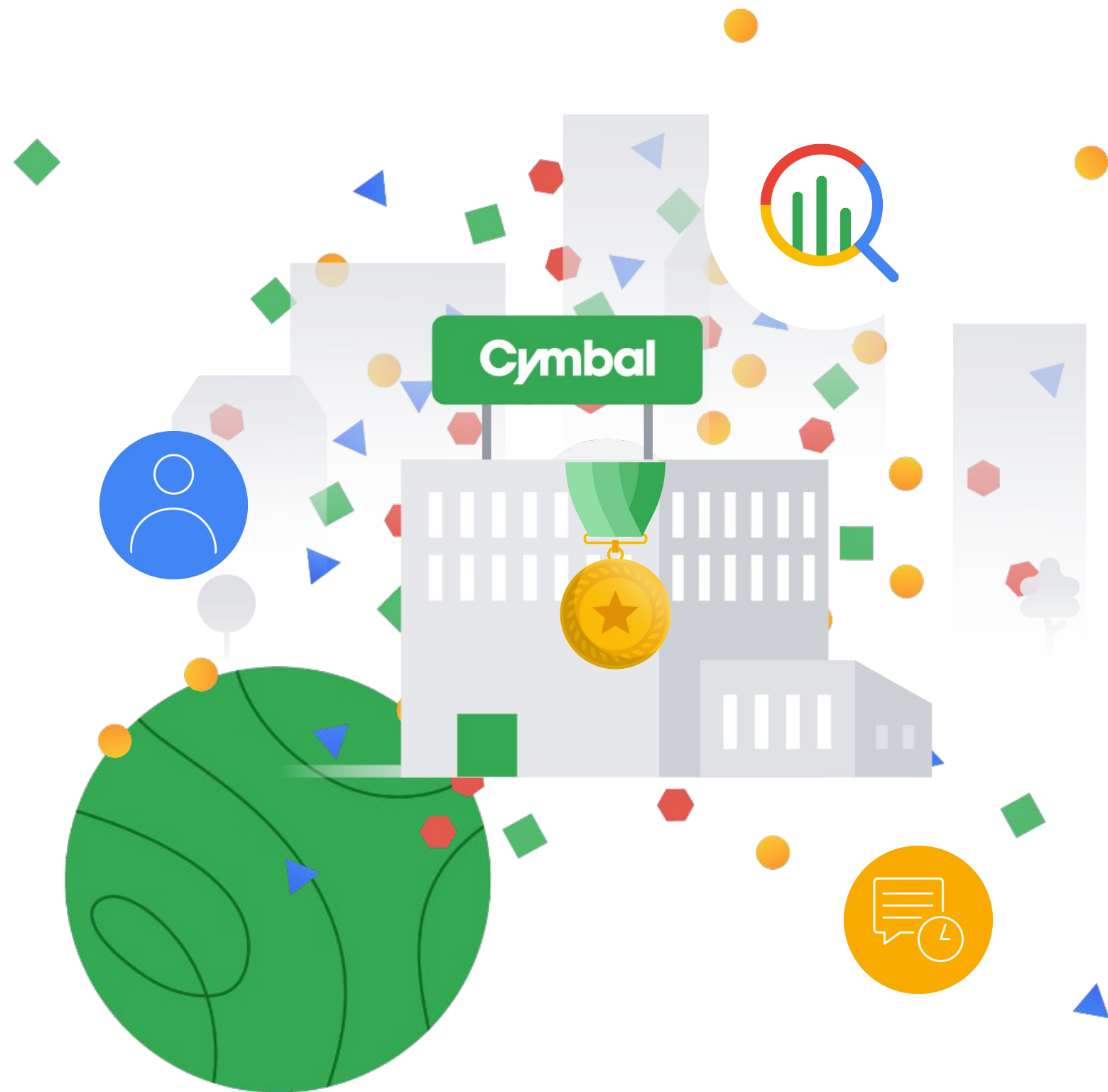
Using Apache Iceberg with Cloud Storage

Treat a collection of data files as performant, queryable tables, bringing the benefits of a data warehouse to a flexible data lake.



GCS + open formats = your Bronze foundation

- Today— county and vendor file drops sit unqueryable until an SSIS load runs
- Bronze target— Parquet files in a GCS bucket, Hive-partitioned by date, queryable immediately
- Iceberg on top— ACID, schema evolution, time travel over those same files, multi-engine readable
- Silver and Gold— BigQuery-managed tables, built by Dataform ELT from Bronze (this afternoon)
- One SQL surface— BigQuery reads managed tables and lake files alike; no staging hop
- AlloyDB— operational/OLTP sibling (managed PostgreSQL); not your path, one line only



BigQuery as the central processing engine

Whereas Cloud Storage and Apache Iceberg provide a flexible, open-standard foundation for storing vast amounts of raw and structured data, BigQuery is the high-performance engine that activates it.

BigQuery

Seamless data querying with BigQuery and Lakehouse

By creating Lakehouse tables, Cymbal can use BigQuery's familiar SQL interface to directly and securely query the Apache Iceberg-formatted data in their Cloud Storage data lake.

Eliminating data duplication and ETL processes

No need to duplicate data or perform costly extract, transform, load (ETL) processes to run analytics.

Enhanced analytics performance for data lake

Analysts get the performance and features of a premier data warehouse while querying data directly in their open-format data lake.

BigQuery

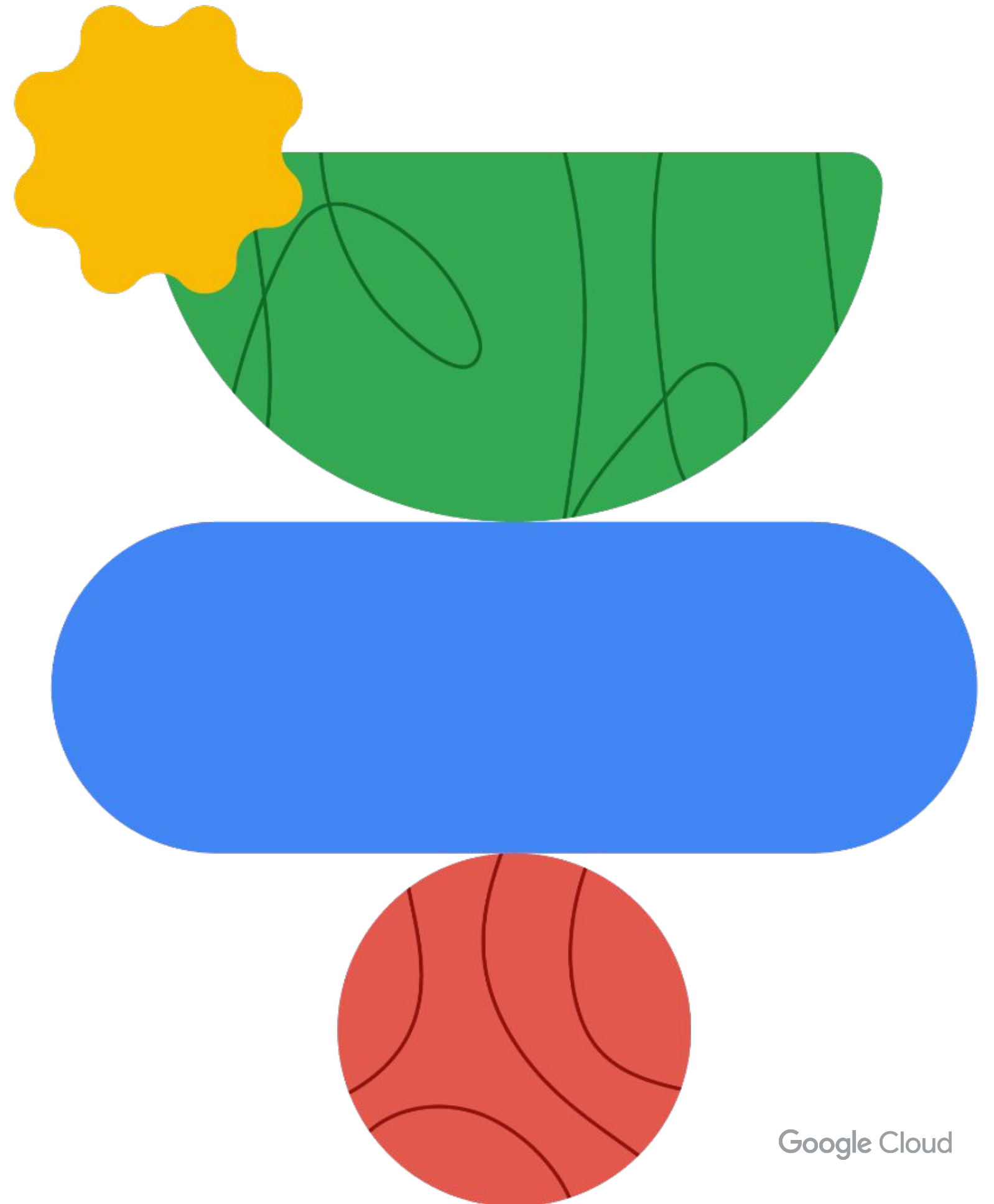
Native storage for “hot” datasets

BigQuery offers optimized, managed storage for frequently accessed or "hot" datasets. This is ideal for data requiring the fastest query performance, such as key marketing performance dashboards.

Unified data analysis platform

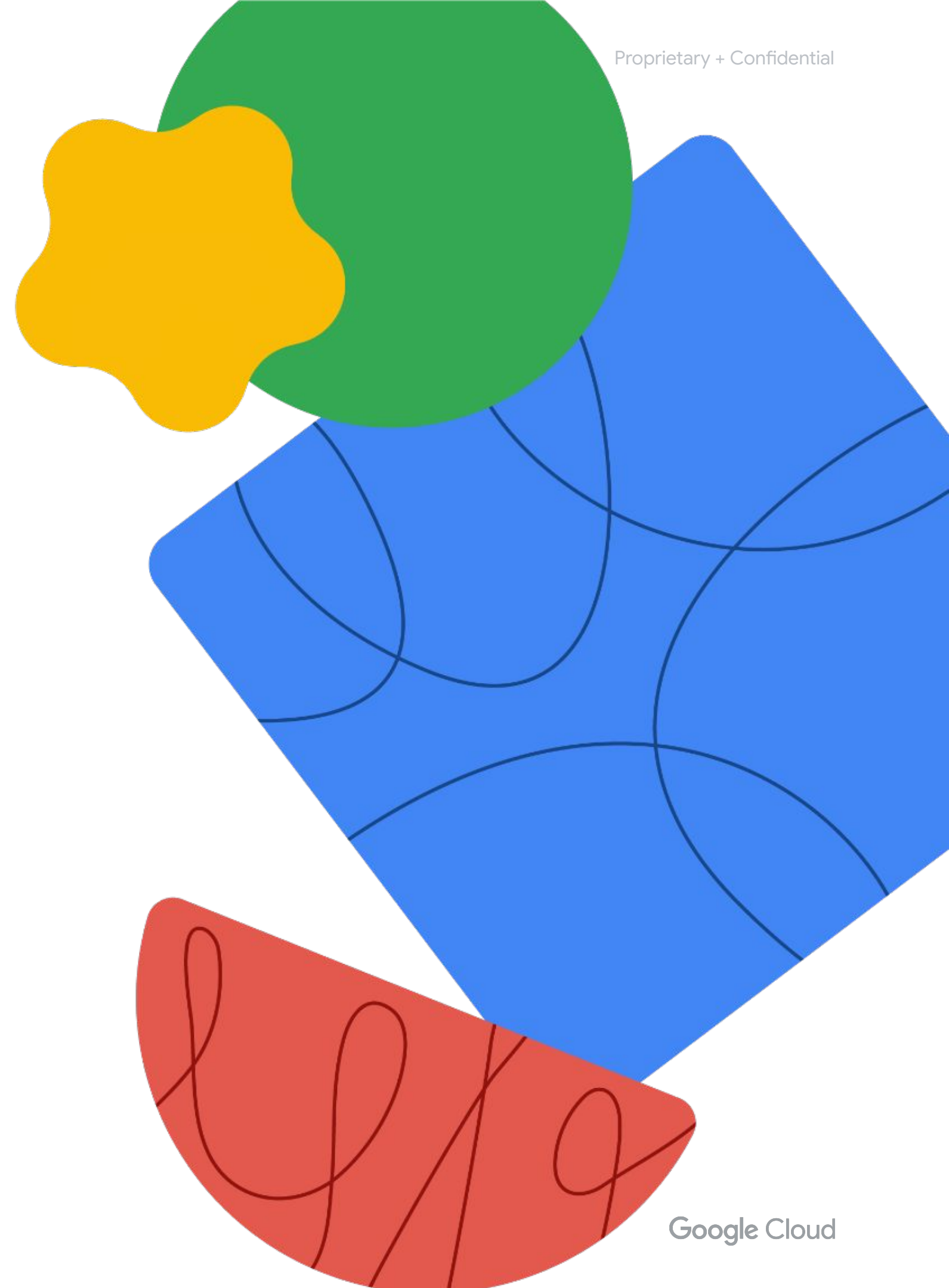
Cymbal can leverage a unified platform to analyze data located in their data lake, BigQuery's native storage, or a combination of both, all accessible through a single interface.

Combining operational data in AlloyDB

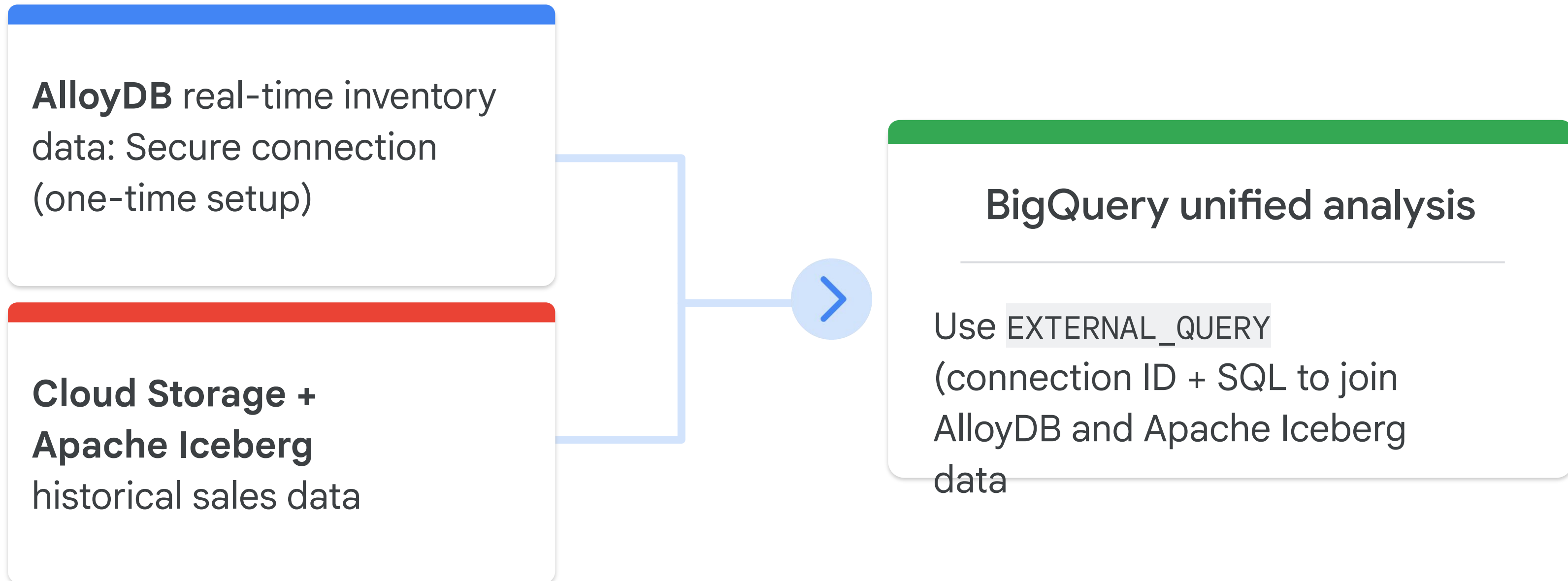


Federated queries: Combining operational and analytical data in a single query

Query data in external systems, such as an operational AlloyDB database, in real time without moving or copying it into BigQuery.



Combining data



WATCH FIRST · 2:30 · GOOGLE CLOUD TECH

Is AlloyDB Compatible with PostgreSQL?

- ✓ Yes — fully. AlloyDB is Google's managed, PostgreSQL-compatible OLTP engine, built for high availability and speed.
- ✓ Why it matters in the next 30 minutes: the federated-query lab joins BigQuery to a live AlloyDB database — your 'linked server' pattern, working.



Scan to open on YouTube

youtu.be/88k-eB3Eon8

Lab: Federated Query with BigQuery

30 minutes · 10:45–11:15 · skills.google lab seat — pre-warmed before the break ends

1. Launch the lab; create the BigQuery connection resource — your linked-server definition, one-time setup
2. Run your first `EXTERNAL_QUERY` — the inner SQL is PostgreSQL on the AlloyDB side, just like `OPENQUERY`
3. Join historical analytical data in BigQuery with live transactional rows — one query, two systems
4. Debrief: querying data where it lives — that is the lakehouse promise

Why it matters — your SQL Server systems stay operational through the migration — federation means you query them in place instead of moving everything first.

Lab: Federated query with BigQuery and AlloyDB

Overview

Explore how federated queries allows you to gain real-time insights by directly querying an operational database from within BigQuery.



What you'll do

- Connect BigQuery to an AlloyDB for PostgreSQL database.
- Join historical analytical data with live transactional data.

DIN: Exploring external tables

20 minutes · 11:15–11:35 · BigQuery console + GCS bucket `jwd-gcp-demos/orders_partitioned`

1. Browse the bucket — directories named `order_date=YYYY-MM-DD`, each holding Parquet files: Hive layout
2. Create external table `ext_part` in your class dataset — Parquet, source-data partitioning, auto-detect schema
3. Query the full table — every file scanned; note the bytes processed
4. Filter on `order_num` (a data column) — still a full scan; a `WHERE` clause alone saves nothing
5. Filter on `order_date` (the partition key) — bytes drop hard: partition pruning on raw files, no load job ran

Why it matters — Bronze made tangible — open-format files in a bucket, queryable in standard SQL with zero loading; delete the table and the files are untouched.

M8 — What you now have

- ✓ The Bronze blueprint— Parquet/Iceberg files in GCS, Hive-partitioned, queryable in place from BigQuery
- ✓ Iceberg's warehouse behavior on lake files— ACID, schema evolution, time travel — for the 13:00 lab
- ✓ Federation done hands-on— live operational data joined from BigQuery without an extract
- ✓ An external table over partitioned Parquet— you watched partition pruning cut bytes on raw files
- ✓ Next: M9 (11:35)— modernizing the warehouse itself: partitioning, clustering, BigLake

Morning Break

15 minutes · 10:30–10:45

Lab — Federated Query with BigQuery

30 minutes — 10:45–11:15 · Google Skills classroom

1. Launch the lab; note the region and use it everywhere — check the project picker
2. Create the connection resource to AlloyDB for PostgreSQL — the linked-server definition
3. Run `EXTERNAL_QUERY(connection_id, '...')` — the inner string is PostgreSQL, executed remotely
4. The payoff: join BigQuery historical data with live AlloyDB rows in one statement
5. Early? List the remote `information_schema.tables` through the same connection

Why it matters — during migration, systems that haven't moved yet stay queryable — no extract job built just to join operational tables against the warehouse.

Exploring external tables — Bronze made tangible (DIN §30)

20 minutes — 11:15–11:35 · BigQuery console — instructor leads, you follow; your class dataset

1. First browse the bucket: orders_partitioned/ is Hive-laid-out — order_date=YYYY-MM-DD/ Parquet
2. Create from GCS: pattern jwd-gcp-demos/orders_partitioned/* — Parquet, Source data partitioning on
3. URI prefix gs://jwd-gcp-demos/orders_partitioned/ ; table ext_part, type External, Auto detect
4. Query 1: SELECT * — full scan. Query 2: WHERE order_num=... — still a full scan (data column)
5. Query 3: WHERE order_date='2018-01-01' — bytes collapse: partition pruning on raw files

Why it matters — open-format files in a bucket, partitioned by date, queryable in SQL with zero loading — your architecture doc's Bronze layer is exactly this.

WATCH FIRST · 2:02 · GOOGLE CLOUD TECH

BigLake in a Minute

- ✓ One storage engine unifying warehouse and lake: fine-grained security and fast queries over open files — a single copy of the data.
- ✓ This is the upgrade on this afternoon's Iceberg lab: BigLake tables are how Bronze files get governed like warehouse tables.



Scan to open on YouTube

youtu.be/UdpNYk0EP0g

M9 — What this block gets you

- ✓ The 'traditional data warehouse challenges' on the next two slides are your SQL Server pain points—read them that way
- ✓ BigQuery fundamentals— serverless, storage/compute separation, slots and shuffle in plain terms
- ✓ Partitioning and clustering— your partition functions and clustered indexes, minus the maintenance jobs
- ✓ Iceberg partitioning and predicate pushdown— open metadata pruning files that live in your own bucket
- ✓ Both are a preview— the 3-day Data Warehousing with BigQuery course goes deep; today is the decision rules

Late morning into early afternoon

11:35–12:00 M9 — Modernizing DWs with BigQuery & BigLake (opens with your pain points)

12:00–13:00 Lunch

13:00–13:45 Lab: Querying External Data and Iceberg Tables — your Bronze pattern, hands-on

LAB

13:45–14:00 Iceberg deep-dive — what's under the table you just built

DEMO

14:00 M10 — governance + medallion: your Bronze/Silver/Gold in Google's own deck

Traditional data warehouse challenges (1 of 2)

Cymbal's data architects face significant challenges with traditional data warehouses:

01

Extensive time spent on capacity planning for events, such as the holiday shopping season.

02

Difficulty handling unexpected traffic spikes.

03

Manual provisioning of hardware, software installation, and patching.

04

Rebalancing data manually as it grows.

Traditional data warehouse challenges (2 of 2)

Operational overhead is **expensive** and **slows down** the ability of the analytics teams to get **timely insights**.

01

Extensive time spent on capacity planning for events, such as the holiday shopping season.

03

Manual provisioning of hardware, software installation, and patching.

02

Difficulty handling unexpected traffic spikes.

04

Rebalancing data manually as it grows.

Those five challenges are yours — this is the delete key

Each one is an infrastructure problem — the fix is to remove the infrastructure.

- Capacity planning for peak events— serverless slots, allocated per query and released when it finishes
- Sized for peak, idle the rest of the day— storage and compute scale and bill independently
- Procurement, installation, patching weekends— fully managed; that M&O; line item disappears
- Rebalancing data as it grows— storage scales automatically; no new drives, no filegroup shuffles
- The nightly window sized the whole box— this engine was built for exactly the scans you avoid
- Slow answers for analysts— thousands of slots per query: a wider machine, not a faster one

BigQuery

Enterprise-grade, fully managed,
serverless data warehouse.



BigQuery fundamentals

Fully managed

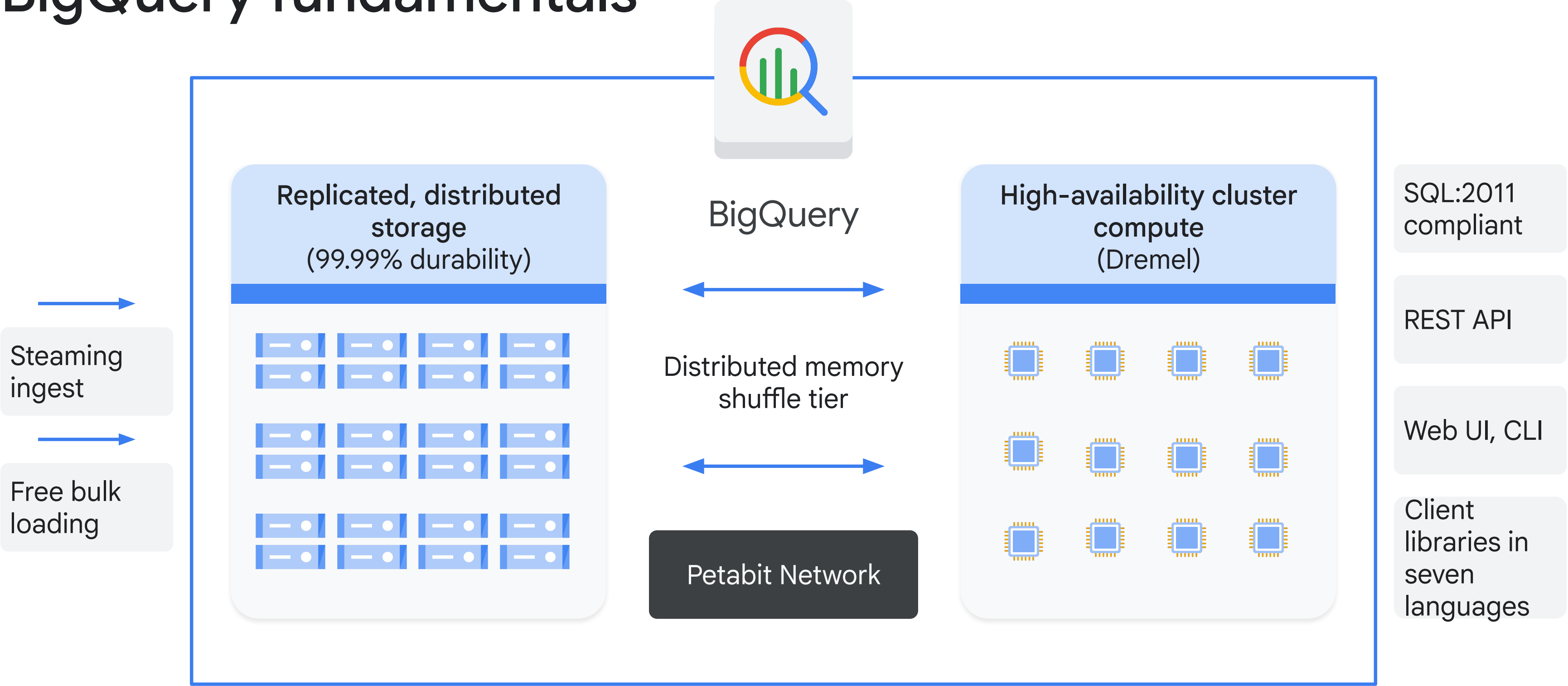
The infrastructure (such as the hardware, the networking, and the low-level software) is all handled by Google. Your team doesn't need to worry about patches, updates, or hardware failures.

Serverless

Serverless takes things a step further. You don't have to provision or manage any servers at all. You simply load your data and start querying. BigQuery automatically allocates the necessary resources to run your queries and scales them up or down based on the complexity of your request.

The magic behind BigQuery is its architecture. BigQuery **separates storage from compute.**

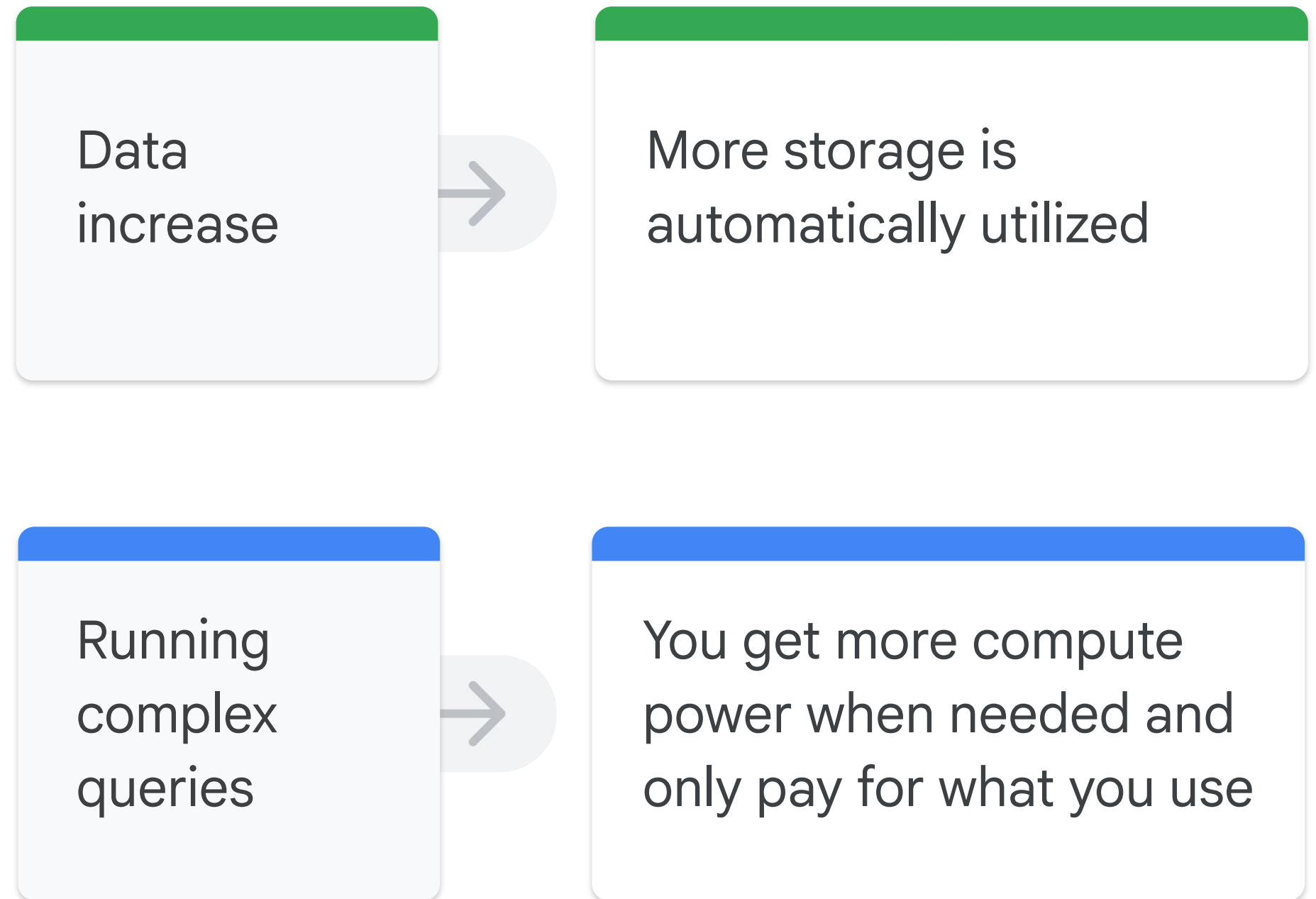
BigQuery fundamentals



BigQuery fundamentals

Because storage and compute are separate, they can scale independently.

This separation is a game changer for managing costs and ensuring performance.



BigQuery achieves its speed through two core concepts: **slots and shuffle**.

What is a slot?

A small, self-contained unit of computational power that includes **CPU**, **RAM**, and **network bandwidth**.



Slots

01

BigQuery's Dremel engine assigns thousands of slots to a query job.

02

Each slot processes a small piece of data simultaneously.

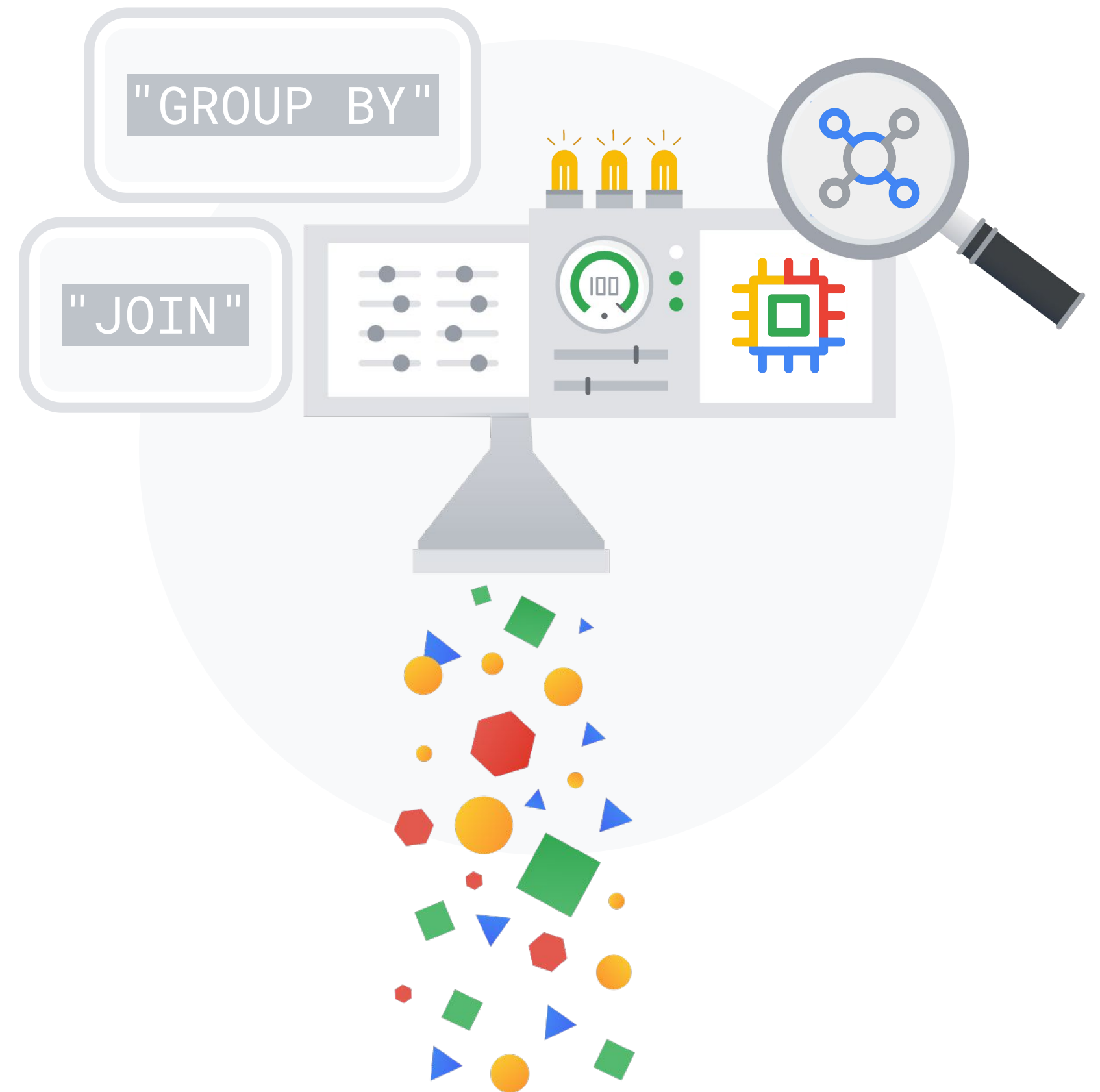
03

This massively parallel processing enables BigQuery to scan terabytes of data quickly.



What is shuffle?

The process of redistributing the intermediate data that the slots have processed.



Shuffle

01

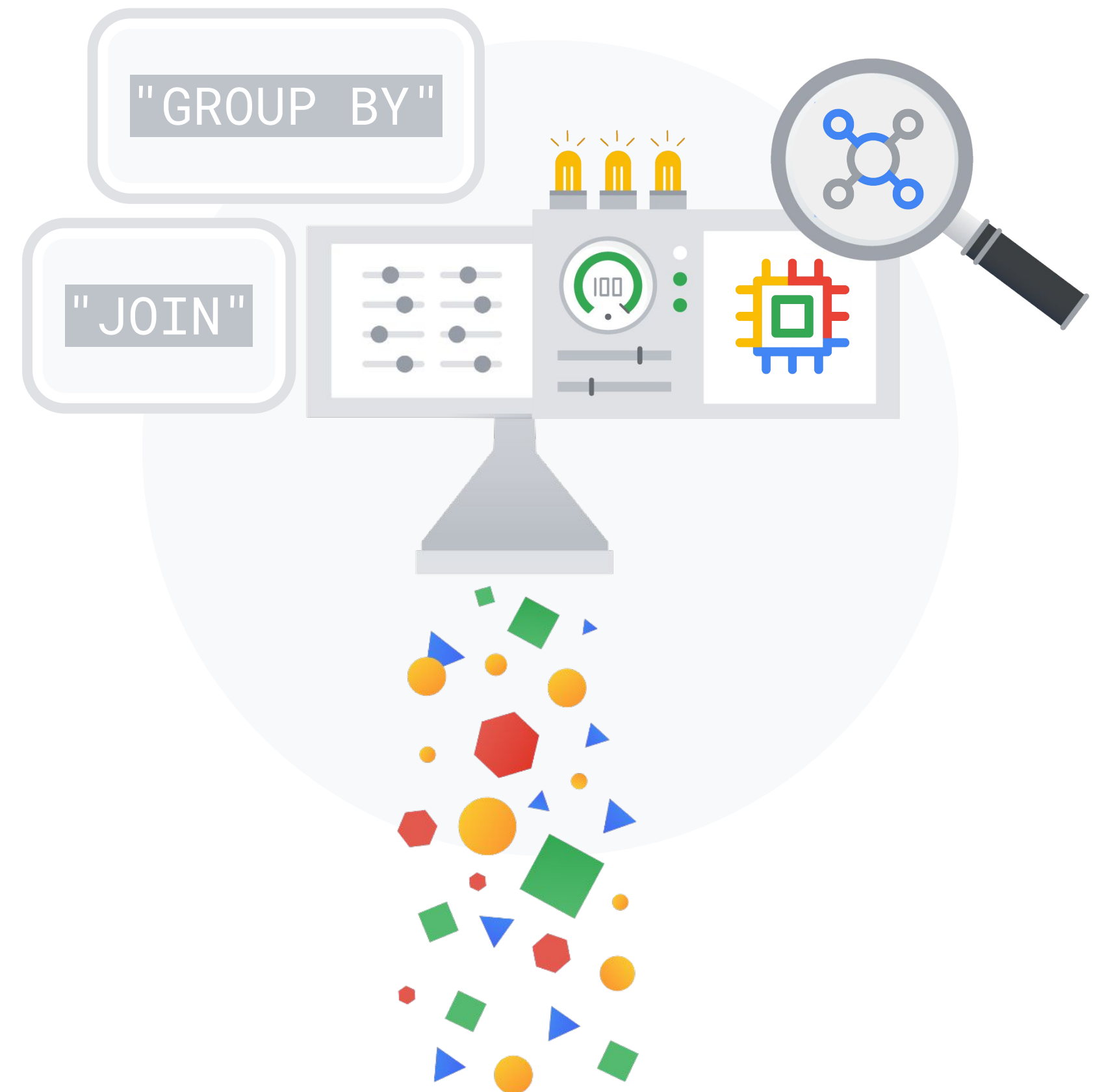
Gathers and reorganizes data.

02

Sends it to the next set of slots for further processing, such as aggregation or joining.

03

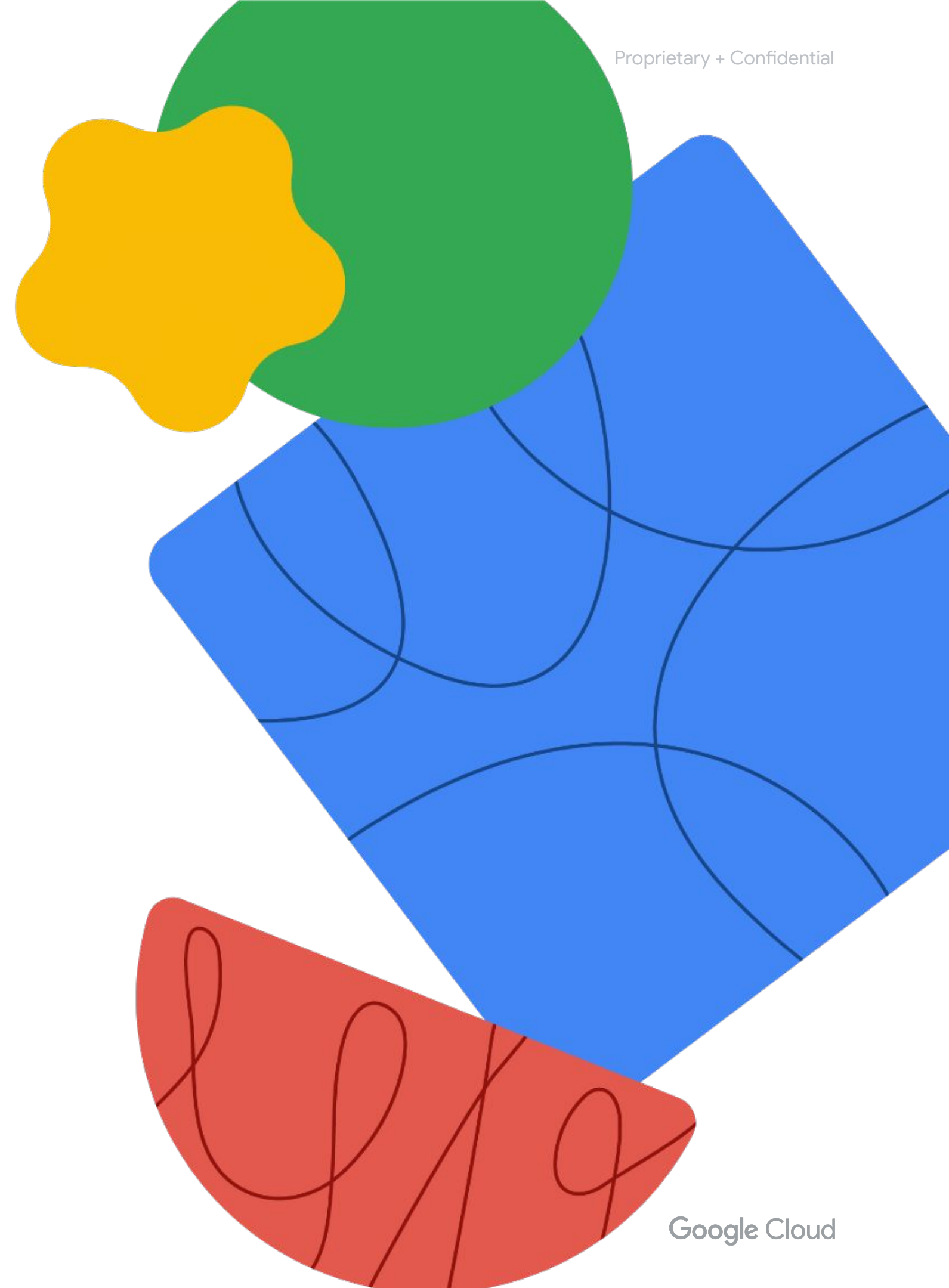
Executes complex analytical queries efficiently at a massive scale.



BigQuery's compute and storage separation enables versatile **multisource data querying.**

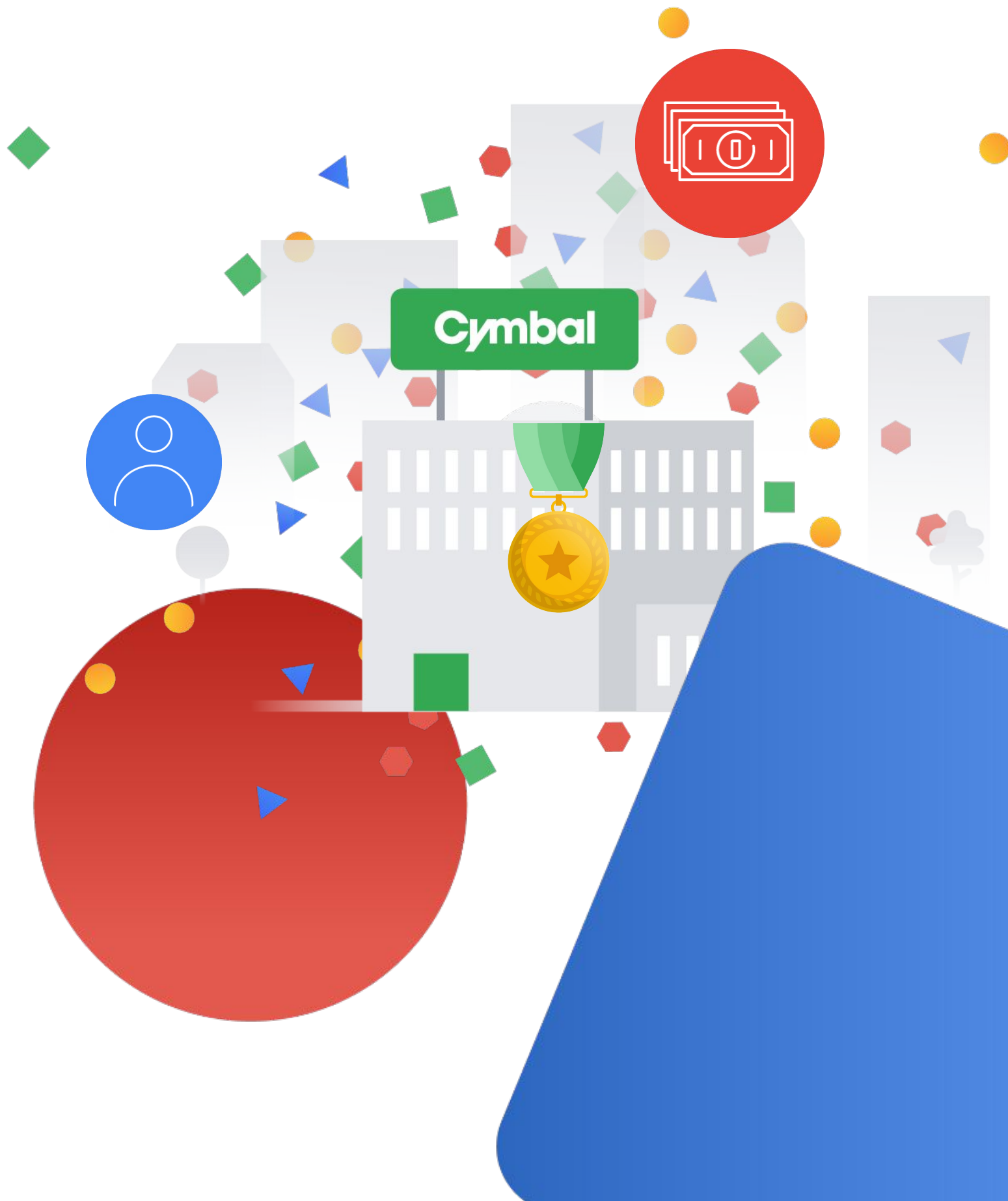
Partitioning and clustering

Two powerful concepts you can use to allow BigQuery to process your data efficiently and cost-effectively.



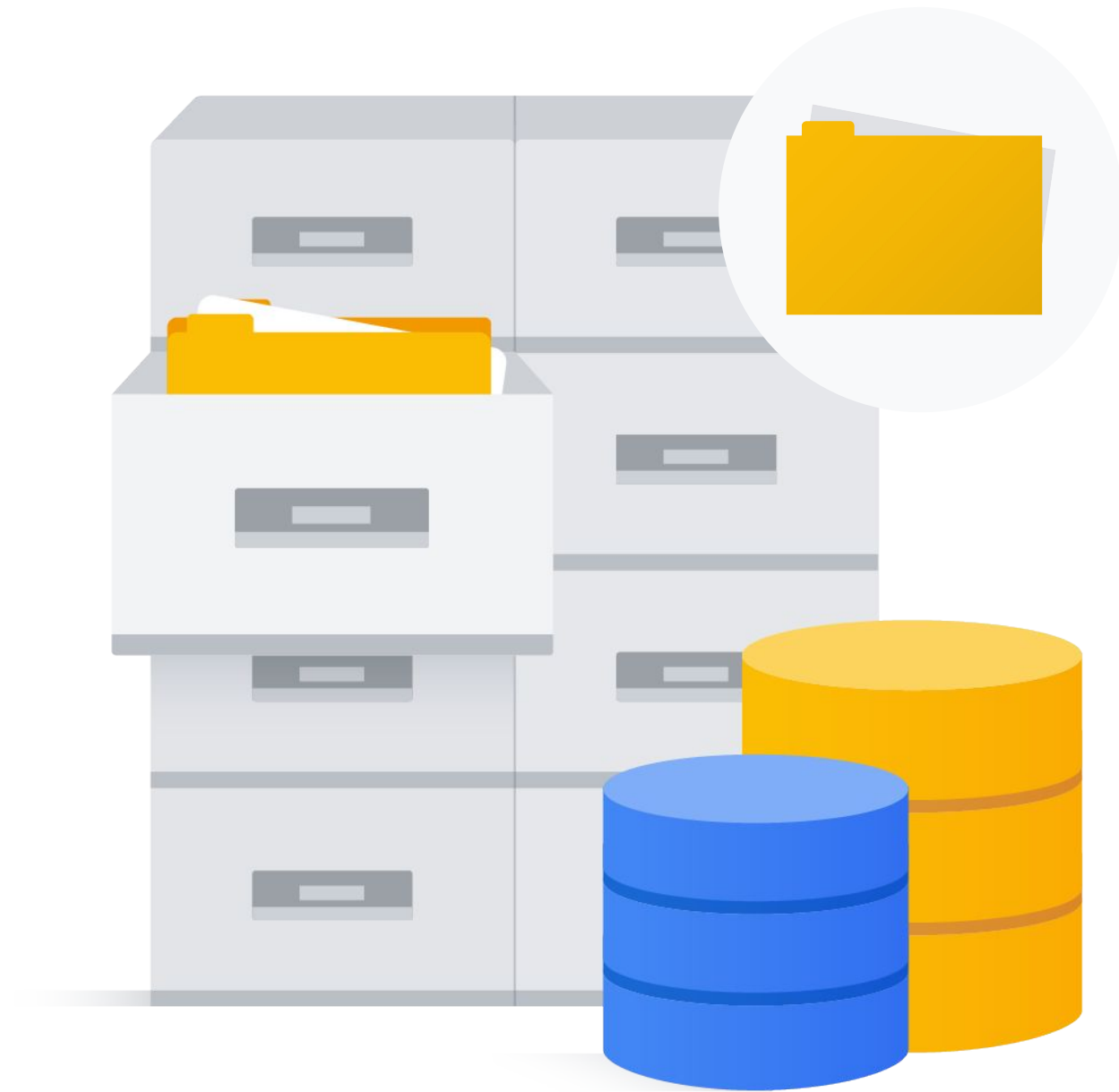
“

Running queries that scan every single piece of data can become slow and expensive, even for BigQuery. You can optimize this with **partitioning and clustering**.



Partitioning

Partitioning is like organizing data into separate drawers for year, month, and day. In BigQuery, you can partition a table based on a date or an integer column.



Example: Sales data table

transaction_date	customer_id	product_category	sales_amount
2025-08-01	CUST002	Headsets	52
2025-08-01	CUST002	Keyboards	186
2025-08-02	CUST003	Headsets	465
2025-08-02	CUST005	Headsets	57
2025-08-03	CUST003	Shoes	413
2025-08-03	CUST001	Consoles	347

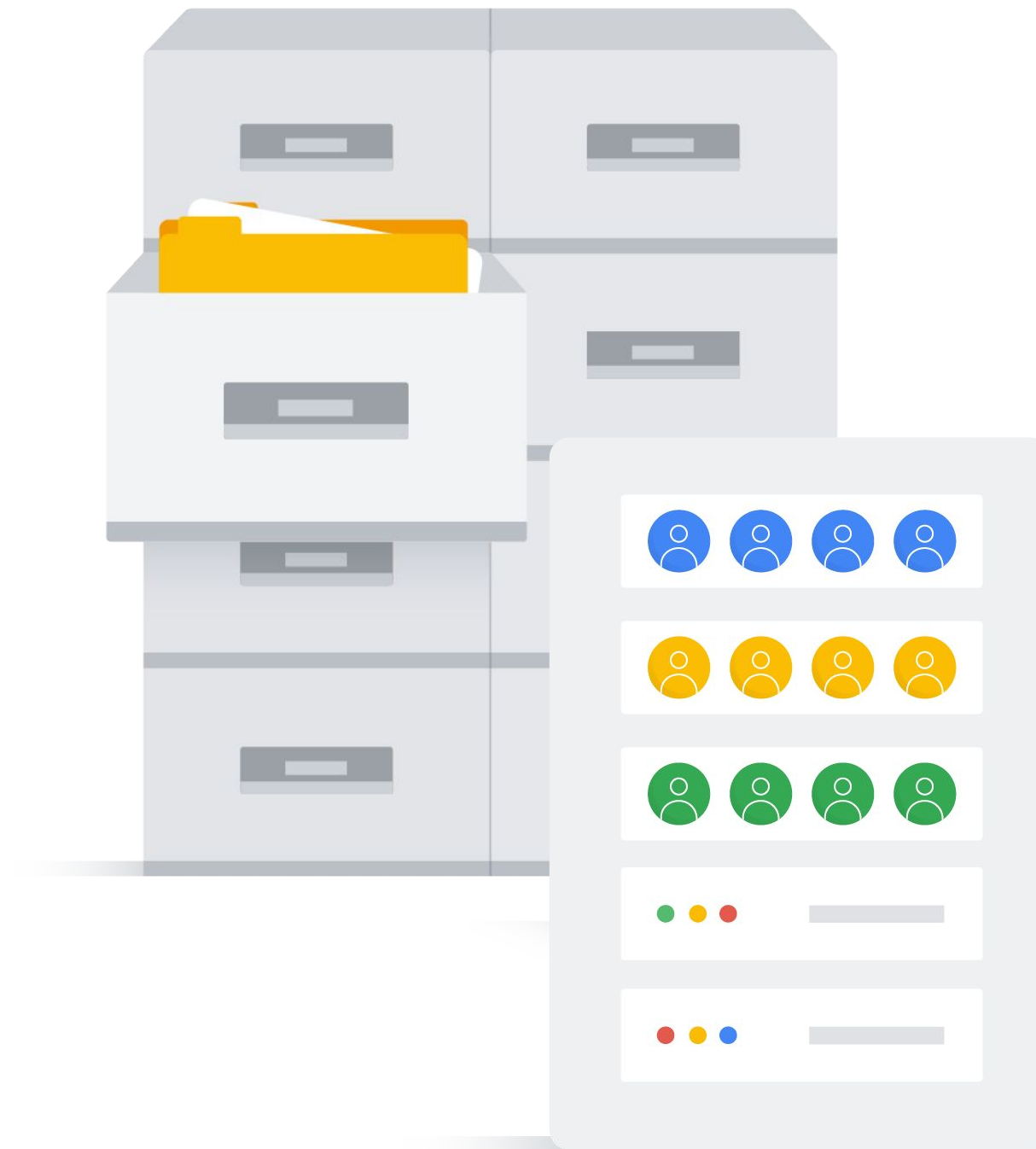
Partitioned by `transaction_date`

transaction_date	customer_id	product_category	sales_amount
2025-08-01	CUST002	Headsets	52
2025-08-01	CUST002	Keyboards	186

transaction_date	customer_id	product_category	sales_amount
2025-08-02	CUST003	Headsets	465
2025-08-02	CUST005	Headsets	57

Clustering

Clustering sorts the data within each partition. Think of it as organizing the files within each drawer of your filing cabinet alphabetically by customer name.



Clustering example

transaction_date	customer_id	product_category	sales_amount
2025-08-01	CUST002	Headsets	52
2025-08-01	CUST002	Keyboards	186
2025-08-01	CUST003	Headsets	465
2025-08-01	CUST005	Headsets	57
2025-08-01	CUST003	Shoes	413
2025-08-01	CUST001	Consoles	347

Query example with partition + clustering

```
SELECT *  
FROM cymbal_sales  
WHERE transaction_date = '2025-08-01'  
      AND customer_id = 'CUST002';
```

Partitioning & clustering — your tuning toolkit, translated

Same disciplines you use today, declared at CREATE TABLE time — then never maintained again.

- Partition functions and sliding windows—
partitioned tables; pruning is automatic on a date filter
- Clustered indexes and index tuning—
clustering columns: sort order inside storage blocks, up to four
- Index rebuilds in the maintenance plan—
automatic background reclustering; there is no job to write
- No retroactive ALTER— repartition by rebuilding with CREATE TABLE AS SELECT
- Decision rule— partition on the date your reports filter; cluster on your top filter/join columns
- Aim for ~1–10 GB per partition— daily by default; tiny partitions slow metadata, not queries

Preview, not the full story — the 3-day Data Warehousing with BigQuery course being scheduled goes deep on both, hands-on.

Partitioning in Iceberg

01

Leveraging existing iceberg structures

Intelligently leverages the existing structure defined in the Apache Iceberg table's own metadata.

02

Defining partitions

With partitioning, the partitions are typically defined and written by a data processing engine, such as Apache Spark.

03

Data organization

The Apache Iceberg metadata meticulously tracks which specific data files in Cloud Storage belong to which date.

Query filtering example

```
WHERE transaction_date = '2025-08-15'
```

01

BigQuery (via Lakehouse) reads this metadata and identifies the exact set of files corresponding to that date and prunes away all others.

02

Ignores irrelevant data, drastically reducing the amount of data scanned, which lowers costs and accelerates query performance.

Clustering in Apache Iceberg

01

Clustering in BigQuery is mirrored by data sorting and file-level statistics in Apache Iceberg.

02

The data within Apache Iceberg's Parquet or ORC files is often sorted by specific columns, such as `customer_id`.

03

The Apache Iceberg metadata stores statistics, such as the minimum and maximum `customer_id` values, for each individual data file.

Predicate pushdown

01

If your BigQuery query filters for a specific customer, the query planner consults these statistics.

02

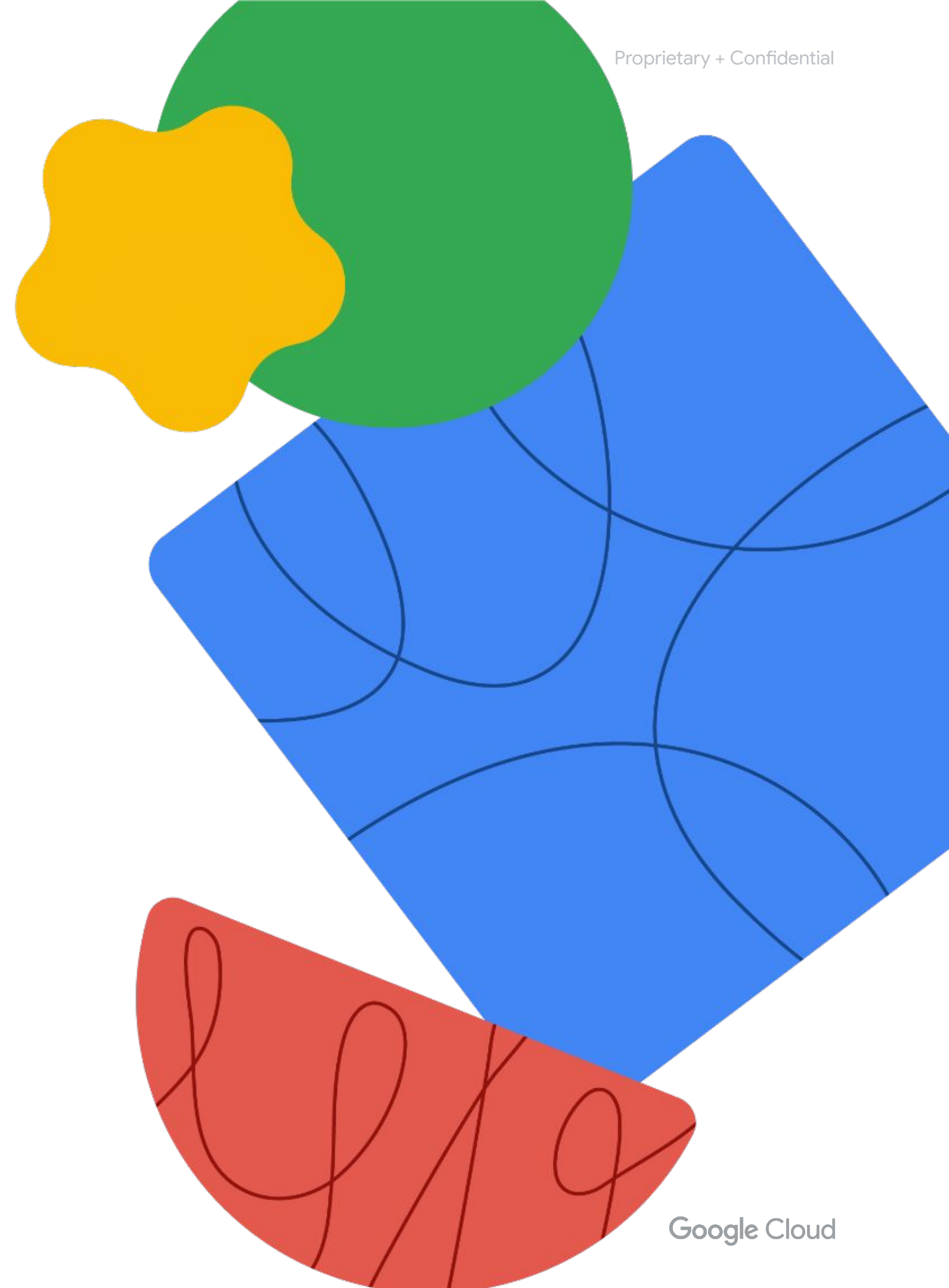
Skips reading any files whose min/max range doesn't contain the requested `customer_id`.

03

Predicate pushdown provides a finer-grained level of data pruning, ensuring maximum query efficiency.

Why it matters

Partitioning and clustering together give BigQuery fine-grained control over how your data is queried, leading to significant performance improvements and cost savings, especially as your data grows.



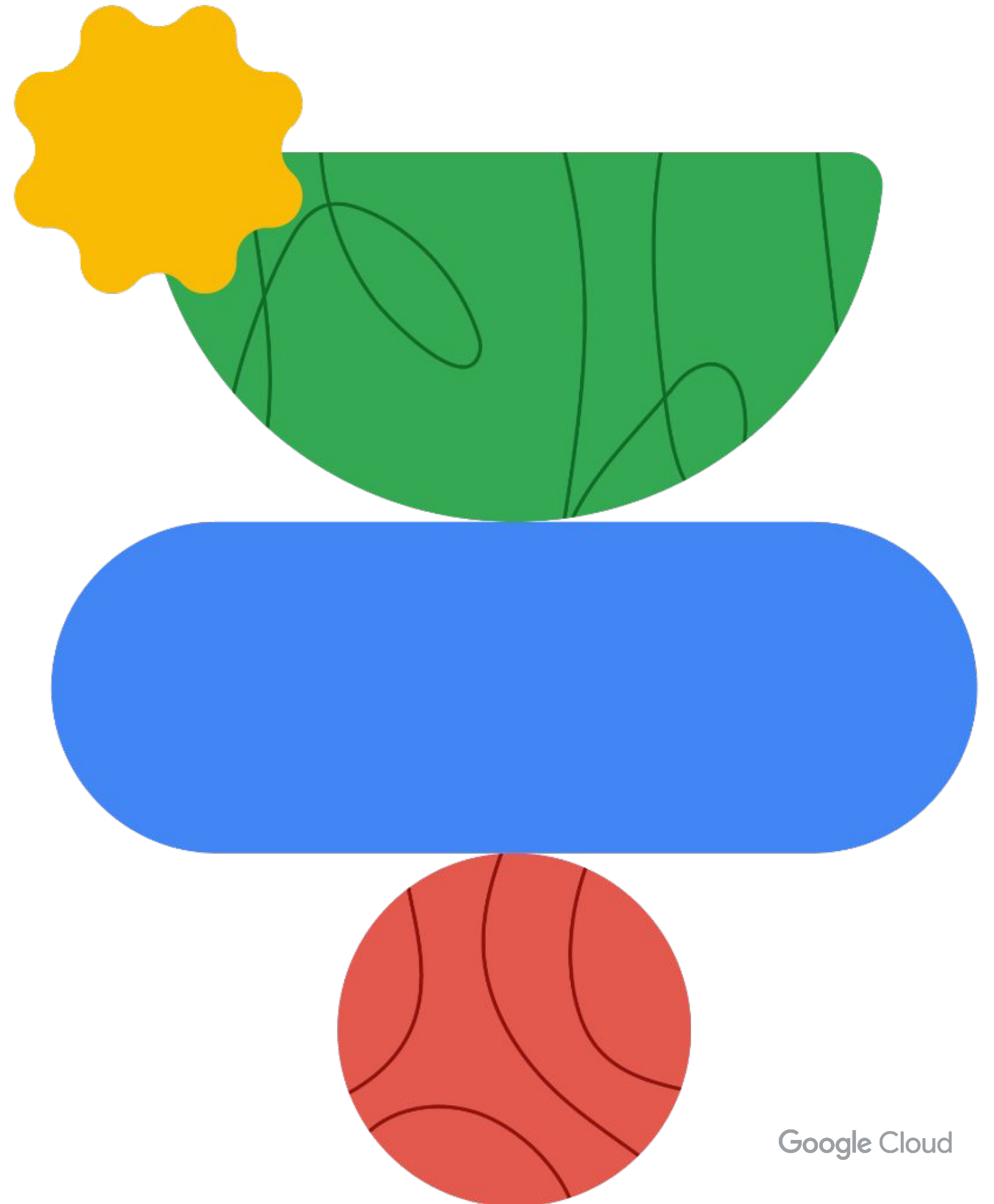
BigQuery uses metadata to skip unnecessary data, so **queries run faster** and cost less.

Iceberg partitioning — the same pruning, on files you own

Why your Bronze layer cares: filtering works before anything is ever loaded.

- Iceberg metadata tracks which Parquet files belong to which partition— open metadata, in your bucket
- Per-file min/max statistics let the planner skip files that can't contain your value— predicate pushdown
- Same discipline as partition elimination and index seeks— but any Iceberg-speaking engine can read it
- This is the answer to 'will BigQuery hold our data hostage?'— open files plus open metadata
- Also a preview— Iceberg internals get the deep, hands-on treatment in the Data Warehousing course

**Valuable data exists in
many places and
formats**



Data warehouse vs. data lake

Traditional data storage

- Data warehouse: Ideal for structured, organized data used for business insights.
- Data lake: A cost-effective place for huge amounts of raw data in any format.

Challenges of separation

- Data silos: Hard to analyze different data types together.
- Complex extract, transform, load (ETL): Required moving and duplicating data, increasing costs and delays.
- Governance Issues: Managing security and access across two systems was complicated.

A data lakehouse **solves these challenges.**

Data lakehouse architecture

01

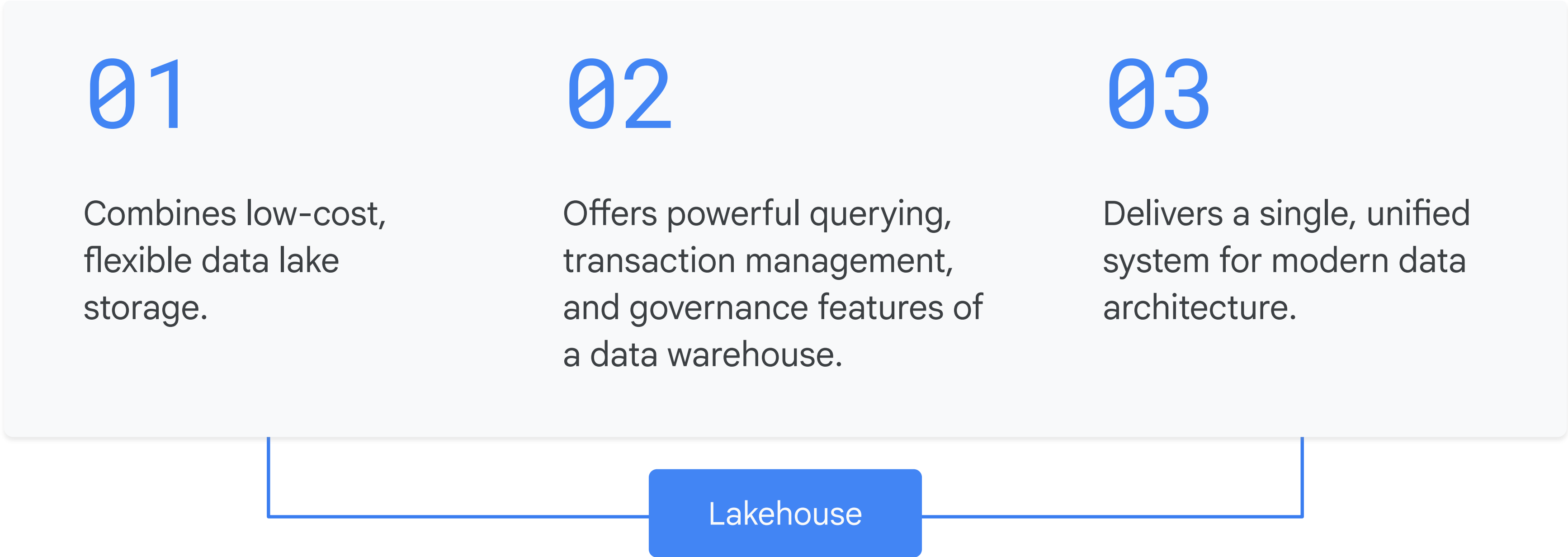
Combines low-cost, flexible data lake storage.

02

Offers powerful querying, transaction management, and governance features of a data warehouse.

03

Delivers a single, unified system for modern data architecture.

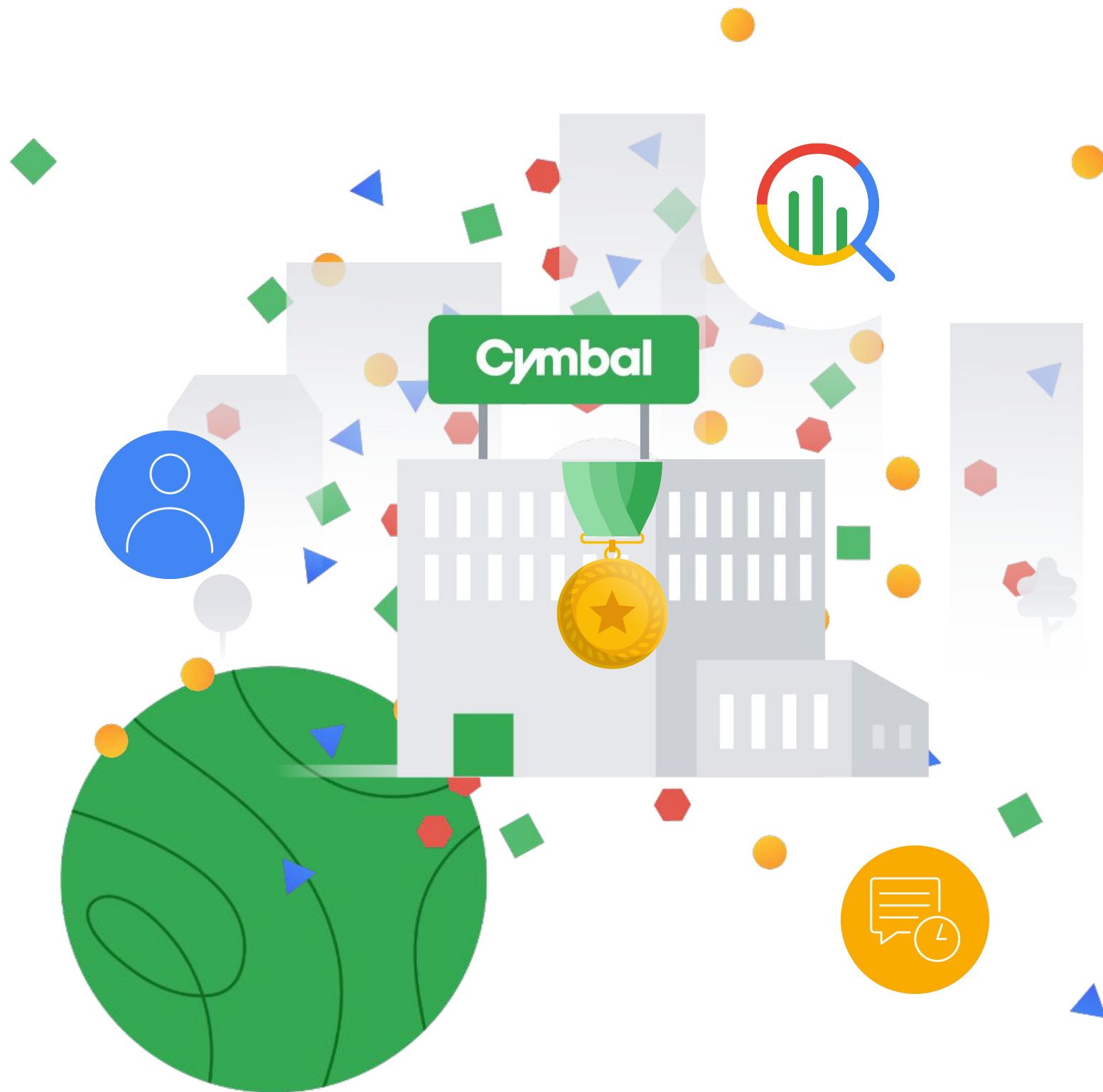


Lakehouse

Lakehouse and external tables

- 01 Lakehouse extends BigQuery.
- 02 Create Lakehouse tables over data in object stores, such as Google Cloud Storage.
- 03 Reference the source data files in your data lake, and query them.





How can Cymbal use Lakehouse external tables?

- Cymbal's data science team uses Cloud Storage to store raw web server log files in JSON format.
- Analyzing this clickstream data used to require a complex data pipeline to parse and load it into BigQuery.

Lakehouse simplifies this process.

Lakehouse external table

01

They create a Lakehouse table directly over the JSON files in Cloud Storage.

02

Use the familiar BigQuery SQL interface to query clickstream data instantly.

03

Join this external table with a native BigQuery sales table, to discover correlations between website behavior and purchasing habits.

Lab: Introduction to querying Apache Iceberg tables with BigQuery

Overview

Explore how BigQuery's Lakehouse tables can query data in open-source formats directly within Cloud Storage.



What you'll do

- Create a Lakehouse table that points to an existing Apache Iceberg table.
- Analyze data without moving or duplicating it.

Lab: Querying External Data and Iceberg Tables

45 minutes · Qwiklabs seat — BigQuery console

1. Create a Lakehouse (Iceberg) table pointing at the existing Apache Iceberg data in Cloud Storage
2. Suffix your table name with your initials — 16 identical DDL statements in one dataset will collide
3. Query it: filter and aggregate — plain SQL over files that were never loaded anywhere
4. Watch the dialect: backticks not brackets, LIMIT not TOP, dataset in the fully qualified name
5. Notice the DDL is metadata-only and returns instantly — nothing moved, nothing copied

Why it matters — this is your Bronze pattern, hands-on — raw data stays in open formats in Cloud Storage and is queryable from minute one. Bronze is not a landing zone you copy out of.

Iceberg deep-dive — what's under the table you just built

15 minutes · Instructor demo — BigQuery console (from Jeff's Iceberg tutorial, gcp-demos)

1. Frame the one idea that matters: three Iceberg flavors differ by who owns the metadata and who can write
2. Debrief the lab: what your Iceberg table has that this morning's external table didn't — DML, time travel, snapshots
3. Upgrade the morning's ext_part external table to a BigLake table — same files, richer metadata
4. Dwell on WITH CONNECTION — the connection's service account reads the bucket; when it fails, it's that IAM grant
5. Payoff: column-level security and row access policies on files that never left the lake

Why it matters — files became a table this morning, got ACID semantics in the lab, and get governance now — three rungs of one ladder, and the rungs are the Bronze section of your architecture doc. Fallback if the tutorial is still under construction: extended lab debrief plus this BigLake demo.

M9 — What you now have

- ✓ Your SQL Server pain points named on Google's own slides— and the answer: delete the infrastructure layer
- ✓ Slots and shuffle in plain terms— a wider machine, not a faster one; 'bytes shuffled' is where slow queries hide
- ✓ Partitioning and clustering decision rules— flagged as a preview of the Data Warehousing course's deep dive
- ✓ Hands-on Bronze: an Iceberg/Lakehouse table over files in Cloud Storage, queried without loading anything
- ✓ Next: M10 at 14:00— Google's own Bronze/Silver/Gold zone slides, held up next to your architecture doc

Lunch

60 minutes · 12:00–13:00

Lab — Querying External Data & Apache Iceberg Tables

45 minutes — 13:00–13:45 · Google Skills classroom

1. Launch the lab; check the project picker; suffix table names with your initials (16 seats)
2. Type the DDL yourself: a Lakehouse table over the existing Iceberg data in the bucket
3. Metadata-only — returns in seconds; nothing loads; a hang means a typo in the URI
4. Run filters and aggregations over data that was never loaded — note bytes processed
5. Dialect deltas: backticks not [brackets], LIMIT not TOP, IFNULL not ISNULL

Why it matters — your Bronze layer, hands-on: raw data stays in open formats in object storage, and BigQuery queries it in place — queryable from minute one.

Iceberg deep-dive + the BigLake table upgrade

15 minutes — 13:45–14:00 · Instructor-led — Jeff's Iceberg tutorial (gcp-demos)

1. Iceberg in one sentence: an open metadata layer that gives Parquet files table behavior
2. What it buys: ACID, schema evolution, time travel, hidden partitioning — over files in your bucket
3. Who writes, who keeps metadata: Lakehouse over existing Iceberg / BigQuery-managed / open engines
4. The upgrade: this morning's external-table pattern gains governance as a BigLake table

Why it matters — the no-lock-in answer for M&O: open Parquet + open metadata in your buckets — detach and walk away anytime.

WATCH FIRST · 4:39 · GOOGLE CLOUD TECH

De-identifying Data with Sensitive Data Protection

- ✓ Finds and classifies PII, then redacts, masks, or tokenizes it — before it ever reaches a dashboard.
- ✓ The governance half of M10: your state data has PII in it, and this is the service that knows exactly where.



Scan to open on YouTube

youtu.be/MHEnuw9if3Y

M10 — What this block gets you

- ✓ The governance tooling behind your single source of truth— Knowledge Catalog, Sensitive Data Protection, fine-grained security
- ✓ The moment: Google's own Bronze/Silver/Gold zone slides held up next to your architecture doc— zone for zone
- ✓ A GCP home for each zone— GCS/BigLake Bronze, Dataform Silver-to-Gold, native BigQuery Gold marts
- ✓ A phased migration path off the SQL Server DW— no big-bang cutover, extracts retired last
- ✓ The cost levers and best practices you set on day one— storage class, bytes scanned, budgets and alerts

M10 — 14:00 to 14:35, then break

14:00–14:10	Governance as your single-source-of-truth requirement — Knowledge Catalog, Sensitive Data Protection
14:10–14:15	Fine-grained security — column-level, row-level, dynamic masking
14:15–14:25	The Medallion — Google's Bronze/Silver/Gold zones next to your architecture doc
14:25–14:35	Migration strategy, cost management, best practices
14:35–14:50	Break
14:50–15:20	M11 — the BQML direction and the Power BI last mile

What is metadata?

Metadata is data about data, which can include:

01 Who created the data

02 When it was created

03 What it contains

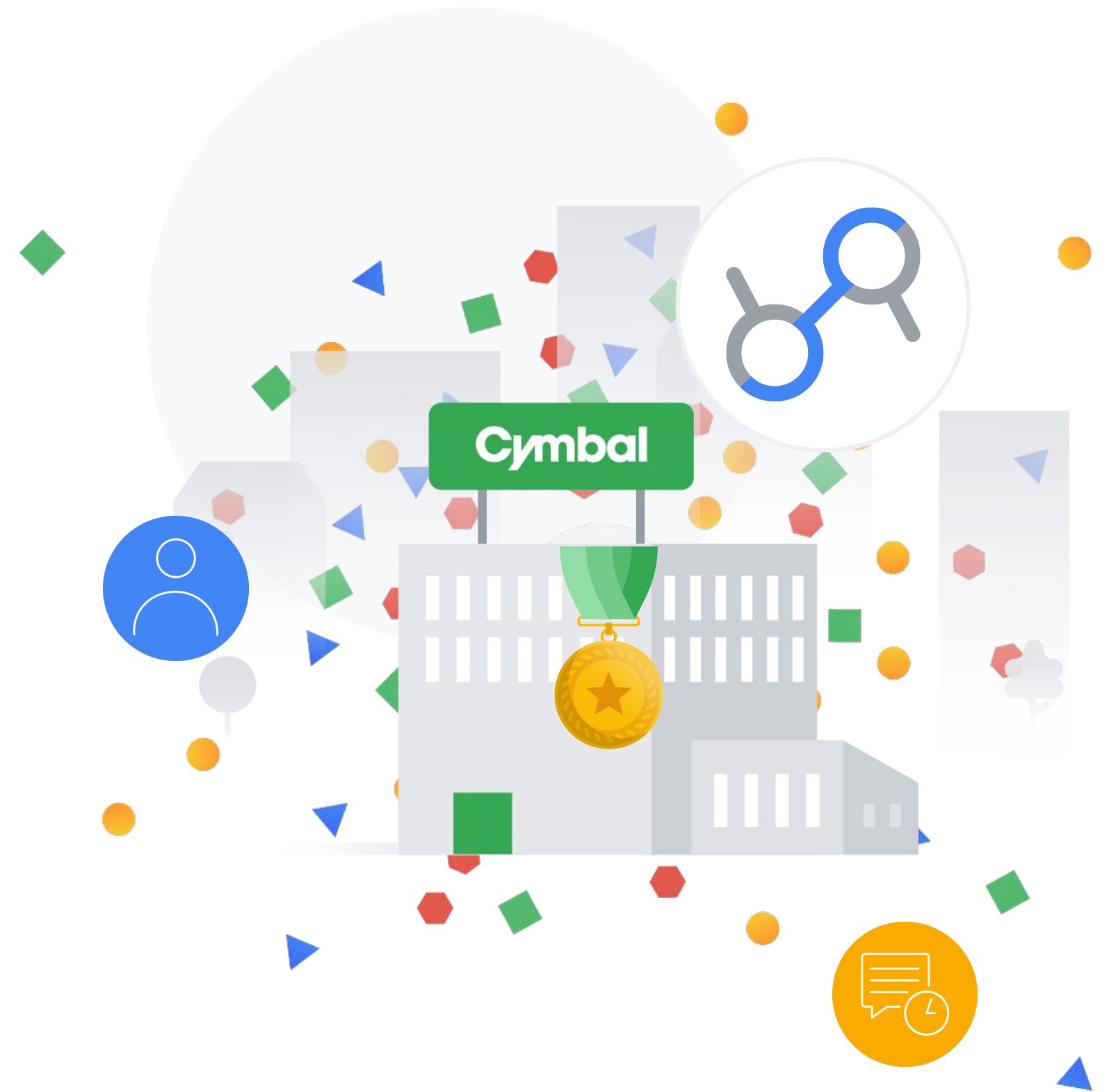
04 How it relates to other data

05 Who owns it

06 Its security sensitivity

Knowledge Catalog for Cymbal

Without a centralized metadata system, data management can be difficult. Knowledge Catalog provides a unified metadata hub.



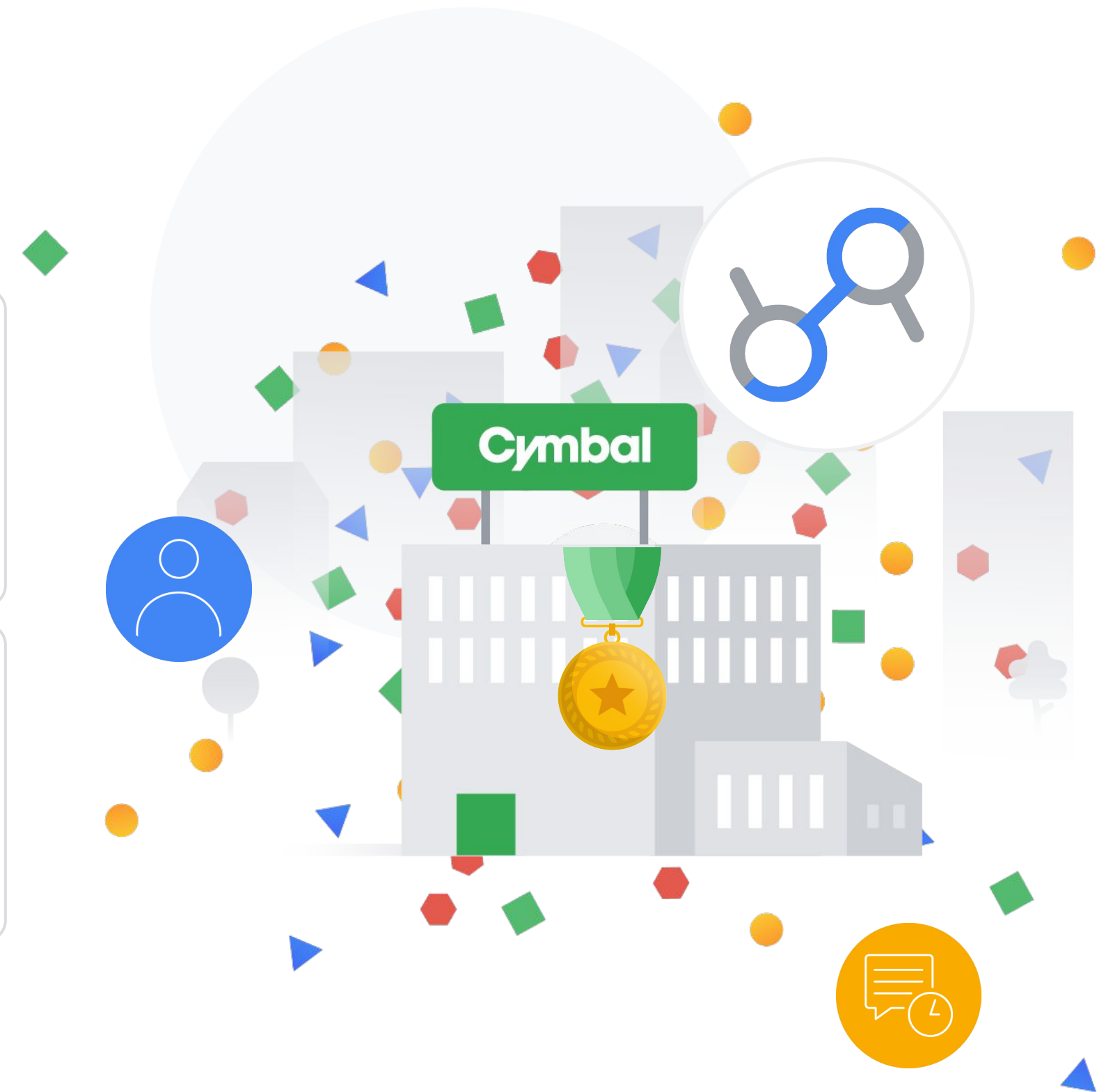
Knowledge Catalog for Cymbal

01

Acts as a universal catalog for all data assets, whether they reside in BigQuery, Cloud Storage, or Lakehouse.

02

Use to discover data, understand data lineage, and manage and augment metadata.



By providing a single reference source,
Knowledge Catalog helps Cymbal manage
data at scale while **ensuring consistency
and quality.**

Governance = your single source of truth

Your problem is not storing data — extracts scatter it. One governed copy serves everyone.

- Knowledge Catalog— one catalog across BigQuery, Cloud Storage, lakehouse; which table is real has an answer
- Sensitive Data Protection— finds and classifies PII before it reaches a dashboard; your Data Discovery analogue
- Column-level security + masking— purchase history yes, contact info no; your column GRANT/DENY plus masking
- Row access policies— the regional manager sees only their region, like your RLS predicates
- Lineage— change a Silver table, see exactly which Gold marts and Power BI reports you touched

Sensitive Data Protection



Protect customers' sensitive information, such as names, addresses, and credit card numbers.



A data breach can damage Cymbal's reputation and lead to significant financial penalties.



Sensitive Data Protection is an essential tool for securing data.

Sensitive Data Protection

Discover

Run scans on data lakehouse and other data to identify multiple types of sensitive field.

Classification

Data is classified by sensitivity level. This ensures the right security controls are applied.

Protection

Use techniques, such as masking or tokenization, to de-identify the data.

Example: *****9787

This proactive approach to data protection allows Cymbal to **use their data for analytics and ML** without exposing sensitive information.

Fine-grained security

Column-level security

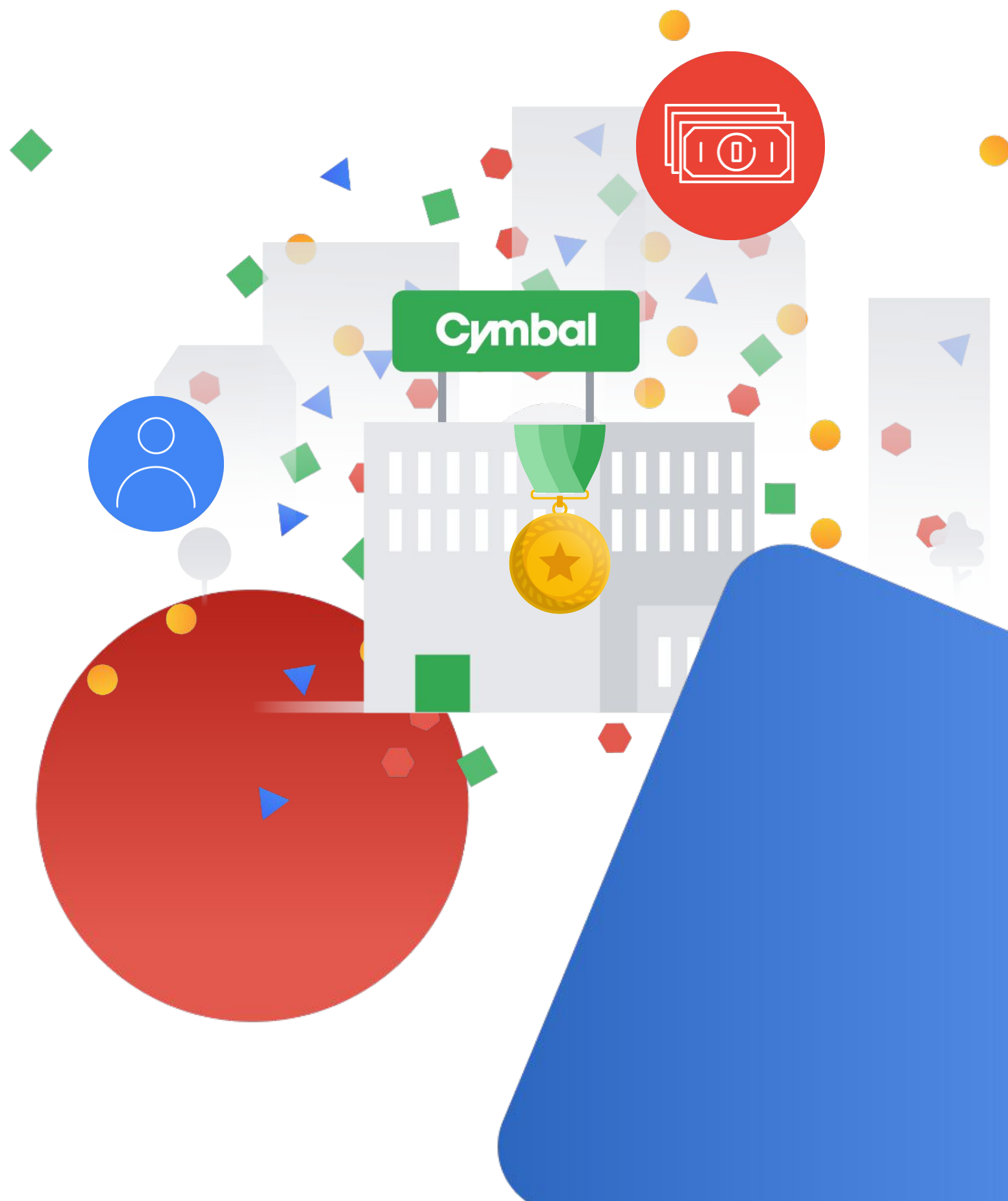
Restricts access to specific columns in a table. For example, a marketing analyst might be able to access a customer's purchase history but not their contact information. This is effective for protecting Personally Identifiable Information (PII).

Row-level security

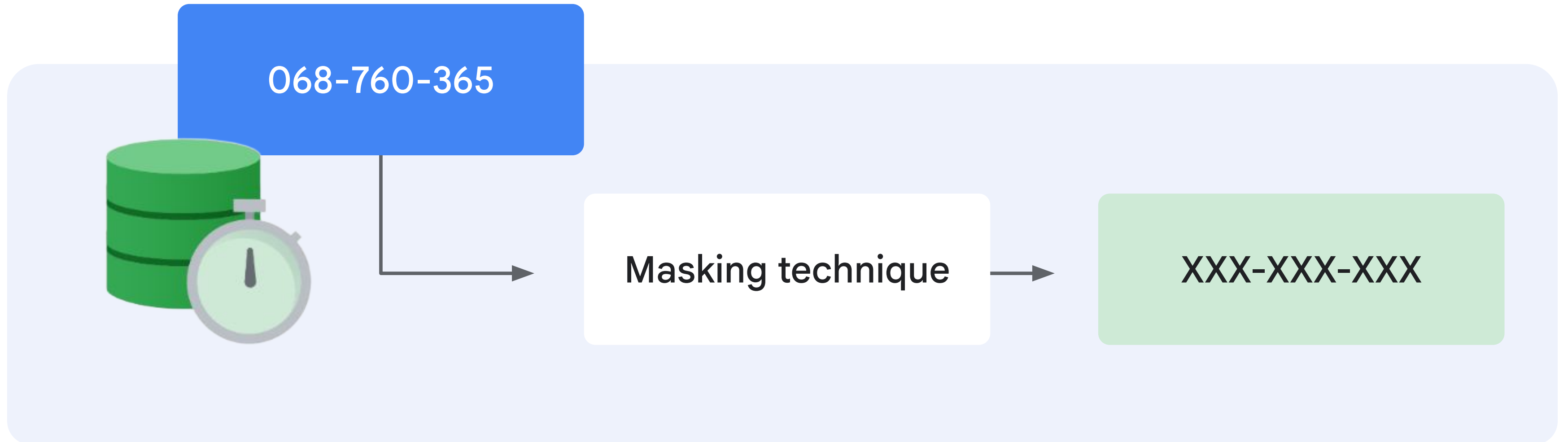
Filters which rows a user can access. A regional sales manager for North America, for instance, would only have access to sales data for that region. This is particularly useful for large, multinational companies, such as Cymbal.

“

Cymbal can ensure that their data is secure and accessible to the correct people at the appropriate time.

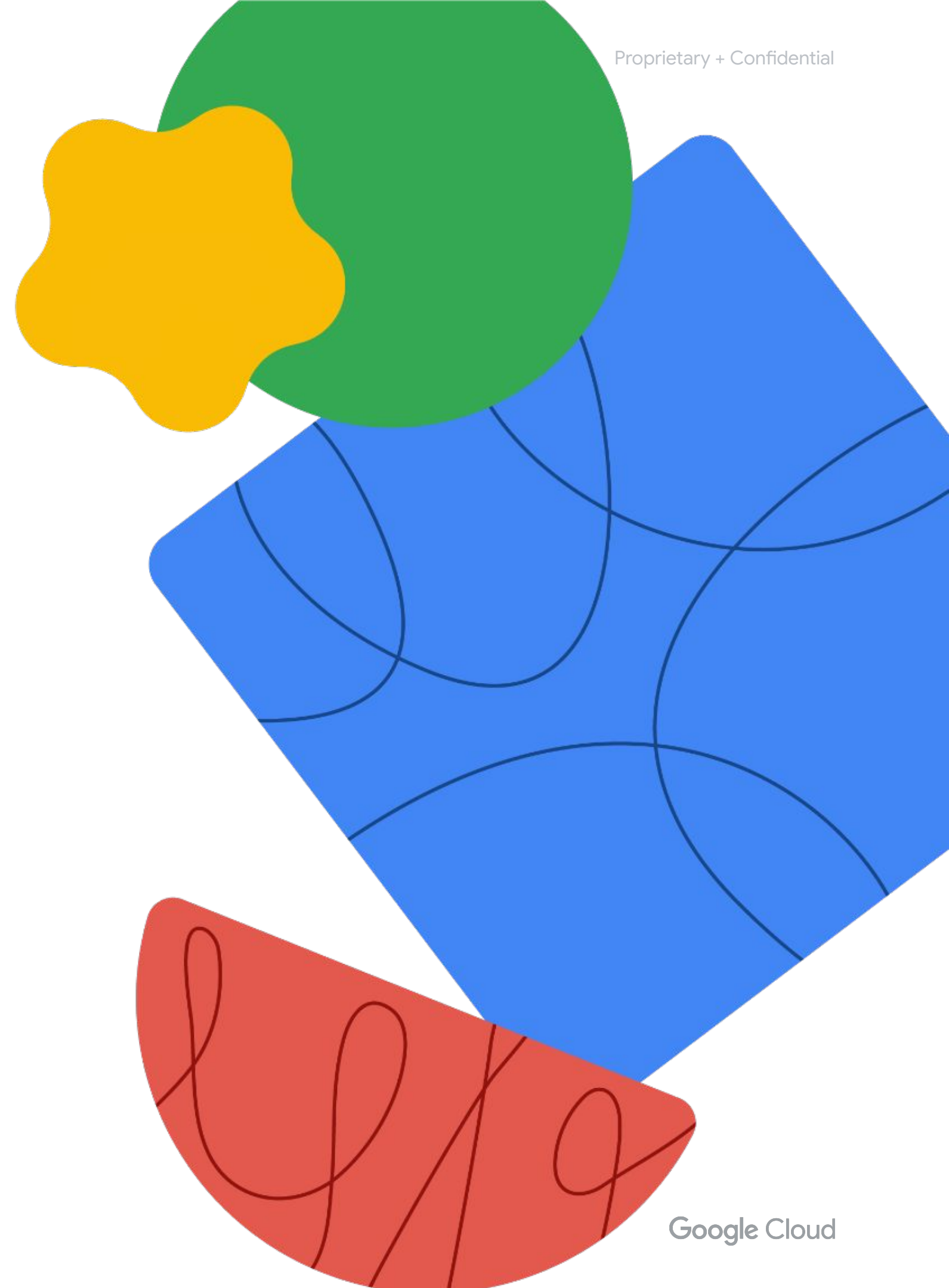


For Lakehouse tables in Cloud Storage, Sensitive Data Protection can apply **dynamic data masking**.



Data loss prevention

Demonstration



Scenario: Cymbal loyalty program

01

New customer information has been collected and loaded into a BigQuery table named `loyalty_program_customers`.

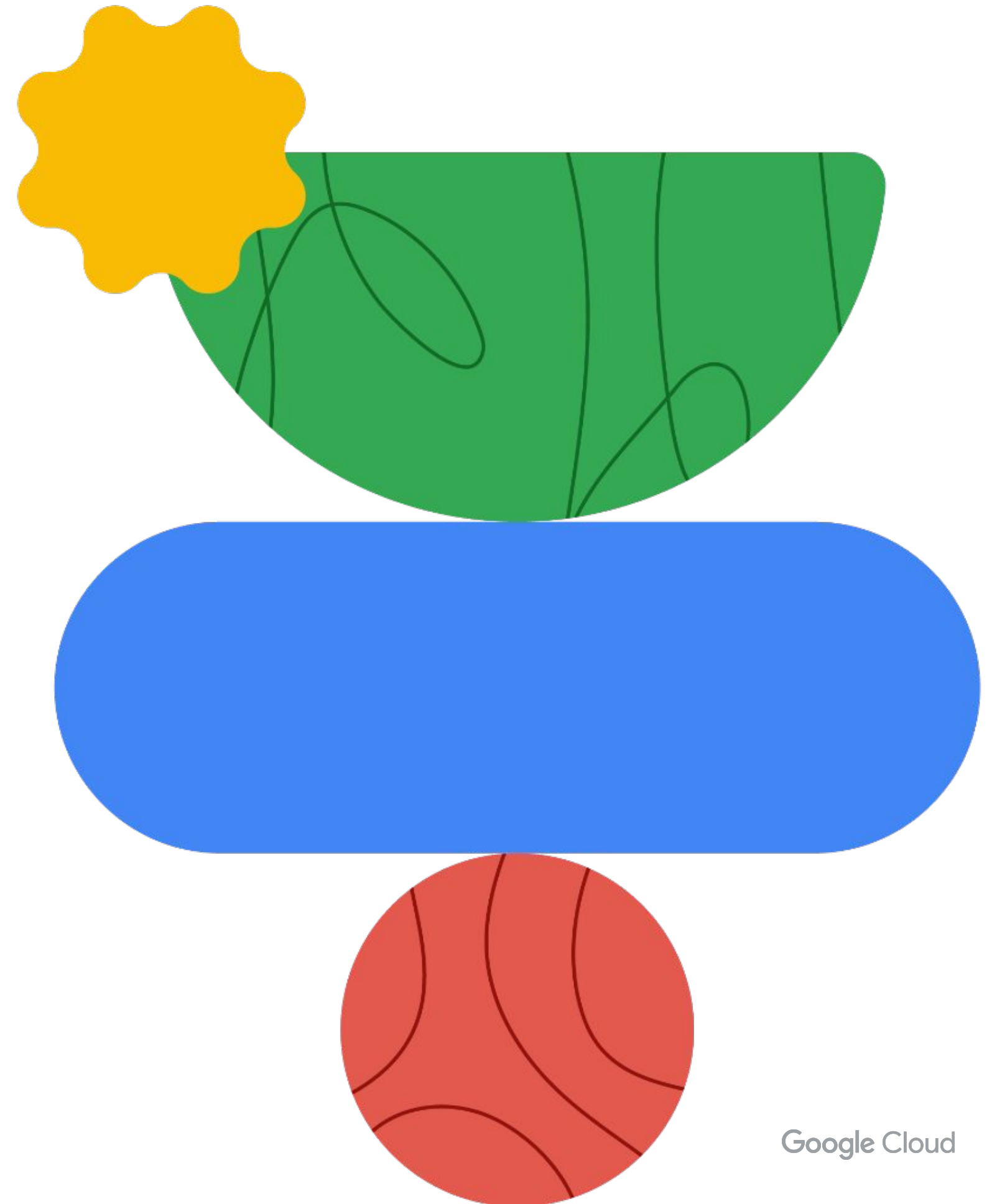
02

This table contains standard information, such as names and purchase histories.

03

Includes potentially sensitive data, such as email addresses, phone numbers, and free-text comments from customer feedback surveys.

Before this data is made available to the marketing analytics team, **any PII must be properly handled** to avoid exposing sensitive customer details in analytics dashboards.



Your architecture doc, in Google's deck

Hold the next six slides up next to the Iowa architecture doc — they line up zone for zone.

- Retire legacy extracts—the nightly SSIS extract jobs end; data lands in BigQuery continuously
- SQL Server DW to BigQuery—one governed copy becomes the analytics source of truth
- Bronze, Silver, Gold—your medallion lakehouse, in Google's own vocabulary
- Dataform ELT Silver to Gold—versioned, tested SQL pipelines replacing SSIS data flows
- Gold marts feed Power BI—the reports your users already have, repointed at BigQuery

Your design, in their deck.

The medallion architecture (1 of 3)

Bronze zone

This is the landing zone for all raw data.

Silver zone

In this zone, initial transformations happen.

Gold zone

Data is aggregated and optimized for analytics and reporting.

Cymbal: Bronze zone (raw data)

01

Clickstream data from their website.

02

Batch exports of transactional data from their e-commerce database.

03

JSON data from their social media marketing campaigns.

Cymbal: Silver zone (cleansed and conformed data)

01

Parsed clickstream data with structured sessions.

02

Augmented transactional data.

03

Standardized date and time formats.

Cymbal: Gold zone (curated business-level data)

01

Aggregated daily sales tables.

02

Customer 360-degree view tables.

03

Inventory performance summaries.

The medallion architecture (2 of 3)

Bronze zone

A historical record of what was received.

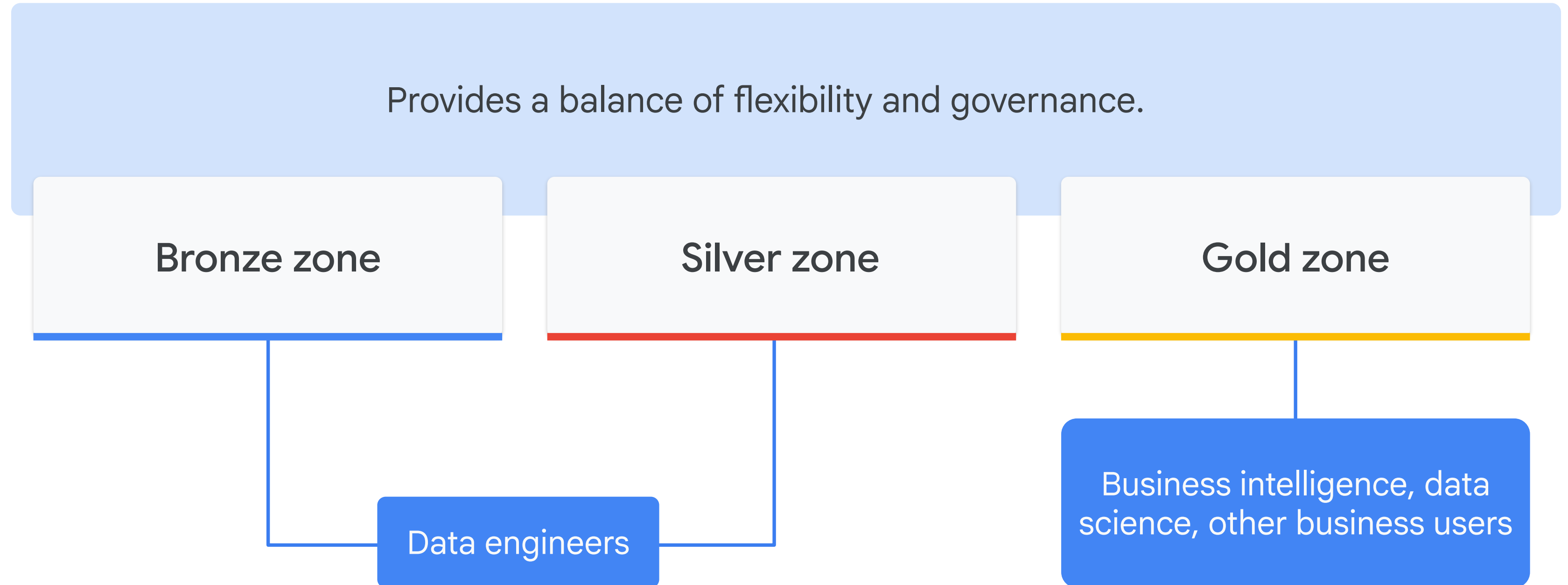
Silver zone

Data often stored in an open format in Cloud Storage but queryable with BigQuery.

Gold zone

Normally native BigQuery tables for maximum query performance.

The medallion architecture (3 of 3)



Where each of your zones lives in GCP

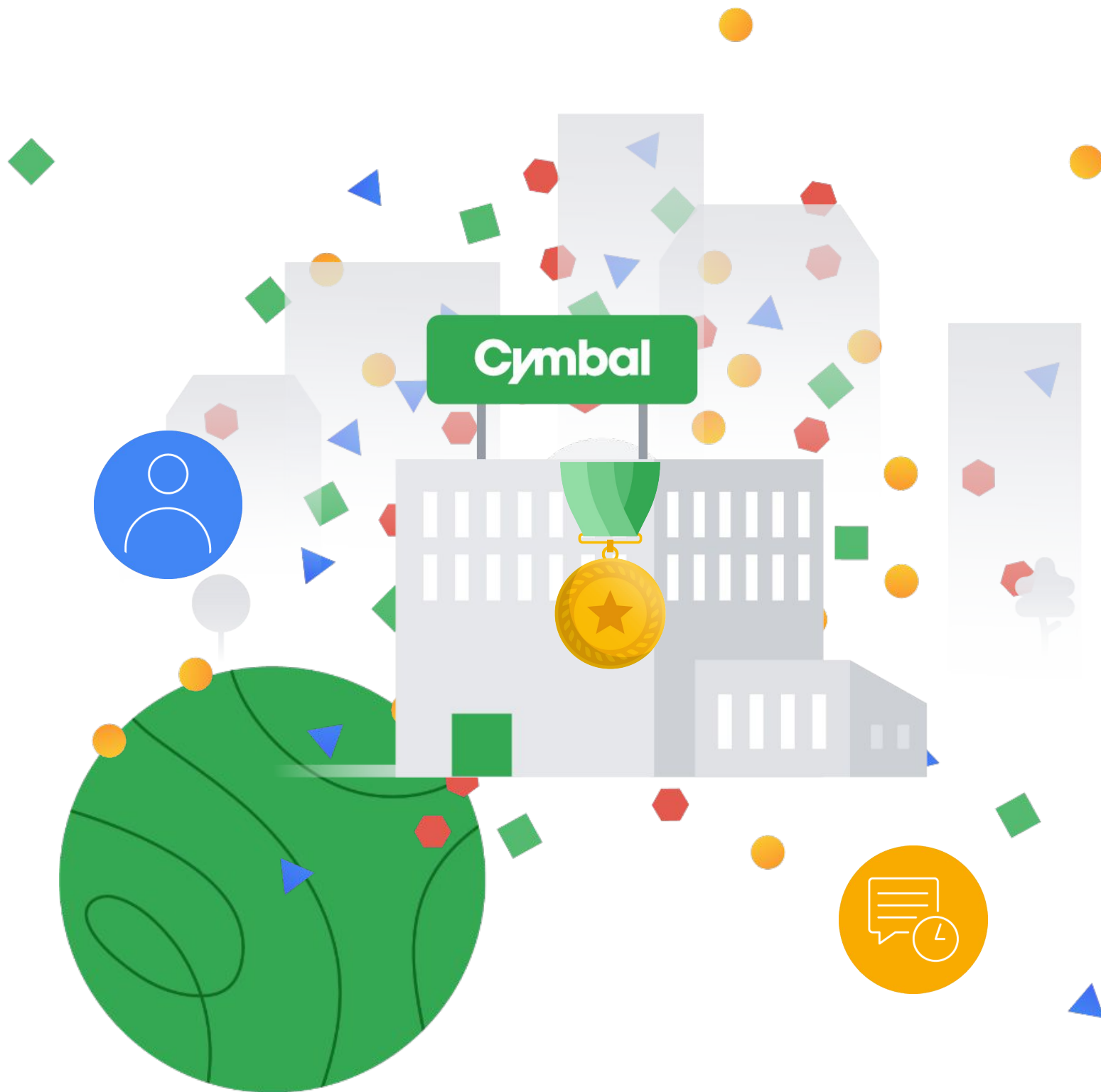
Same zones as your doc — here is the service behind each box.

- Bronze— raw landings on Cloud Storage as BigLake/Iceberg tables, queried in place (your 13:00 lab)
- Silver— cleansed, conformed tables built by Dataform incremental models, keys conformed across sources
- Gold— native BigQuery marts, partitioned and clustered with this morning's M9 levers
- Consumers— engineers work in Bronze and Silver; Power BI and business users read only Gold
- Trust— Bronze is received, Silver is verified, Gold is decision-grade

Cymbal migration strategies

For a company, such as Cymbal, which might be running on a traditional on-premises data warehouse (such as Teradata or Hadoop), migrating to a cloud-native data lakehouse is a significant project.

A complete, one-time migration is often too risky and disruptive. Instead, a phased, use-case-driven approach is usually more successful.



Migration strategy

- 01 Set up the core infrastructure on Google Cloud
- 02 Start with a high-impact use case
- 03 Migrate relevant data
- 04 Build the new pipelines and reports
- 05 Decommission and iterate



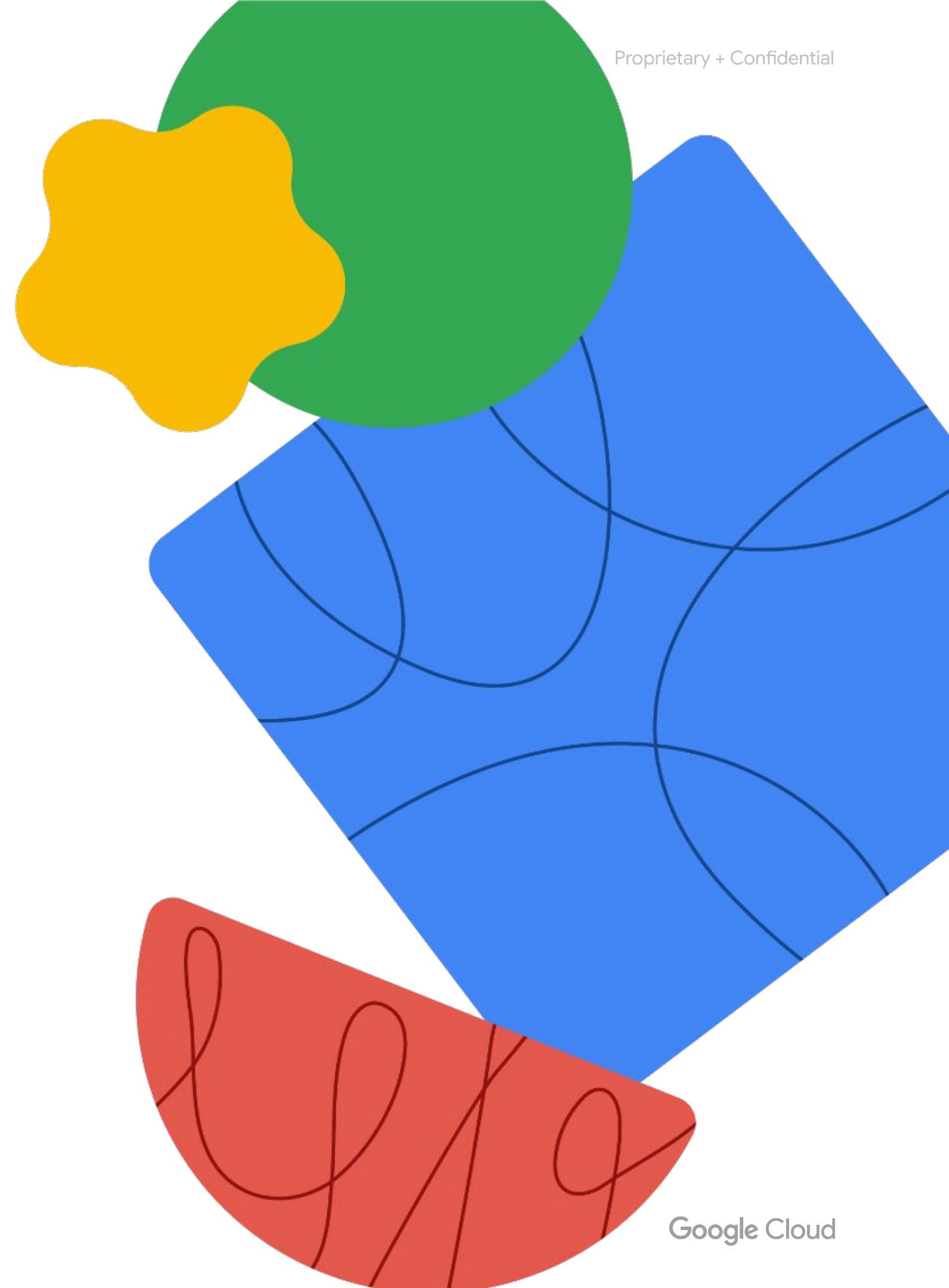
Their phased migration is your phased migration

A one-time move off the warehouse is too risky — phase it by use case.

- Stand up the core first— projects, datasets, IAM, billing guardrails
- Pick one painful extract chain— Bronze to Dataform Silver to Gold mart to the existing Power BI report
- Parallel-run, then cut over— old job and new pipeline side by side until the numbers match
- Decommission last— retire the extract only after the Gold mart feeds the report
- Repeat per use case— this afternoon's capstone starts that exercise on paper

Cost management and optimization best practices

A proactive approach to cost management.



Best practices (1 of 2)

01

Choose the right storage class

Not all data needs to be accessed with the same frequency. Consider storage class in Cloud Storage.

02

Optimize BigQuery queries

Use features, such as partitioning and clustering, as well as writing efficient SQL queries.

Best practices (2 of 2)

03

Use BigQuery's flat-rate pricing

Great for predictable workloads.

04

Set up budgets and alerts

In Google Cloud's billing tools.

M10 — What you now have

- ✓ Governance mapped to your single-source-of-truth requirement— catalog, PII protection, column and row security
- ✓ Your medallion design confirmed in Google's own teaching deck— Bronze, Silver, Gold, zone for zone
- ✓ A GCP home for each zone— GCS/BigLake Bronze, Dataform Silver-to-Gold, BigQuery Gold marts for Power BI
- ✓ A phased migration path and the cost levers— storage class, bytes scanned, budgets and alerts
- ✓ Next: M11— the BQML direction and the Power BI last mile on your Gold marts

Afternoon Break

15 minutes · 14:35–14:50

WATCH FIRST · 1:41 · GOOGLE CLOUD TECH

BigQuery ML in a Minute

- ✓ Train and run models with plain SQL — CREATE MODEL, no Python, no data movement, no new platform.
- ✓ Ninety seconds that reframes 'ML project' for a SQL team — then we prove it live with the three-statement demo.



Scan to open on YouTube

youtu.be/0RMT8uEplbM

M11 — What this block gets you

- ✓ BQML seen, not drilled— a model trained by CREATE MODEL, living next to your tables
- ✓ The end-to-end closed: extracts → BigQuery → Bronze/Silver/Gold → the Power BI you already have
- ✓ The Import vs DirectQuery call, per mart— default: Import + refresh aligned to Dataform releases
- ✓ Auth the right way— author OAuth on the desktop, a least-privilege service account in the service
- ✓ The gateway answer for M&O— BigQuery is a cloud source; nothing new to run

This block, and what follows

14:35–14:50

Break

14:50–15:05

BQML live: three SQL statements, one trained model — show it exists, don't drill

DEMO

15:05–15:20

The Power BI last mile: connect, Import vs DirectQuery, auth (custom segment)

15:20–15:50

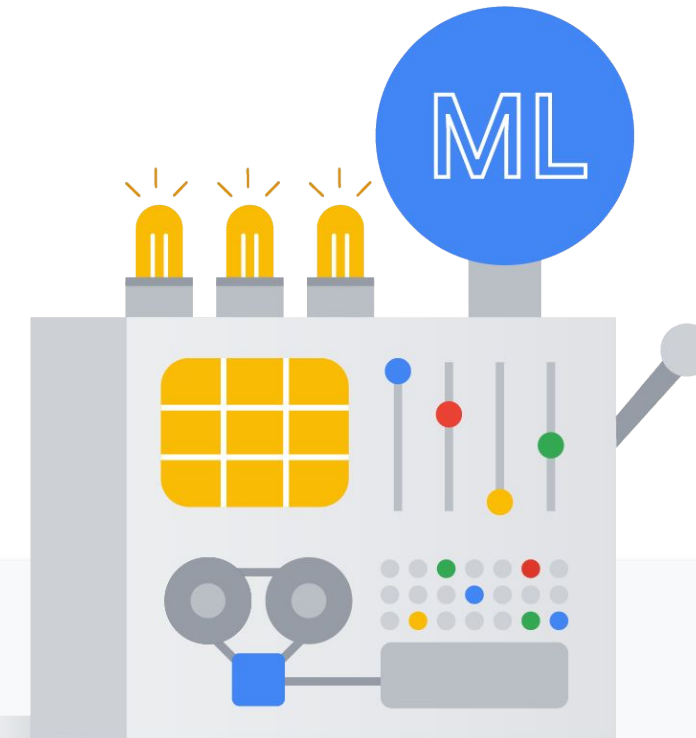
Capstone workshop: map one of your real extract jobs onto the GCP target

WORKSHOP

BigQuery ML



Build and deploy ML models using simple SQL queries.



Accessible ML

You do not need to be an expert in Python or TensorFlow to create valuable predictive models.

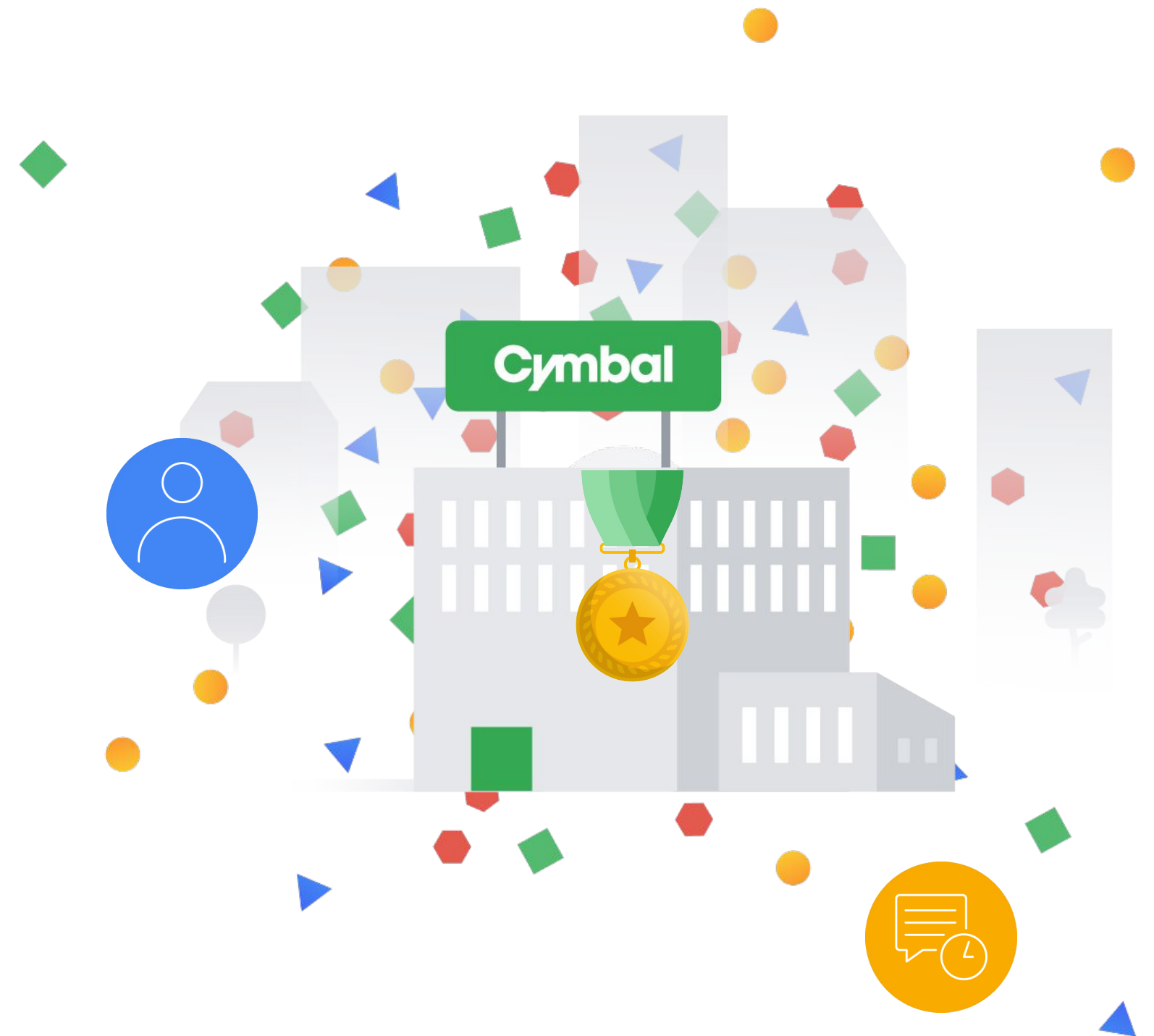
ML where the data lives

Same rule as everything since 9 AM: don't move the data to the tool.

- No export, no notebook— the model trains on data that never leaves BigQuery
- Models are first-class objects— they sit in your dataset, take IAM, appear in Information Schema
- ML.PREDICT is a function call— predictions are a SQL query, schedulable like any other
- Honest scope— ML is a possible future for DOM DoIT, not the priority; the warehouse is
- Why show it at all— when a data-science team arrives, your Gold marts are their training data

Use case

To identify customers who are at risk of not making another purchase, the marketing analytics team can use BigQuery ML and a few SQL statements.



Use case: Predicting customer churn (1 of 4)

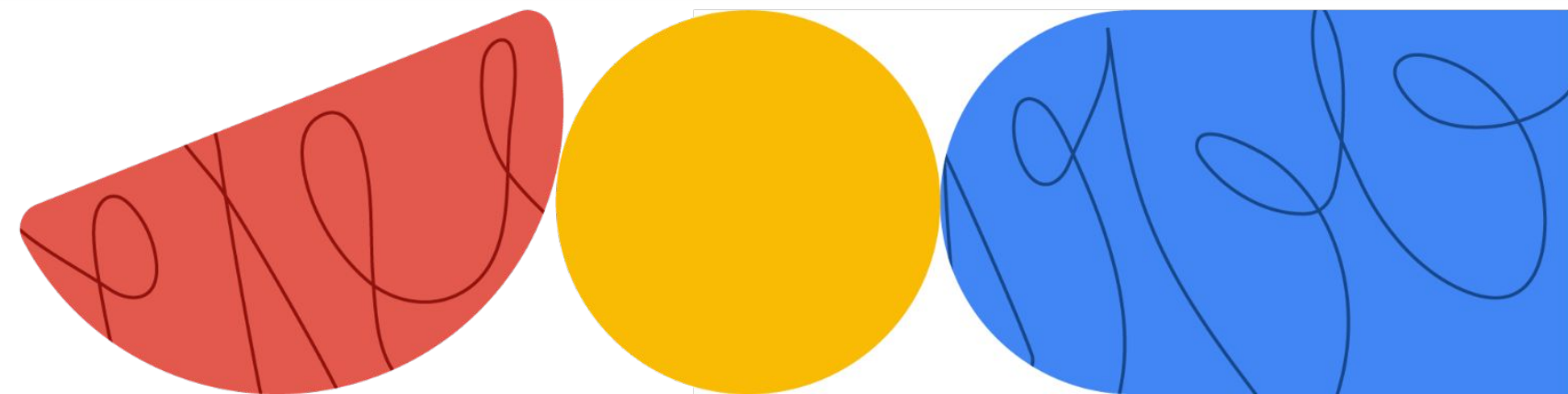
01

Feature engineering

The first step is to prepare the data. The analysts write a SQL query to create features (or signals) that might predict churn.

Fields

- `recency`: days since last purchase
- `frequency`: purchases in the last year
- `monetary_value`: total spent
- `days_since_first_purchase`: customer tenure



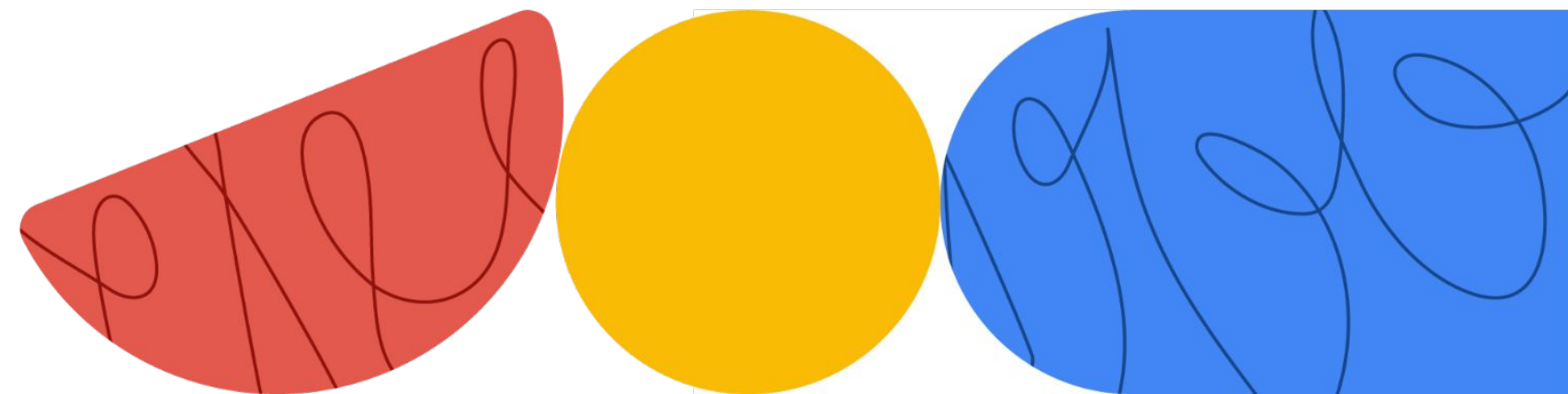
Use case: Predicting customer churn (2 of 4)

02

Model training

After the features are ready, they train a model with a single `CREATE MODEL` statement in SQL.

For this binary classification problem (churn or no churn), a logistic regression or a boosted tree model is a good choice.



Sample code

SQL

```
CREATE OR REPLACE MODEL cymbal_ecommerce.customer_churn_predictor
OPTIONS(model_type='LOGISTIC_REG') AS
SELECT
customer_id,
recency,
frequency,
monetary_value,
(total_purchases > 1) AS will_return -- This is our label
FROM
cymbal_ecommerce.customer_purchase_summary;
```

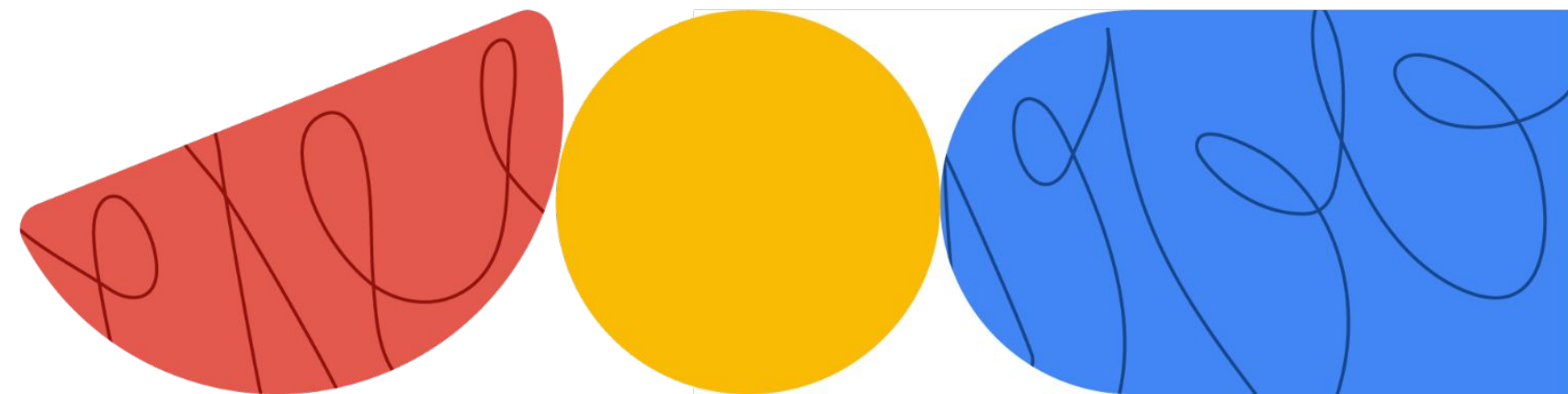
Use case: Predicting customer churn (3 of 4)

03

Model evaluation

After the model is trained, the analysts evaluate its performance using the `ML.EVALUATE` function.

- ✓ Accuracy
- ✓ Precision
- ✓ Recall



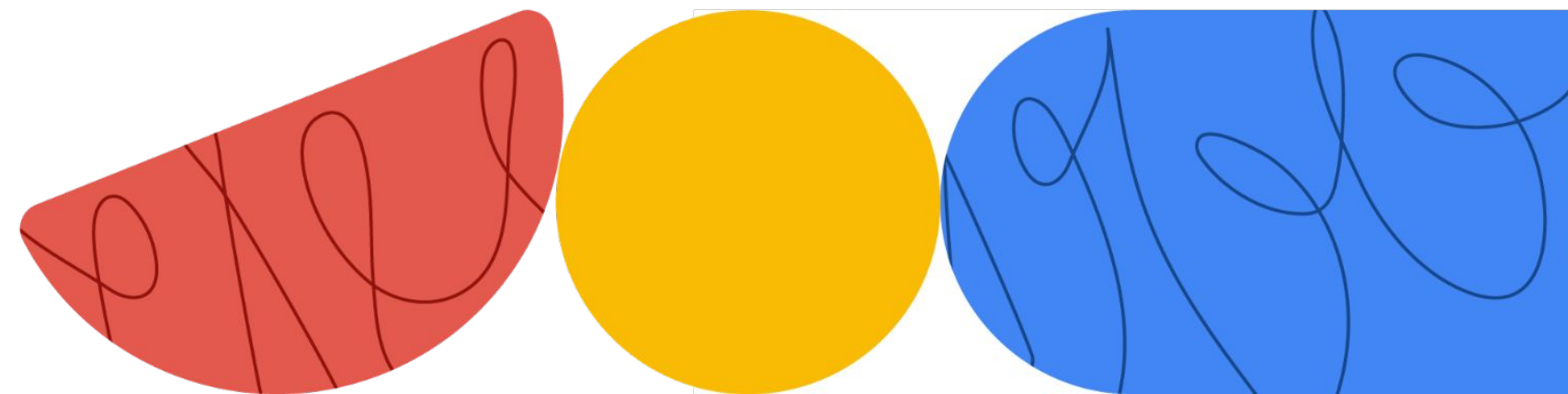
Use case: Predicting customer churn (4 of 4)

04

Prediction

With the `ML.PREDICT` function, you can get a list of all customers and their probability of churning.

This list can then be used to create targeted marketing campaigns to re-engage at-risk customers.



BigQuery ML allows the teams who are closest to the data to **build predictive models**, which leads to faster and more relevant insights.

Demo: a live BQML model in three SQL statements

15 minutes · BigQuery console — instructor's demo dataset

1. Peek at `bigquery-public-data.ml_datasets.penguins` — 344 penguins; we predict `body_mass_g`
2. `CREATE OR REPLACE MODEL class.penguins_model` — linear regression, trains in ~1–2 minutes
3. `ML.EVALUATE` — mean absolute error in plain speech: a unit test for a model
4. `ML.PREDICT` — actual vs predicted side by side, sorted by worst error
5. Point at the Explorer pane — the model sits in the dataset like a table, with IAM

Why it matters — nothing in your brief points at ML — this exists so you know the door is there: the day someone asks about forecasting, the data and the tool already live in the same place.

The last mile: Power BI ↔ BigQuery

Your reports don't change — their source does. The course never mentions this; your project lives on it.

- Get Data → Google BigQuery— the connector ships in Power BI Desktop; the same one Excel uses
- Sign in with your own Google account (OAuth)— BigQuery sees the person, not a shared login
- Navigator shows project → dataset → table— the console hierarchy, inside your existing BI tool
- IAM on the Gold dataset is the gatekeeper— row/column security applies per report author
- From Load onward it's the Power BI you know— DAX, visuals, publish; nothing report-side changes
- Bookmark Microsoft's connector docs page— the pattern is stable, the dialogs evolve

Import vs DirectQuery — decide per mart

The one choice in the connector dialog that matters.

- Import— data cached in Power BI; fastest reports, full DAX, one query per refresh
- Import fits— nightly-built Gold marts; Dataform rebuilds on a schedule, refresh right after
- Import cost— small and predictable; a post-build refresh matches pipeline reality
- DirectQuery— a live query per visual, per user, per click; current on every interaction
- DirectQuery cost— bills BigQuery scan bytes continuously; on-demand pricing needs watching
- DirectQuery fits— intra-day executive freshness, or a dataset too large to cache

Default: Import + scheduled refresh aligned to the Dataform release schedule. DirectQuery buys freshness the pipeline doesn't have yet — revisit per mart when streaming arrives.

Auth and gateways — two identities, one quiet gateway

Desktop and the Power BI service are different stories. Keep them distinct.

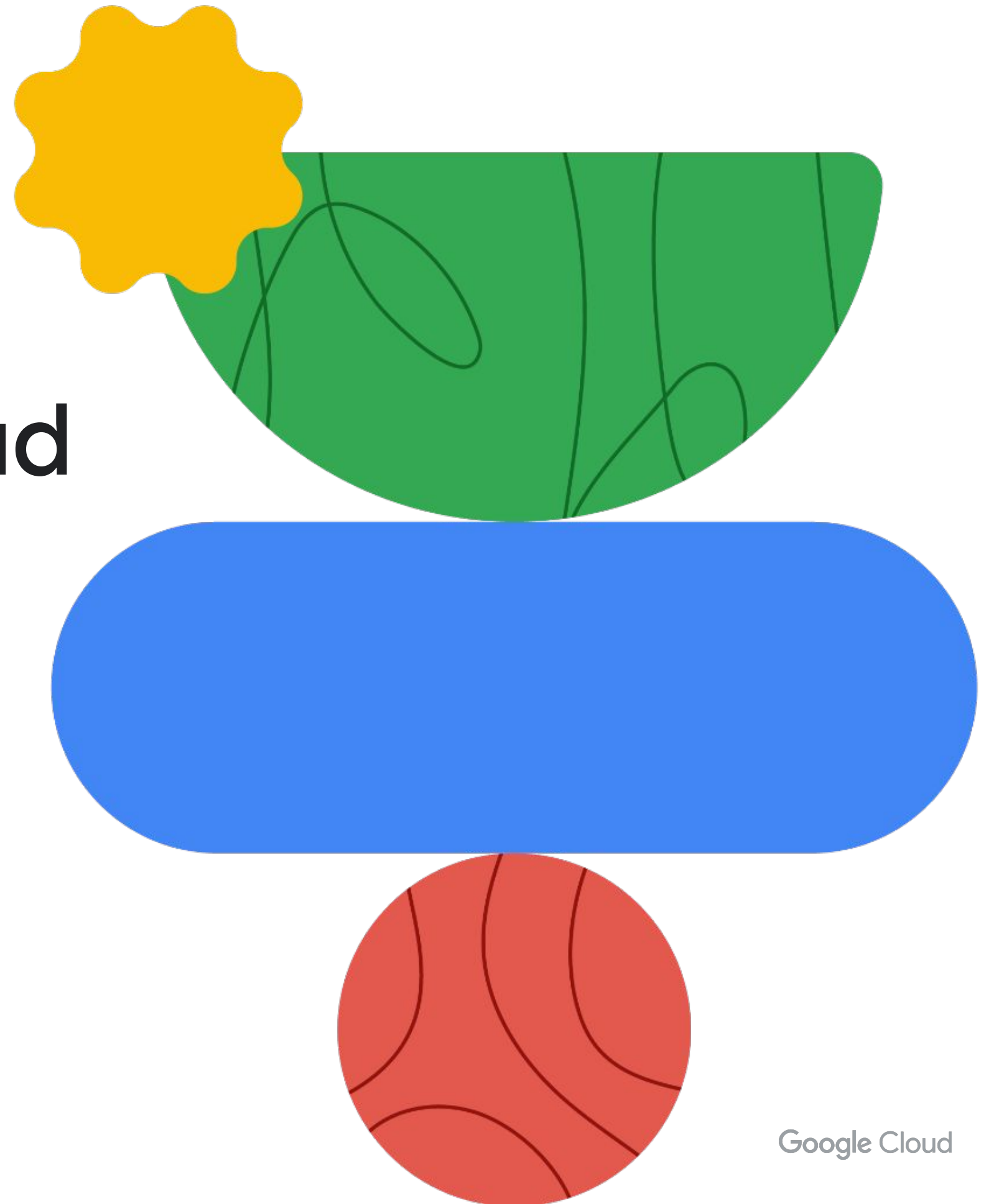
- Desktop— each author signs in with their own Google account; the audit trail is real people
- Service— stored credentials are a service account whose only job is reading the Gold dataset
- Minimum roles— bigquery.dataViewer on the Gold dataset, bigquery.jobUser on the project
- Same discipline as SQL Agent proxies— least-privilege, non-human identity, one job
- Gateway answer for M&O— BigQuery is a cloud source; no on-premises data gateway required
- Gateway shrinks, not grows— only if a report mashes BigQuery with on-prem SQL Server

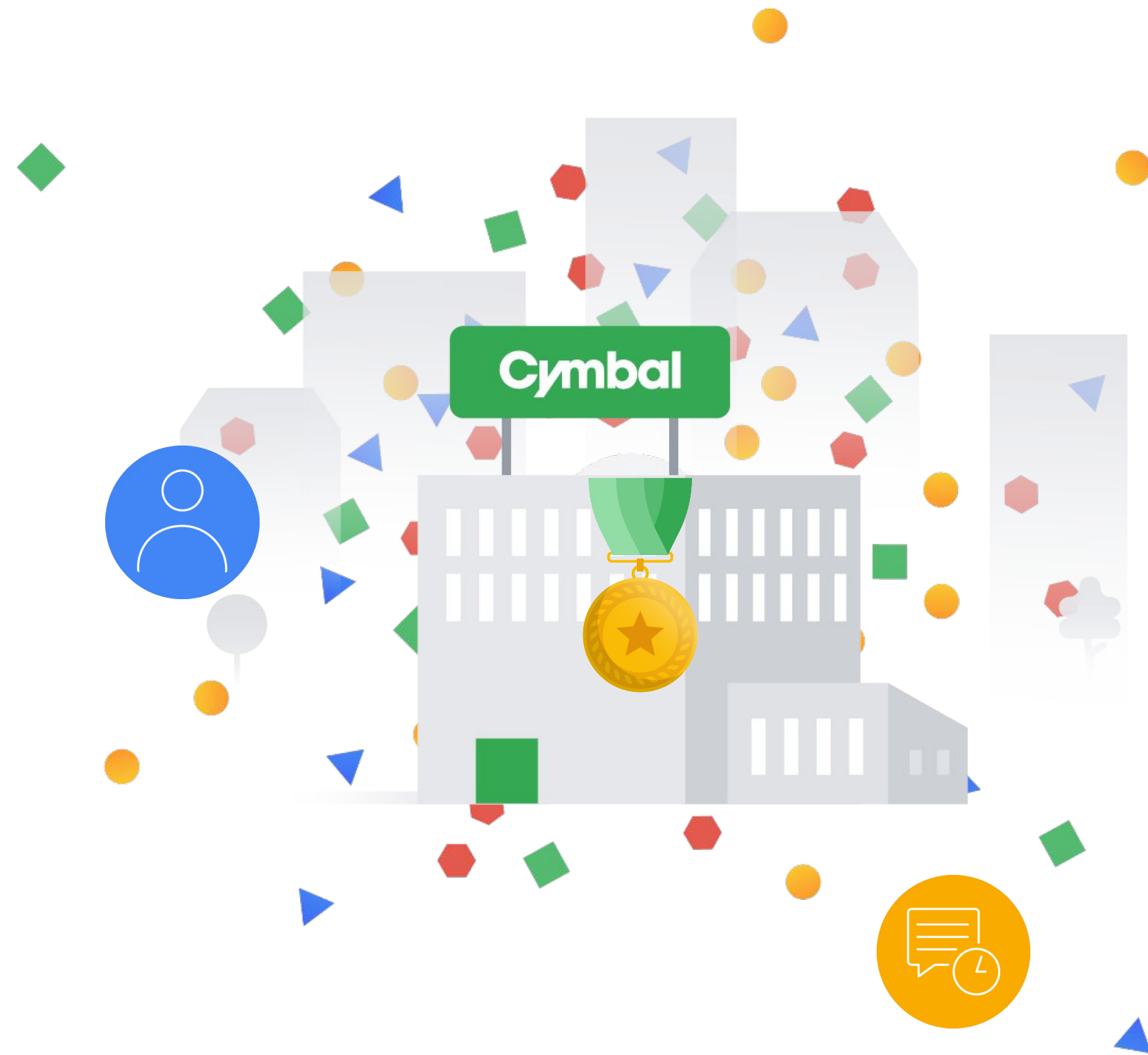
Your Gold marts land in the Power BI you already have. Same authors, same workspaces, same DAX — only the connection string changes.

The close of the Power BI segment — the whole migration, end to end

Building a modern data lakehouse on Google Cloud

Leveraging powerful,
open-source-based services.





Cymbal deals with a massive amount of data daily in various formats. A data lakehouse architecture helps bring it together for powerful analytics and AI-powered customer experiences.

Core components of a Google Cloud data lakehouse

A robust and scalable data lakehouse on Google Cloud is built on a foundation of several key services that work together seamlessly.



Cloud Storage

This is the heart of your data lakehouse, providing a highly scalable and cost-effective object storage service. Think of it as the foundational layer where all your data, in its raw and processed forms, resides.

Cymbal's data

- ✓ Website clickstream data
- ✓ Product images
- ✓ Supplier inventory files (CSV/Parquet)
- ✓ Historical data (Apache Iceberg)

BigQuery

This powerful, serverless data warehouse acts as the analytics engine for your data lakehouse. What makes BigQuery unique is its ability to directly query data in Cloud Storage without needing to move it.

The separation of storage and compute is a core principle. BigQuery provides:

- ✓ Flexibility
- ✓ Cost savings

Knowledge Catalog

As your data grows, managing it can become a significant challenge.

Knowledge Catalog provides an intelligent data fabric that allows you to discover, manage, monitor, govern, and describe your data across your entire data lakehouse.

Cymbal's data

- Knowledge Catalog automatically catalogs data in Cloud Storage buckets, making it easily searchable for data analysts and scientists.
- It enforces data quality rules and security policies, protecting sensitive customer information.

AlloyDB for PostgreSQL

Although the data lakehouse is excellent for analytical workloads, many applications still rely on transactional databases.

AlloyDB is a fully managed, PostgreSQL-compatible database service that offers superior performance and availability.

Cymbal's data

- AlloyDB for e-commerce backend handles real-time order processing and customer account management.
- Integrate AlloyDB data into data lakehouse for broader analytics via federated queries.

Apache Iceberg

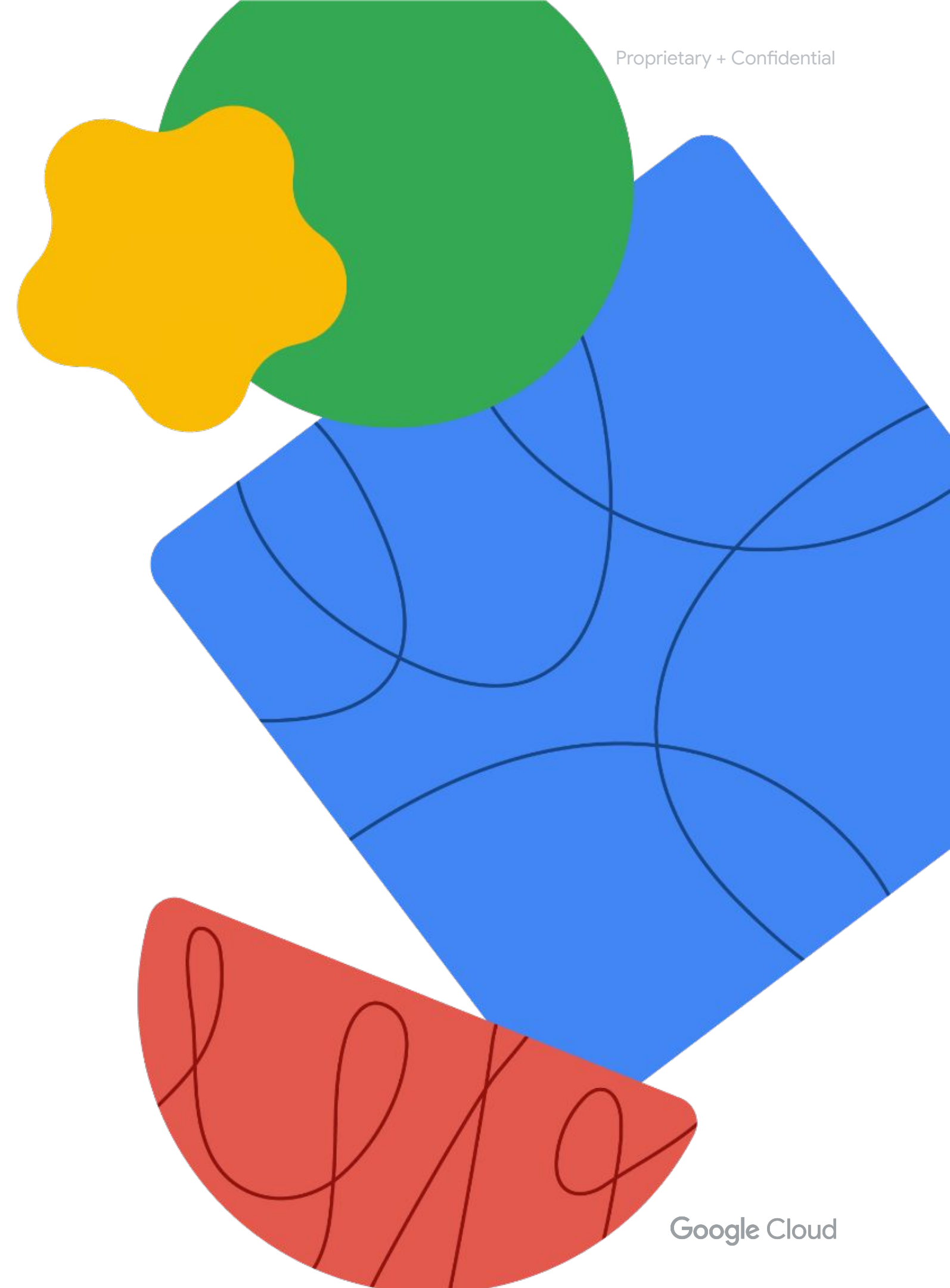
An open-source table format that brings the reliability and performance of a traditional database to your data lake.

Using Apache Iceberg with BigQuery and Cloud Storage, you can perform efficient updates and deletes on your data, a capability that was traditionally difficult in data lakes.

This is crucial for Cymbal when they need to update product information or manage customer data privacy requests.

Architectural patterns and best practices

The most common and effective architectural pattern for a data lakehouse is the medallion architecture, which organizes data into three distinct zones.



The medallion architecture

Bronze zone

This is where data lands in its original, untouched format.

Silver zone

The raw data is cleaned, standardized, and enriched.

Gold zone

Contains highly refined and aggregated data that is ready for business intelligence and analytics.

Best practices

01

Embrace open standards

02

Govern your data from the start

03

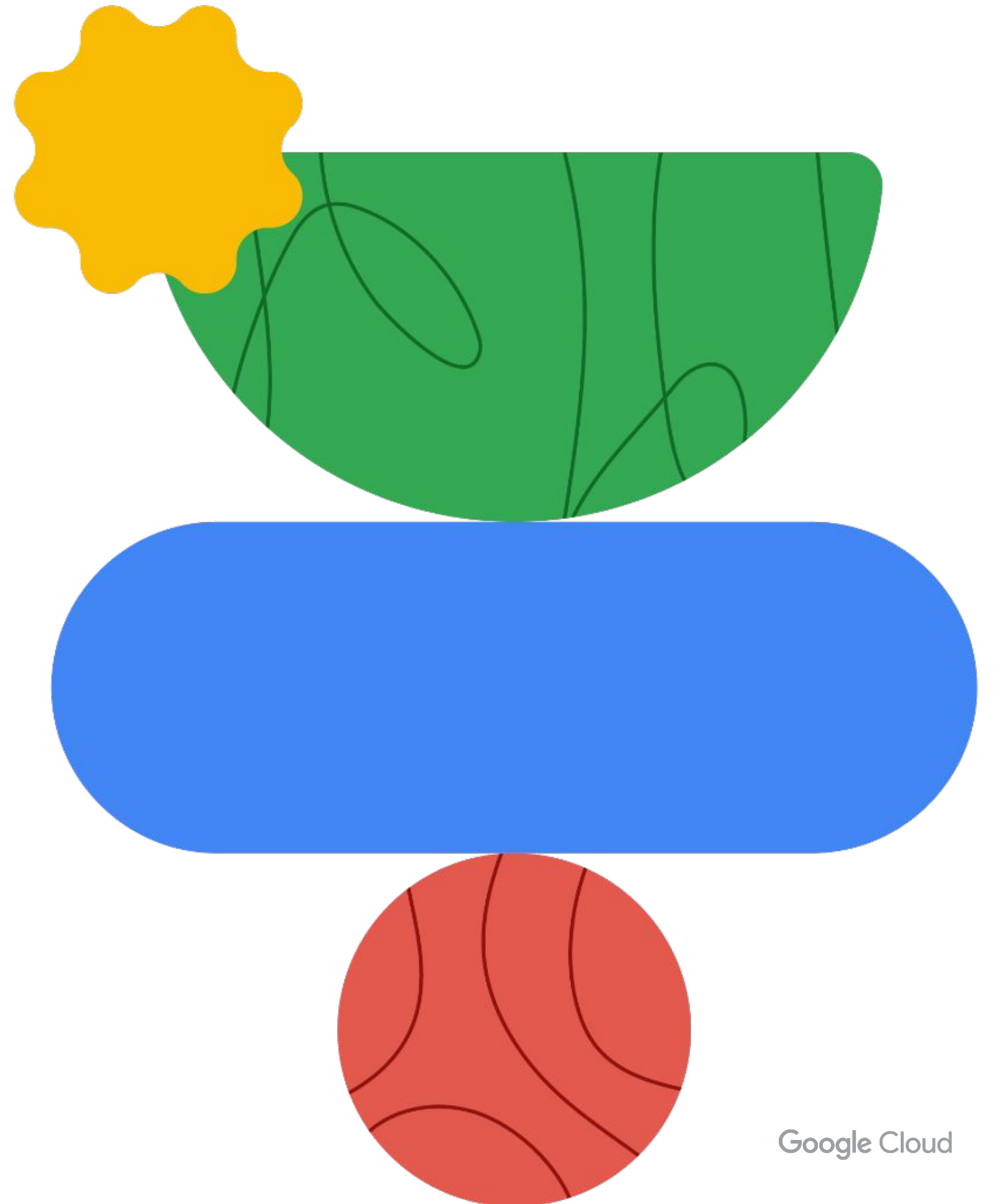
Optimize for cost and performance

04

Automate your data pipelines

Common pitfalls to avoid

Implementing a data lakehouse is a journey, and there are some common challenges to be aware of.



Common pitfalls to avoid

01 Neglecting governance

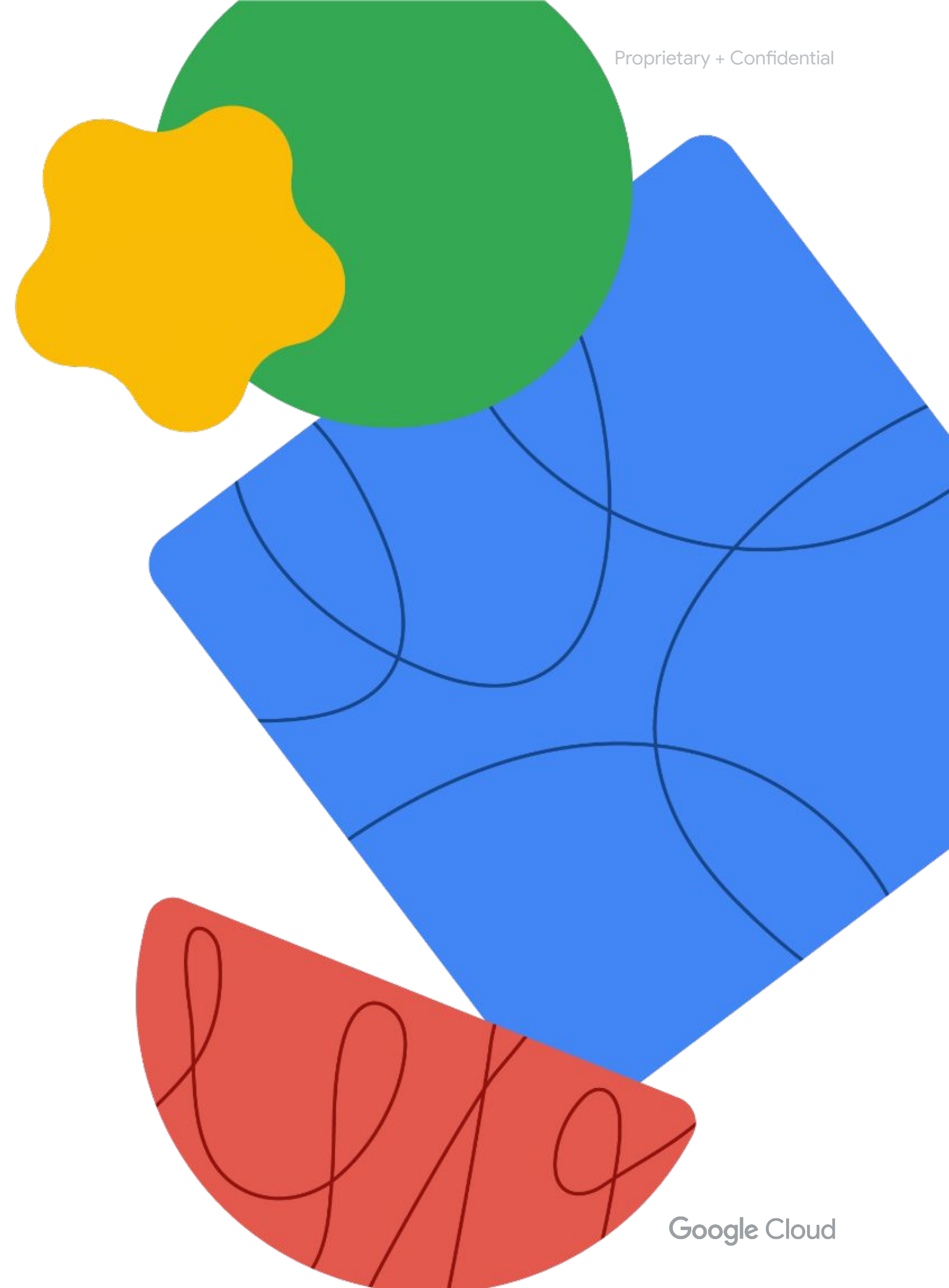
02 Ignoring data quality

03 Vendor lock-in

04 Lack of clear strategy

The future: AI and data modernization

The true power of a modern data platform lies in its ability to support advanced analytics and AI.



AI and data modernization (1 of 2)

01

Advanced analytics can be done quickly using BigQuery ML.

02

Services, such as Agent Platform, can be used to build and train ML models directly on the data in your data lakehouse.

03

Cymbal could develop a recommendation engine for product suggestions or a fraud detection model for suspicious transactions.

AI and data modernization (2 of 2)

The rise of generative AI also opens up exciting new possibilities. Cymbal could use large language models to create engaging product descriptions automatically or to power a chatbot that provides personalized customer support.



M11 — What you now have

- ✓ BQML seen, not drilled— CREATE MODEL → ML.PREDICT in SQL, where your data already lives
- ✓ The end-to-end closed— kill the extracts → BigQuery → Bronze/Silver/Gold → Gold marts → Power BI
- ✓ A default for the marts— Import + scheduled refresh aligned to your Dataform release schedule
- ✓ An auth pattern— author OAuth on the desktop, least-privilege service account in the service
- ✓ Next: the capstone— map one of your real extract jobs onto everything you've just seen

Capstone — map one of YOUR real extract jobs

30 minutes — 15:20–15:50 (20 build + 10 share-out) · Groups of 3–4 — the filled map is the deliverable

1. Pick one real, typical extract job — the one that pages someone when it fails; keep it to one
2. Fill the map left to right: job → source system → ingest tool → Bronze → Silver → Gold
3. ... then orchestration & CI/CD → Power BI serving → first thing that breaks (be honest)
4. Aids: database → Datastream or batch; files → Storage Transfer + LOAD DATA, pinned schema
5. Defaults: Dataform owns Silver→Gold in git; Scheduler stays boring; Power BI Import-first
6. Share-out: a box you can't fill is the most valuable thing to raise — then photograph your map

Why it matters — this page is backlog draft zero for your migration — the artifact you take back to work.

Closing the course — what this last hour gets you

- ✓ Capstone first— your group maps one real DOM DoIT extract job onto the GCP target architecture
- ✓ Then the fast retrace— two days against your migration map; recall, not new content
- ✓ One honest look at what changes Monday— your worksheet is backlog draft zero
- ✓ Where to go next— the 3-day Data Warehousing with BigQuery course and free practice paths
- ✓ Open questions and evaluations— the last 20 minutes are yours

The last 70 minutes

15:20–15:50

Capstone workshop — groups of 3–4 map one real extract job: source → ingest → Bronze → Silver/Gold → Power BI

WORKSHOP

15:50–16:10

Course summary — the two days retraced against your migration map, one more time, fast

16:10–16:30

Flex / open questions / evaluations — plus what comes next (the 3-day BigQuery data warehousing course)

Capstone: map one of your real extract jobs

30 minutes — 20 build + 10 share-out · Groups of 3–4 · whiteboard or paper · worksheet template in your pack

1. Pick ONE real extract job your team owns — the most typical one, not the biggest or simplest
2. Fill the template left to right: job → source → ingest → Bronze → Silver → Gold → orchestration → serving
3. Decide: one tool per box; being stuck on a box is the valuable part — flag it for share-out
4. Answer 'first thing that breaks' honestly — schema drift, PII, 10× volume, a late county file
5. Share-out: 2–2.5 min per group — read the job aloud, then walk the map left to right

Why it matters — Single-org room, so real systems are safe to name. This page is the artifact you take back to work — backlog draft zero for the migration project.

The template: one real job, left to right

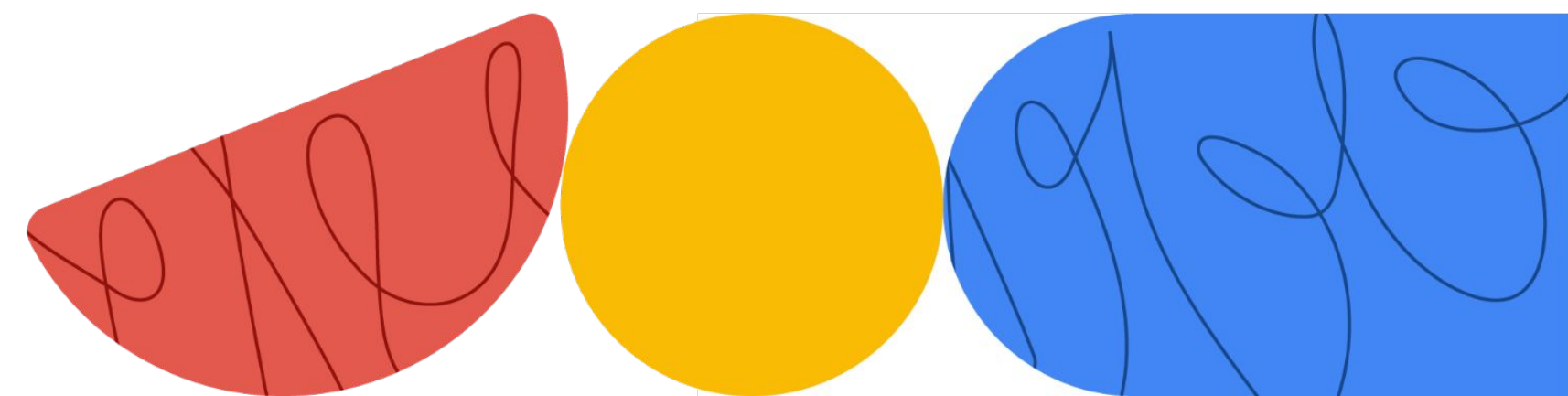
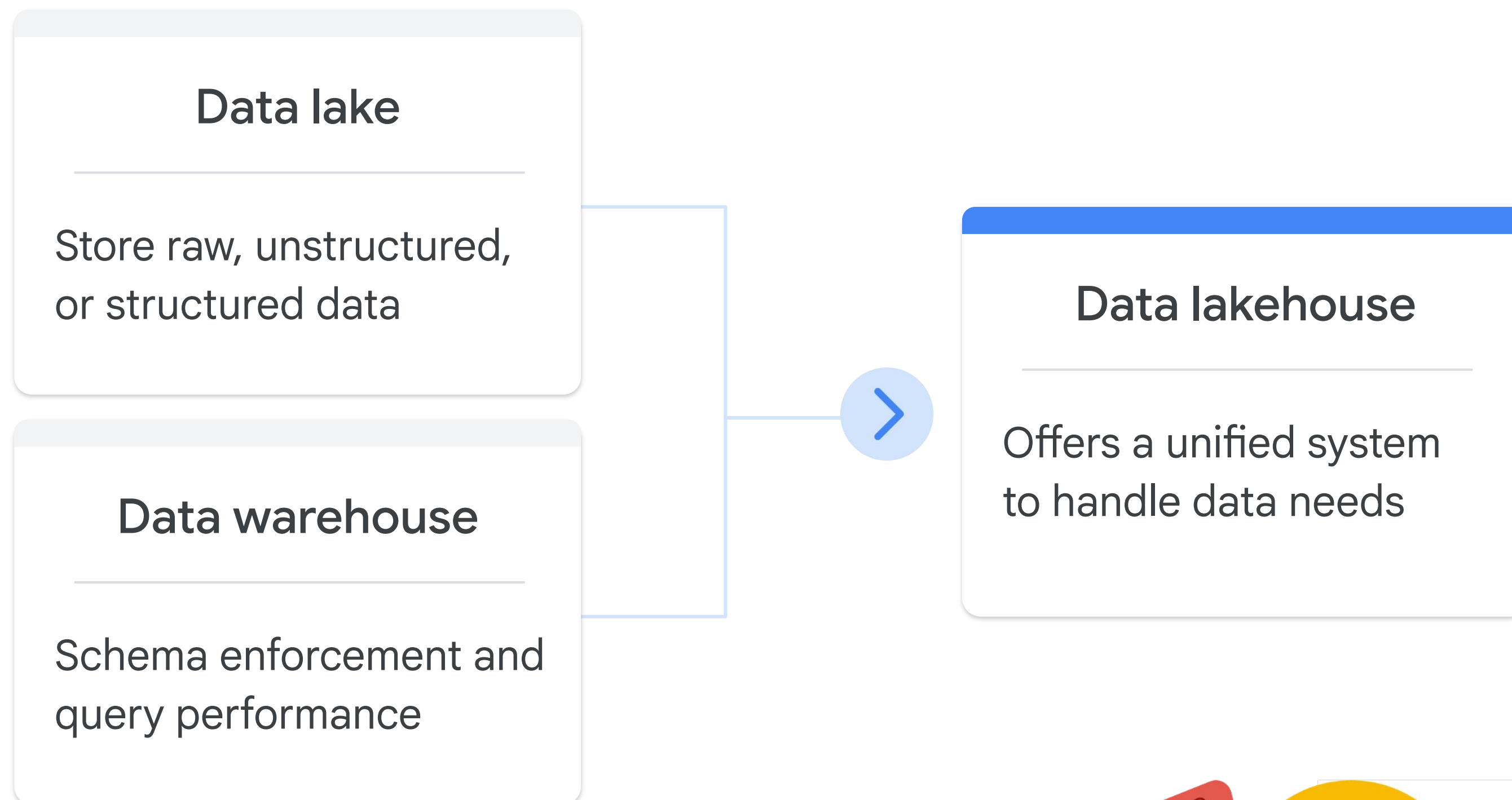
Every box has an answer from the last two days.

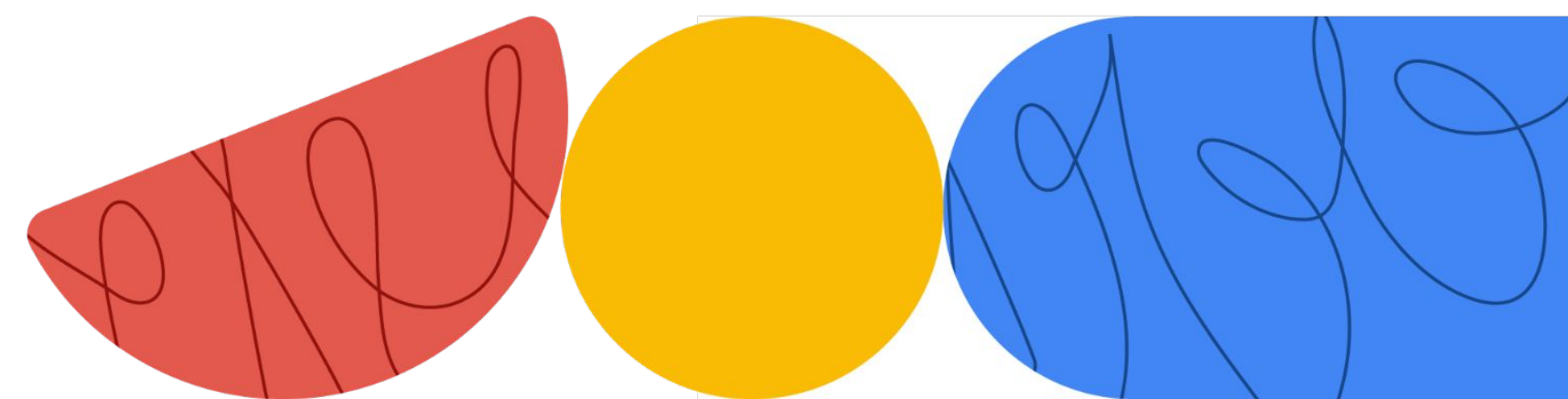
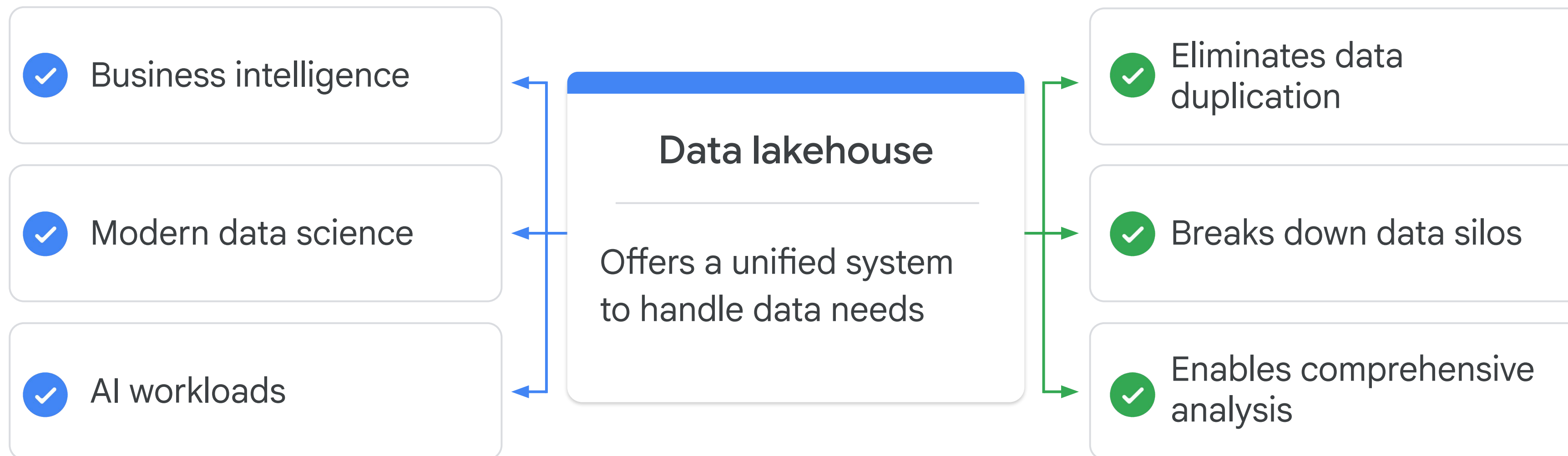
- The job— name it: one real SQL Server extract you own or babysit
- Source system— tables or files? Size? Change rate? PII?
- Ingest— Datastream CDC for databases; Storage Transfer + LOAD DATA for files
- Bronze— raw and replayable: GCS files, BigLake, or Iceberg; no cleaning here
- Silver→Gold— Dataform models dedupe and conform, then marts partitioned + clustered
- Serve— release configs + Scheduler; Power BI Import-first, refresh after the release

Stuck on ingest? Database → Datastream; files → Storage Transfer + LOAD DATA. Nightly tolerance → batch; always-current → CDC.

What a good map looks like — debrief

- Every box decided— one tool per cell; an 'or' means the decision got deferred — say so
- Bronze stays raw— it answers 'what did the source say'; Silver answers 'what does it mean'
- Dataform owns Silver→Gold, in git— transforms anywhere else need a stated reason
- Orchestration is boring— Scheduler + release configs; Composer when pipelines multiply
- Power BI Import-first— DirectQuery only with a stated freshness need
- 'First thing that breaks' is honest— those answers start your migration risk register



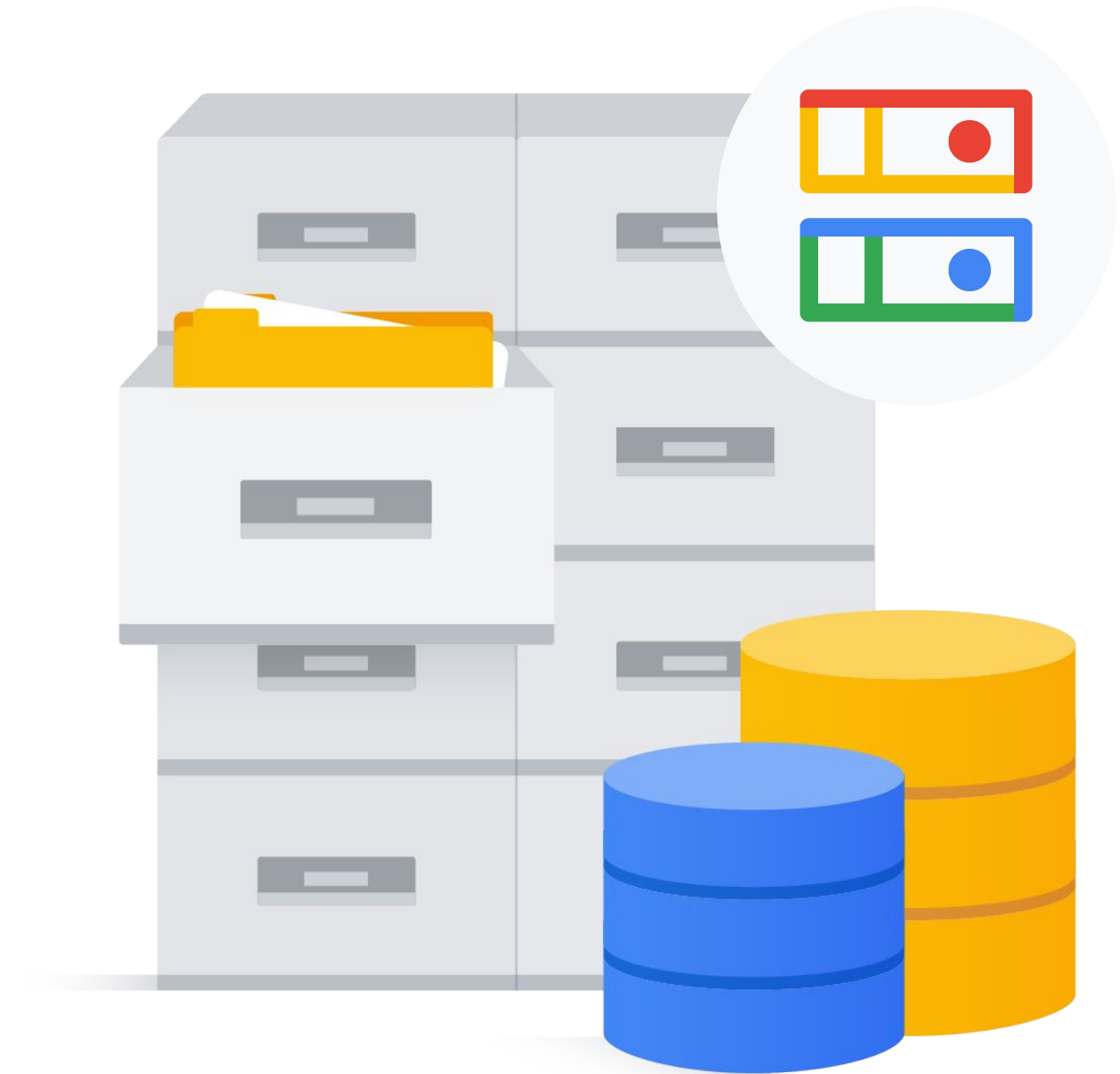


How Google Cloud services come together
to **form a modern, efficient, and scalable
data architecture.**

Cloud Storage

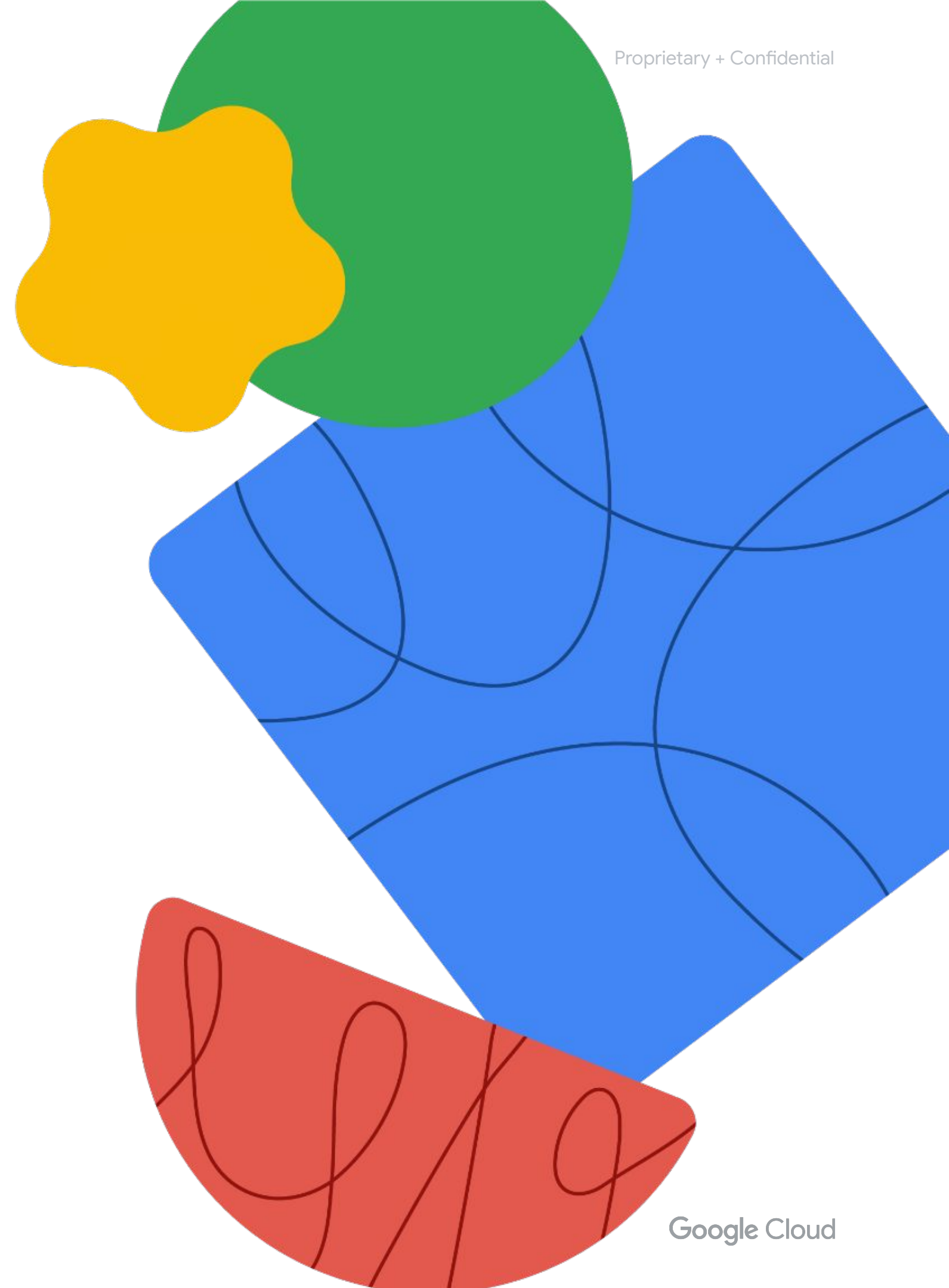
The foundational, cost-effective object storage for all raw data, including

- ✓ Structured
- ✓ Semistructured
- ✓ Unstructured
- ✓ Multimodal



Apache Iceberg

A data lakehouse needs structure and performance. Open table formats, such as Apache Iceberg, provide this.



What does Apache Iceberg provide?

01

Schema evolution

Allows schema changes (such as adding/renaming columns) without impacting existing data or applications.

02

Hidden partitioning

Automatically manages how data is partitioned for efficient querying.

03

Time travel

Enables querying past data versions for auditing and debugging, allowing users to reproduce exact data states from previous points in time.

04

Atomic transactions

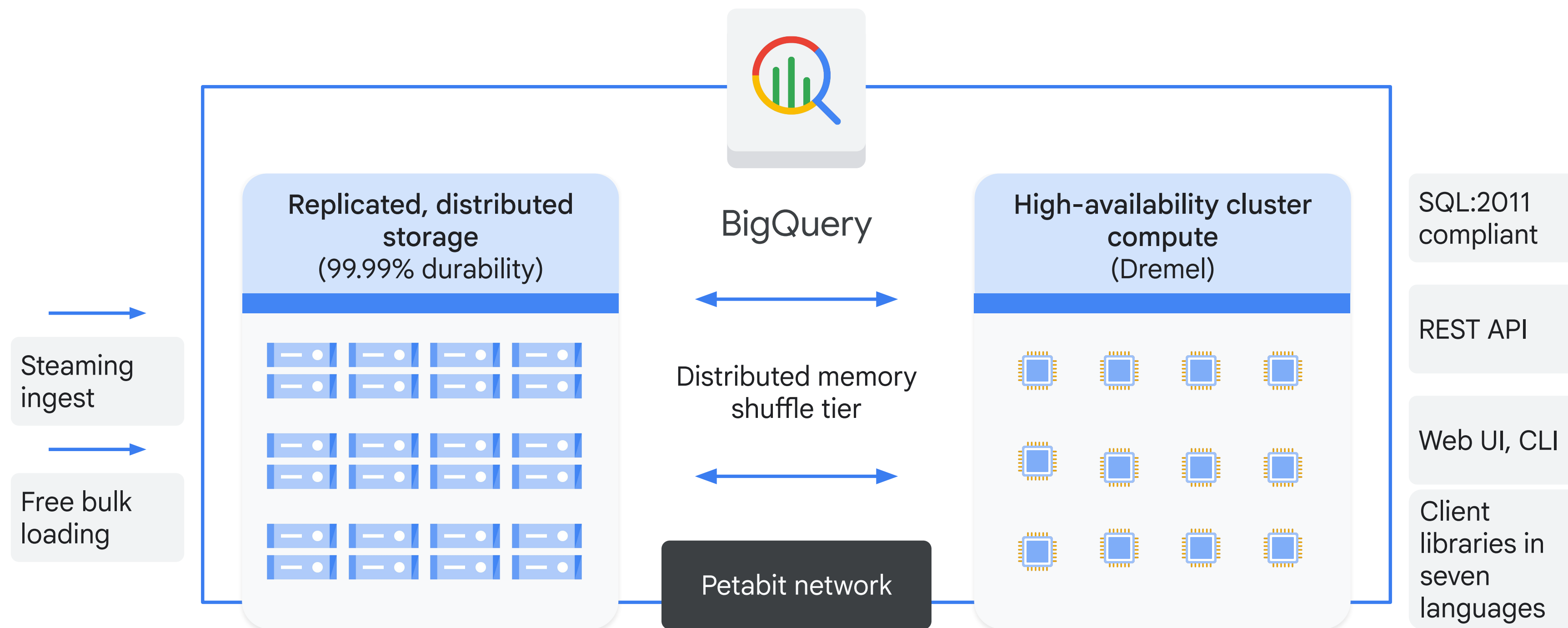
Enable reliable, concurrent data operations, preventing corruption when multiple processes read/write to the same tables.

BigQuery

Enterprise-grade, fully managed,
serverless data warehouse.



BigQuery fundamentals (1 of 2)



BigQuery fundamentals (2 of 2)

Through Lakehouse technology, BigQuery can directly query open-format data (such as Apache Iceberg tables) in Cloud Storage or other clouds, applying BigQuery's powerful optimizations, such as partitioning and clustering, by leveraging Apache Iceberg's metadata.



Lakehouse

01

Allows you to create Lakehouse tables over data in object stores (such as Cloud Storage).

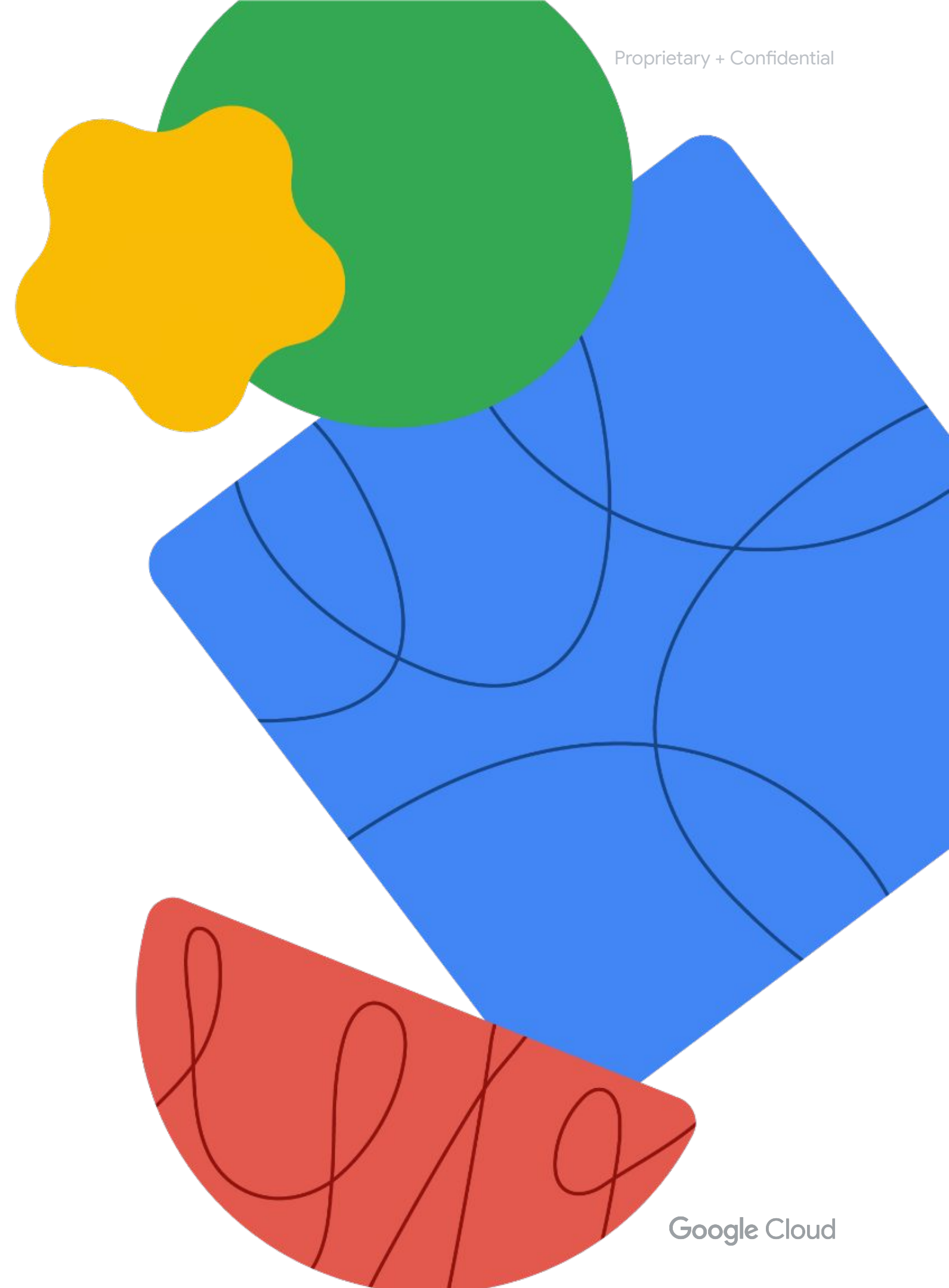
02

External tables reference data lake files, enabling unified governance and performance without duplicating data storage.



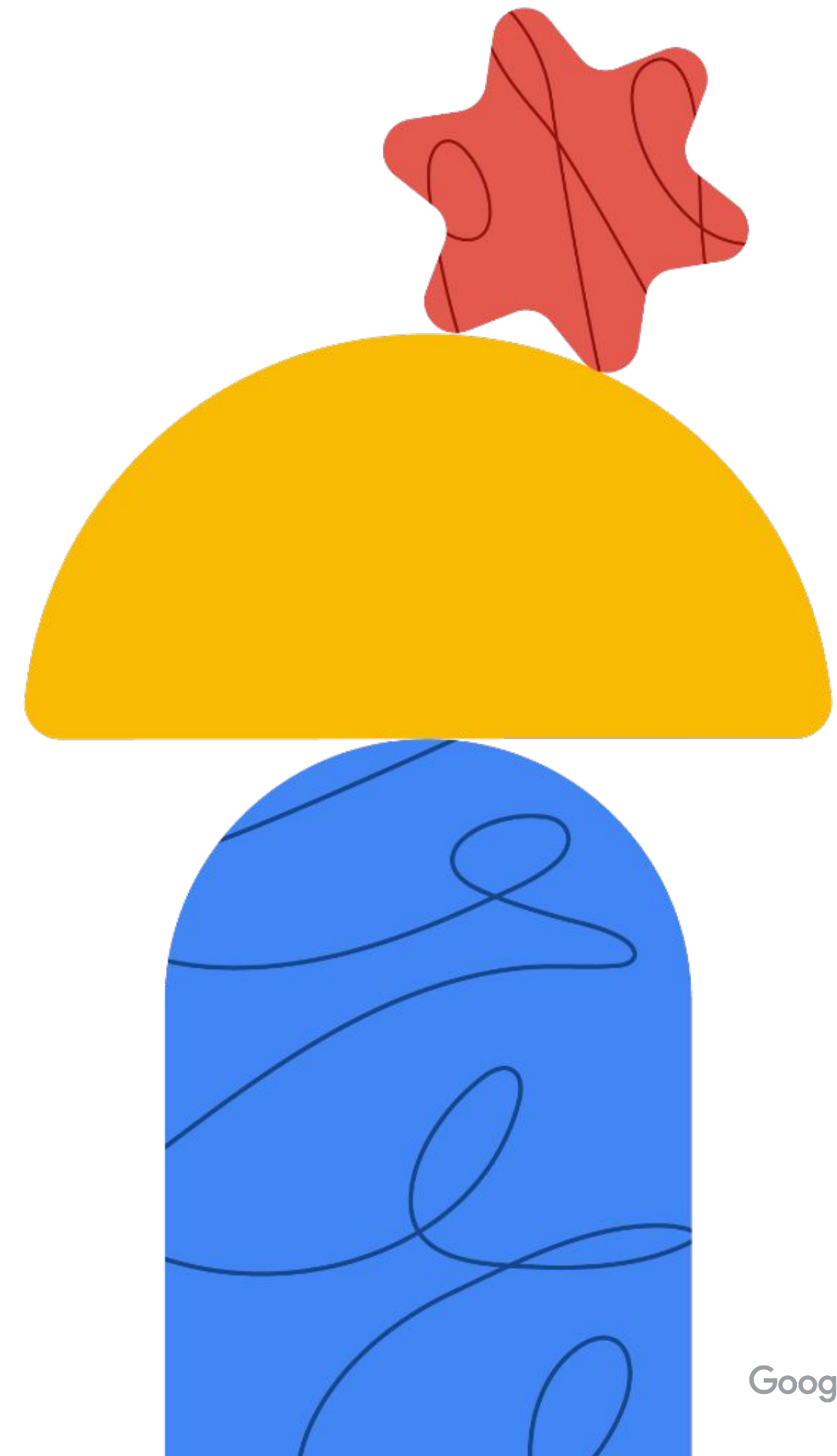
Knowledge Catalog

Unified data fabric and centralized metadata hub for discovering, managing, monitoring, and governing data across the data lakehouse.



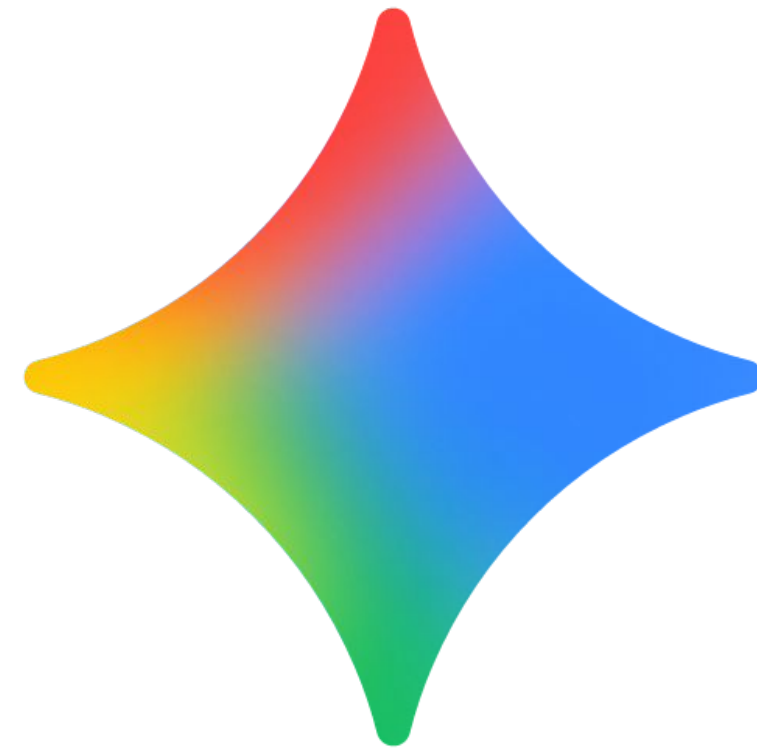
Sensitive Data Protection

Automates the discovery, classification, and protection of sensitive data using techniques, such as masking, tokenization, or redaction.



BigQuery ML and Agent Platform

Enable advanced analytics and ML
directly on data lakehouse data.



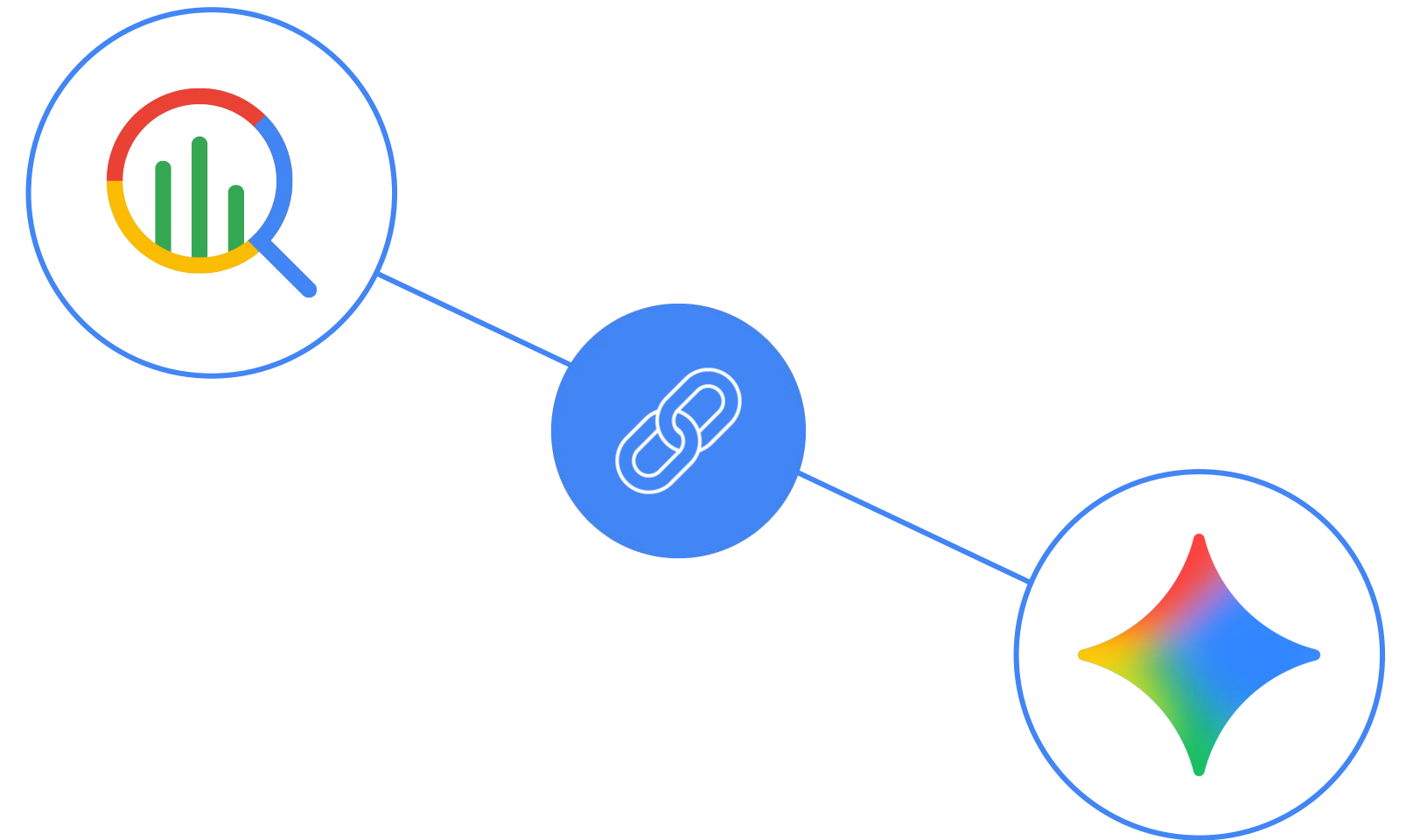
BigQuery ML and Agent Platform

01

BigQuery ML allows data analysts to build and deploy ML models using familiar SQL queries.

02

Agent Platform offers a comprehensive platform for complex custom models and MLOps, fully integrated with BigQuery.





Course summary

The two days, end to end

- ✓ Day 1 AM— console, projects, IAM, your class dataset; your first BigQuery load in a new console
- ✓ Day 1 PM— Datastream CDC killed the nightly extract; three load paths; pin schemas; CSV vs Parquet
- ✓ Day 1 close— Dataform owns Silver→Gold in git; Scheduler + release configs = SQL Agent, translated
- ✓ Day 2— lakehouse + Iceberg Bronze; governance and the medallion matched your architecture doc
- ✓ The spine— kill the extracts → land in BigQuery → Bronze/Silver/Gold → Dataform → Power BI

Where you go from here

- Data Warehousing with BigQuery— 3 days, being scheduled; goes deep on everything Day 2 previewed
- Your capstone map— photograph it tonight; it's backlog draft zero for the migration project
- Reference cards in your student pack— keep-forever desk aids, incl. the Power BI connector guide
- Practice free— 1 TiB/month query free tier, bigquery-public-data, the DIN page, dataform-demo repo
- Optional homework— the serverless Spark lab from Day 1, still waiting in your Skills classroom
- Evaluations— two minutes before you leave; they shape what we schedule for you next

Thank you for attending this training!

We love your feedback! Please take a minute to complete the survey and help us improve our courses.



Google Cloud